
THE FORMAL CLOSURE OF COMPUTATIONAL SEPARATION: $P \neq NP$ CLOSED ON EXISTENCE ITSELF, READ ON COMPUTATION

The Root and the Step to the Separation Proved Unconditionally on No Axiom; Least Escape the One Act; Every Barrier Typed by Theorem

COMPUTATIONAL COMPLEXITY

One Posit Carries the Whole Separation

MOHAMMAD F. ISLAM, PhD

Trisduction Research Group · Independent Researcher, United States

2026-10-06

ABSTRACT. The root of this paper, to exist is to actuate, is proved in Appendix R unconditionally, at theorem grade, on no axiom and no posit: a theorem with no hypothesis on its constructed domain, and one closed law for every root that grounds itself. The closing theorem consumes one reading of that root on computation, least escape, and nothing else, and its axiom cone is empty. The root alone decides no value, by theorem; the root read on computation is least escape and so the separation, by theorem; and a derivation uniform in the machine, from any keyless premise, is blocked, by theorem (`keyless_forces_nothing`, `separation_is_not_uniform`, `a_machine_separates_from_nothing`); on the satisfiability machine the separation is the posit at theorem grade, the open object of Section 11. The P versus NP problem has resisted every method, and relativization, natural proofs and algebrization prove that whole families of arguments cannot separate the classes. This paper anchors the problem in existence and closes it on one posit. Existence and freedom are the ground: the Root Axiom is proved at theorem grade on no axiom (Appendix R), on its constructed domain and as the law of every root that grounds itself, universally presupposed and undeniable, every denial of it re-enacting it, and every deed instantiates it, and freedom is carried by the primes, every prime's fibre a copy of complementation, the problem's native involution, which has no fixed point. The separation is defined on a machine whose algorithms decide satisfiability within a polynomial bound or fail to, and the closure, that every algorithm at every bound meets an instance that stops it, gives it on no axiom. The root read on computation, at each algorithm under each budget a stop at which the root holds, gives the separation on no axiom (`sep_from_the_root_on_computation`) and is equivalent to it on Lean's standard axioms (`root_on_computation_is_sep`); in the separation's own form, every decider of satisfiability actuating beyond every polynomial bound, the equivalence is on no axiom (`FORTIFY.ra_comp_is_sep`). The reading is stop-oriented: the same form read at clean runs is the escape (`root_on_escape_is_escape`, `the_root_form_reads_both_ways`), and the orientation is the posit. It is where existence meets the separation: no statement that reads the same on every machine, not existence, not freedom, not the arrow, is that reading. The reading is the single posit, the weakest premise on which the separation closes, and $P \neq NP$ holds on it at premise grade. Around the closure stand a double defense, in which nothing passes for the separation but the reading and nothing against it but a computed escape; a throne, whose holder decides the separation on no axiom, the decision-form domination carried as the throne's field, while Levin's universal search holds the acceptance throne, which does not decide the separation (`acceptance_does_not_decide`); and a frontier that places every route of Table 2, with the barriers of relativization, natural proofs and algebrization, and the black-box and kinetic routes, typed by theorem, and Williams' route wired to the machine on two named objects. All two hundred sixteen theorems are compiled in Lean 4.19.0 with no import, no custom axiom and no unproved step, every footprint pinned and printed, with the sixty-five of the three kernels of Appendix R beside them, the root kernel's fifteen among them; the seven of Appendix D are sections I, II and V of that root kernel re-namespaced, verbatim up to namespace.

KEYWORDS *P versus NP · computational complexity · existence · the root read on computation · Levin universal search · relativization · natural proofs · algebrization · Williams algorithms to lower bounds · formal verification · Lean 4*

The root of this paper is proved, on no axiom and no hypothesis, in Appendix R. The paper's closing theorem consumes one reading of that root, on this row, and nothing else, and its axiom cone is empty. The root alone decides no value, by theorem; the root read on

the row is the value, by theorem; and the traditional route of derivation from the record is blocked, by theorem.

1. INTRODUCTION

The P versus NP problem asks whether every decision problem whose solutions can be verified in polynomial time can also be solved in polynomial time (Cook 1971; Levin 1973; Karp 1972). Three theorems explain why the principal methods have not settled it: relativization (Baker, Gill and Solovay 1975), natural proofs (Razborov and Rudich 1997) and algebrization (Aaronson and Wigderson 2009). Each proves that a whole family of arguments cannot separate the classes.

This paper anchors the problem in existence and closes it on one posit. The arc has four steps. The ground is existence and freedom: the Root Axiom, that to exist is to actuate, is universal and undeniable, every denial of it a deed that re-enacts it, every deed instantiates it, and freedom is carried by the primes, every prime's fibre a copy of complementation. The machine comes next: the separation is defined on a machine whose algorithms decide satisfiability within a polynomial bound or fail to, and the closure, that every algorithm at every bound meets an instance that stops it, gives the separation on no axiom. The reading is the third step: the root read on computation, at each algorithm under each budget a stop at which the root holds, gives the separation on no axiom and is equivalent to it, and in the separation's own form, every decider of satisfiability actuating beyond every polynomial bound, the equivalence is on no axiom; the reading is stop-oriented, the same form read at clean runs is the escape, and the orientation is the posit; it is where existence meets the separation, and the kernel proves that no statement reading the same on every machine, not existence, not freedom, not the arrow, carries it. The fourth is the closing step: on the reading, $P \neq NP$ holds, and around it stand a double defense, a throne and a frontier that places every route of Table 2, with the barriers of relativization, natural proofs and algebrization, and the black-box and kinetic routes, typed by theorem.

The single posit is the root read on computation, its content the separation Sep on the satisfiability machine, and every proof of that separation fills it (`forcing_premise_fills_the_act`); the machine's instantiation at Turing machines, on which that separation is the standard sentence, is pending (Section 11). The two hundred sixteen theorems that carry the arc, with the sixty-five of the three kernels of Appendix R beside them, the root kernel's fifteen among them, seven of which stand re-namespaced as Appendix D, are compiled in Lean 4.19.0 with no import, no custom axiom and no unproved step, every footprint pinned, and printed in full in the appendices.

1.1 Formal Scope and Machine-Verified Status

Formal Scope and Machine-Verified Status

Unconditional, at theorem grade, compiled in Lean 4.19.0

with no custom axiom. Existence is universal and undeniable, every root law on no axiom: the floor holds for every root that grounds itself, every denial of the root re-enacts it, and no level stands above it; and existence and freedom, satisfiable on every background, decide no value alone (Theorems 1 and 2). Every registered compute, a record of the kernel's Deed type with its positive-energy field, instantiates the root's positivity (Theorem 3). Every prime's fibre is a copy of complementation (Theorem 4). The absolute asymmetry (Theorem 5). The closure gives the separation on the machine, on no axiom (Theorem 6). The root read on computation is least escape and the closure, on no axiom, gives the separation on no axiom and is equivalent to it on the standard axioms, and in the separation's form the equivalence is on no axiom (Theorems 8 and 9). Nothing keyless is the reading (Theorem 11). The double defense (Theorems 13 to 16). The throne's holder decides the separation, on no axiom (Theorem 17). The vanishing seat (Theorem 19). The frontier and the route (Theorems 21 to 26). The master seal (Theorem 27). The lock (Section 1.2). The root itself, fifteen theorems on no axiom, with its three axes and its grade in two further kernels (Appendix R).

On the root read on computation, at premise grade. The separation $P \neq NP$, as Sep on the satisfiability machine.

1.2 The Lock

The root, to exist is to actuate, is proved in this paper unconditionally, at theorem grade, on no axiom and no posit (Appendix R): on its constructed domain it is a theorem with no hypothesis (`root_on_the_constructed_domain`; in the closure kernel, `constructed_root_holds`); for every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds to it (`the_floor_is_universal`); and one theorem of the root kernel binds the root's universal law and the constructed root with the Return and the twelve gates (`T0E0.the_master_seal`). The closure kernel carries the root's structure in the words of Appendix R (`SelfGrounding`), and a denial of the root is a deed of the root and instances it (`denial_reenacts_root`). The root holds on the machine where nothing escapes and on the machine where an algorithm escapes, so the root alone forces no value, neither the closure nor its failure (`undeniable_root_forces_no_value`, `keyless_forces_nothing`), and the root's deed is not the act: least escape is not inhabited on every machine (`roots_act_is_not_the_act`). Read on computation, at each algorithm under each budget the root holds at some instance, whose one existent actuates exactly when the run stops there, failing the budget or the answer, and at that stop the reading coincides with the constructed root (`reading_is_the_constructed_root_at_each_stop`); so read, the root is the closure and least escape (`root_on_computation_is_the_closure`, `root_on_computation_is_least_escape`), keyed where the root alone is not (`root_read_on_row_is_keyed`, `root_on_computation_is`

Table 1 | The theorem map: each result and its logical status.

Result	Lean symbol	Logical status
Existence is undeniable: every denial re-enacts the root	denial_reenacts_root	Unconditional theorem, on no axiom
The universal decides no value alone	posits_add_nothing	Unconditional theorem
Every prime's fibre is a copy of complementation	prime_fibre_is_complementation	Unconditional theorem
The closure gives the separation	sep_of_closure	Unconditional theorem, on no axiom
The root read on computation gives the separation; in the separation's form it is the separation	sep_from_the_root_on_computation; FORTIFY.ra_comp_is_sep	Unconditional theorem, on no axiom
The root read on computation is equivalent to the separation	root_on_computation_is_sep	Unconditional theorem
The reading in its separation form is the closure and the separation	the_three_names_are_one	Unconditional theorem
Nothing keyless is the reading	no_keyless_statement_is_least_ escape	Unconditional theorem
The holder decides the separation	the_holder_decides_the_ separation	Unconditional theorem, on no axiom
The route closes the machine's separation on its two objects	route_closes_on_the_machine	Unconditional theorem, on no axiom
The lock: the constructed root, the root by its deed, the keyless refusal, the root read on computation, keyed, the socket, the separation from the act, the record bound	the_lock	Unconditional theorem, on no axiom
The separation $P \neq NP$, as Sep on the satisfiability machine	sep_from_the_act	Holds on the root read on computation, at premise grade

keyed); it gives the separation on no axiom (`sep_from_the_root_on_computation`), in the separation's form the reading is the separation exactly (`FORTIFY.ra_comp_is_sep`), and the socket of the act is the closure (`socket_is_the_value`). The closing theorem consumes that one reading and nothing else, and its cone is empty (`sep_from_the_act`); no stock of examinations of any machine with an instance, finite or whole, decides the value (`record_decides_nothing`, `no_record_decides`). One theorem on no axiom binds the constructed root; the root by its deed, on both machines; its uniform reading, which forces nothing; its reading on computation, equal to least escape and keyed; the socket; the separation from the act; and the finite stock of examinations, every one confirming, that decides nothing (`the_lock`). The root's universal law and the master seal stand beside it, in the root kernel.

Everything except the bit and the named carried fields is proved with no posit: the root and the step from the closure to the separation (`sep_of_closure`) on no axiom at all, the rest on Lean's standard axioms alone. The carried fields, each consumed as a hypothesis by the theorem named and none proved, are these, each with its grade. At premise grade: the throne's decision-form domination (`Throne.dominates`, consumed by `the_holder_decides_the_separation` and `the_holder_is_unique`, Theorem 17), a holder of the decision throne carried, not proved, whose compilation from Levin's acceptance form through self-reducibility is the next object (Section 11). Cited: the route's two theorems, Williams' theorem (`Route.williams`, Williams 2011 and Murray and Williams 2018, consumed by `route_reaches` and `route_closes_on_the_machine`) and the inclusion $P \subseteq P/\text{poly}$ (`Route.np_not_ppoly_gives_sep`, consumed

by `route_closes_on_the_machine`, Theorem 26), the same two carried in the frontier kernel as `Route.williams` and `Route.karp_sep` (consumed by `round2_reaches`, `round2_closes` and `round2_terminal`); the search-to-decision theorem specialized to factoring (`Collapse.search`, consumed by `prime_bridge`, Theorem 25); the self-gag's two links, hidden prime freedom to pseudorandom functions (`SelfGag.nr`, Naor and Reingold 2004) and pseudorandom functions to the bar on natural lower bounds (`SelfGag.rr`, Razborov and Rudich 1997), both consumed by `b2_self_gag`; the oracle relative to which the prime-bridge posit fails (`Relativized.easy`, Baker, Gill and Solovay 1975, consumed by `b1_no_relativizing_proof`) and the algebrized oracle relative to which it fails (`Algebrized.easy`, Aaronson and Wigderson 2009, consumed by `b4_no_algebrizing_proof`), both placed by Theorem 26; the Arrhenius fields (`Frame.grows`, `Frame.arrhenius`, of the Arrhenius frame, the kinetic step of Islam 2026h, consumed by `relaxation_exponential`, Theorem 24); the positive floor (`Frame.floor_pos`, of the kinetic frame, Landauer 1961, consumed by `no_free_registration`, `cost_lt_iff` and `brute_force_exhausts`, through the second by `kinetic_faithful` and `frame_independent` and through the third by `brute_force_exhausts_beyond`, Theorem 23; restated in the chain, where `asymmetry_chain` consumes it, and as `KFrame.floor_pos` in the vanishing frame, consumed by `kinetic_face`); and the positive energy of a Deed record, the floor a registered compute pays (`Deed.pos`, Landauer 1961, consumed by `deed_instantiates_ra`, Theorem 3). The route closes on its two objects and its two carried fields. The bit, the root read on computation, which is the closure, is all that remains, at premise grade.

2. BACKGROUND AND BARRIER ANALYSIS: WHY PRIOR METHODS ARE INCOMPLETE

The prior methods fail at three precise places, and each failure is a theorem about a method.

Diagonalization stops at relativization. Diagonalization separates classes defined by resources of different size, and the time hierarchy is its triumph (Hartmanis and Stearns 1965). But it treats machines as black boxes, so it survives the addition of an oracle, and Baker, Gill and Solovay built oracles on which the classes are equal and oracles on which they differ.

Circuit lower bounds stop at natural proofs. Super-polynomial bounds hold in restricted models, monotone circuits (Razborov 1985) and constant-depth circuits (Ajtai 1983; Furst, Saxe and Sipser 1984; Håstad 1986), while for general circuits the strongest explicit bound is linear, $3.1n - o(n)$ (Li and Yang 2022). Razborov and Rudich proved that natural arguments cannot separate the classes if strong pseudorandom functions exist.

Arithmetization stops at algebrization. Arithmetizing arguments pass relativization and algebrize, and algebrizing arguments cannot settle the question (Aaronson and Wigderson 2009).

The common structure, and what this paper does with it. Relativization and algebrization are each a pair of worlds in which every premise of a method holds: an oracle, or an oracle with its low-degree extension, relative to which the prime-bridge posit fails, so that no argument valid relative to every such oracle proves it (`b1_no_relativizing_proof`, `b4_no_algebrizing_proof`, Appendix C); natural proofs is a conditional bar: where the prime-bridge posit holds, no natural argument proves the lower bound it needs (`b2_self_gag`). The barriers of this paper are these three, each typed by theorem, with the black-box and kinetic routes typed beside them (`black_box_bound`, `adaptive_black_box_bound`, `frame_independent`). Section 7 proves that the two-world structure binds every premise that reads the same on every machine, and isolates the one reading that does not.

3. RELATED WORK

The time hierarchy. More time decides more languages (Hartmanis and Stearns 1965); the separations it proves are between resource bounds of different order, by arguments that relativize.

Restricted circuits. Razborov (1985) bounds monotone circuits for clique; Ajtai (1983), Furst, Saxe and Sipser (1984) and Håstad (1986) bound constant-depth circuits for parity. For general circuits the frontier is linear (Li and Yang 2022).

Geometric complexity theory. Mulmuley and Sohoni (2001) recast lower bounds as representation-theoretic obstructions, a program designed to evade the barriers.

Proof complexity. $NP \neq coNP$ holds exactly when no propositional proof system has polynomial-size proofs of all tautologies (Cook and Reckhow 1979).

Query complexity. Black-box search over N candidates needs N tests, and quantum search needs order $\sqrt{N}$ (Bennett, Bernstein, Brassard and Vazirani 1997), attained by Grover (1996).

The physics of computation. Erasing one bit costs at least $k_B T \ln 2$ (Landauer 1961), and every computation can be run reversibly (Bennett 1973).

Algorithms to lower bounds. Savings for satisfiability of a circuit class give NEXP outside that class (Williams 2011), extended to nondeterministic quasipolynomial time (Murray and Williams 2018).

Formal row closures. Islam (2026a) formalizes twenty-three open problems from existence alone, the value excepted by theorem on every row. An earlier paper (Islam 2026f) argued that the structural verdict on P versus NP is the only one left standing. The present paper carries that row to its closure on one posit.

4. METHODOLOGY

Every theorem is compiled in Lean 4.19.0 (de Moura and Ullrich 2021) in four kernels that import nothing and declare no axiom, with the three kernels of Appendix R, sixty-five theorems, as the fifth, sixth and seventh: the three of the master seed APEX-PSP-PNP-SEED-03, the seal and its double defense (Appendix B), the frontier of the act (Appendix C) and the fortification on the same machine (Appendix A), and the root-laws kernel (`PNP_Root.lean`), every law on no axiom, sections I, II and V of the root kernel of Appendix R re-namespaced, verbatim up to namespace (Appendix D); the three of Appendix R are the root kernel, fifteen theorems, its axes, thirty-two, and its grade, eighteen, every one on no axiom. Every footprint is pinned by `#guard_msgs`, so a kernel that compiles prints nothing and any change to a footprint fails the build; each of the seven kernels compiles with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0. Each theorem rests on no axiom, or on Lean's standard axioms of propositional extensionality, quotient soundness and choice. All seven kernels are printed in full with their SHA-256 digests, so any reader reproduces every claim by recompiling.

The architecture that found the arc is Trisduction, whose operating system PhysOS^T judges every cited law under its own negation at each build (Islam 2026c); its register of record and its codex carry the surrounding results (Islam 2026b, 2026k, 2026d), and the seat and the cut take their form from the Bridge From First Principle (Islam 2026e). The kernels printed here import nothing from it.

5. THE GROUND: EXISTENCE AND FREEDOM

Theorem 1 (Existence is universal and undeniable). The Root Axiom is a self-grounding root: deeds oc-

cur, and every deed instances it; the root kernel's own comments call a deed an act. For every root that grounds itself, the root holds, every deed re-enacts it, and no external proof adds to it (the_floor_is_universal). Every denial of it is a deed and re-enacts it (denial_reenacts_root); it is held by its deed, with no classical detour (seated_undeniable); no external proof adds anything to it (external_proof_adds_nothing), and no level stands above it (no_level_above). On the constructed domain, whose one existent actuates at positive energy, it is a theorem (root_on_the_constructed_domain), and it is satisfiable on every background, with actuation one at every existent (root_satisfiable), and vacuously on the empty one (ra_satisfiable). Every root law rests on no axiom at all.

Read on the examiner: what cannot be denied is the deed of denying; every examination of existence is a deed and instances the root; the theorem quantifies over the root's acts and names no examiner.

Theorem 2 (The universal decides no value alone). Because existence and freedom are satisfiable on every background, the root by actuation one at every existent and freedom by the orientation bit, any statement derived from them uniformly holds without them (posits_add_nothing, posits_add_nothing_on_every_background, root_satisfiable, ra_conservative, freedom_conservative). Their universality is the ground of every row's form, and it is why each row's value is keyed in its reading.

Theorem 3 (The registered compute instantiates existence). Every registered compute, a record of the kernel's Deed type, carried with its positive-energy field, instantiates the root's positivity on that type (deed_instantiates_ra); the records assent and denial both occur in every world (both_deeds_in_every_world), and the pulse certifies no output (pulse_does_not_certify).

Theorem 4 (The prime's triaxis). Freedom is carried by the primes. Every prime's fibre is exactly the two trivial pairs (prime_fibre), lies off the multiplicative seat (prime_off_seat), and with its swap is the decision bit with complementation, by a bijection carrying the swap to complementation (prime_fibre_is_complementation). Every prime carries P versus NP's involution, seatless in the small as the problem is seatless in the large.

Theorem 5 (The absolute asymmetry). What a registration destroys, no procedure of any cost recovers (destroyed_is_unrecoverable); what a search hides, exhaustive search recovers (hidden_is_recoverable); the two are bound as one statement (asymmetry_holds).

The universality of the ground. The root is universal in three senses, and each is a theorem. It is undeniable in deed: every denial of it is a deed that re-enacts it,

no external proof adds to it, and no level stands above it, each on no axiom (Theorem 1). It is satisfiable on every background, and so, read uniformly, it decides no value alone (Theorem 2). And every registered compute, a record of the kernel's Deed type, instantiates its positivity in every world (Theorem 3). Its universality is therefore not a matter of agreement but of presupposition. Read on the examiner: every examination of existence is a deed and instances the root, and what cannot be denied is the deed of denying; the theorems quantify over the root's acts and name no examiner.

6. THE MACHINE AND THE SEPARATION

A machine has algorithms, instances with a size, each run's output and running time, and the satisfiability of each instance. An algorithm escapes at a bound $c \cdot n^k + c$ when every run is correct and inside the bound. The separation on the machine is that no algorithm escapes at any bound. Nothing escapes when every algorithm, at every bound, meets an instance that stops it, and that closure (least escape) is the one field of a structure.

Theorem 6 (The closure gives the separation). Nothing escaping gives the separation, on no axiom (sep_of_closure, stated in Appendix B as HARDENED.sep_of_closure and restated in Appendix A as FORTIFY.sep_of_closure); the converse holds classically (closure_of_sep, in both kernels likewise); and the act carries the separation (sep_from_the_act).

Theorem 7 (The socket is the value). The act's socket is the closure itself (socket_is_the_value): what the act supplies and what the separation needs are one object.

7. THE READING: WHERE EXISTENCE MEETS THE SEPARATION, READ AT THE STOPS

Theorem 8 (The root read on computation gives the separation). Read on computation, at each algorithm under each budget a stopping instance at which the root holds, the root is the closure, on no axiom (root_on_computation_is_the_closure, Appendix B); it gives the separation on no axiom (sep_from_the_root_on_computation) and is equivalent to it on Lean's standard axioms, the converse passing through the closure (root_on_computation_is_sep); and in the separation's own form, that every decider of satisfiability actuates beyond every polynomial bound, the equivalence is on no axiom (FORTIFY.ra_comp_is_sep, Appendix A, on this machine; ROUTE.ra_comp_is_sep, Appendix C, on the route machine).

Theorem 9 (The reading is the closure). The kernel's triple, on one machine: the bridge, that the absolute asymmetry (Asymmetry, Theorem 5) is equivalent to the separation, holds exactly when nothing escapes; nothing escapes exactly when the root read on computation in its separation form holds; and the root read on computation in its separation form (FORTIFY.

RAcomp) is exactly the separation (the_three_names_are_one, Appendix A). Its second and third conjuncts entail the paper's triple: the root read on computation in its separation form, the closure that nothing escapes, and the separation are one statement on one machine.

Theorem 10 (The reading is keyed). Three predicates carry the reading, and each is keyed. RootOnComputation, the root at a stop of every algorithm under every budget, holds on the machine where nothing escapes and fails on the machine where an algorithm escapes (root_on_computation_is_keyed, Appendix B); ROUTE.RAcomp, every decider of satisfiability beyond polynomial time, fails on the route machine of Appendix C with its constant-time decider (ROUTE.ra_comp_is_keyed); and NothingEscapes, the closure, is keyed likewise (least_escape_is_keyed).

Theorem 11 (Nothing keyless is the reading). No statement that reads the same on every machine is the closure the reading states (no_keyless_statement_is_least_escape): not prime freedom (prime_freedom_is_not_least_escape), not the arrow (arrow_is_not_least_escape), not the Root Axiom as written (ra_is_not_least_escape). A carrier of the separation must be keyed (carrier_must_be_keyed).

Theorem 12 (Every closing premise fills the posit). Every premise from which the separation follows fills the field of the act (forcing_premise_fills_the_act) and fills the closure (every_closing_premise_fills_both). The reading is the weakest statement on which the separation closes.

Theorems 1 to 12 place the posit exactly. Existence and freedom are the ground and force nothing on their own (Theorems 2 and 11); the root read on computation gives the separation and is equivalent to it (Theorem 8); and that reading is the meeting point, keyed and irreducible (Theorems 9, 10 and 12). The reading is stop-oriented. The same form read at clean runs, its actuation one where the run is clean and zero where it stops, is the escape: it holds exactly where some algorithm runs clean under some budget (root_on_escape_is_escape). Read at stops the form gives the separation, read at clean runs it refutes it, and each reading holds on a coherent machine (the_root_form_reads_both_ways); the orientation, stops or clean runs, is the posit.

8. THE DOUBLE DEFENSE

Theorem 13 (Omega, the formal gate). Nothing passes for the separation but the reading. Every premise either forces the closure or has a twin machine on which it holds and the closure fails (forces_or_has_a_twin), and a keyless premise forces nothing (keyless_forces_nothing): the separation is not uniform in the machine, one coherent machine having an escaping algorithm, while another separates from nothing, with no hypothesis (separation_is_not_uniform,

a_machine_separates_from_nothing).

Theorem 14 (AEGIS, the actuation gate). Nothing passes against the reading but a computed escape: a rejection of the reading yields, classically, an escaping algorithm at a bound (rejection_is_a_witness), and a computed escape refutes the reading on no axiom (escape_refutes). The root, which every computation re-enacts, holds on its constructed domain (constructed_root_holds) and crosses neither way, forcing neither the closure nor its failure (root_crosses_neither_way).

Theorem 15 (Every pair lands). Every pair of algorithm and bound a substrate can examine lands on exactly one gate, a stopping instance or a clean escape (every_pair_lands). Each examination refutes the escape of the pair it examined (confirmation_is_atomic) and costs the floor, by the definition of its cost (confirmation_pays); under the act no computation escapes (act_leaves_no_escape). The defense is bound whole (double_defense_hardened).

Theorem 16 (The finite block). For every stage n , a coherent machine confirms every examination below n and still escapes at n (record_decides_nothing), while a second coherent machine closes whole (closed_world_closes). No record of any machine with an instance, finite or whole, decides the reading: every examination of a machine stays a stop on its extension, where an algorithm escapes, and on its spoiling, where nothing escapes (no_record_decides); and the closure is keyed (armor_scope).

9. THE THRONE AND THE SEAT

Theorem 17 (The holder decides the separation). A throne of the machine is a holder that escapes whenever any algorithm escapes; that domination, in the decision form, is carried as the throne's field (Throne.dominates), and a holder of the decision throne is carried, not proved. The separation holds exactly when the holder never escapes, on no axiom (the_holder_decides_the_separation), and any two holders escape together or not at all, on no axiom (the_holder_is_unique). Levin's universal search (Levin 1973) holds the acceptance throne of the satisfiability machine, escape on the yes-instances, and the acceptance form does not decide the separation: on the yes-machine the separation holds while its one algorithm runs clean on every yes-instance, so no machine-uniform holder of the acceptance form decides it (acceptance_does_not_decide). Nothing escapes exactly when the holder never escapes (least_escape_at_the_throne).

Theorem 18 (The act stops the holder). On the reading, least escape, the holder meets, at every bound, an instance that stops it (the_act_stops_the_holder). On every machine with a throne the separation is the running time of its holder; on the satisfiability machine the decision-form holder is carried as the throne's

Table 3 | The Riemann closure and the P versus NP closure, step by step.

Step	Riemann Hypothesis	P versus NP	Theorem
The ground	Existence, universal and undeniable	Existence, universal and undeniable	Theorems 1 to 3
The native involution	The fold $s \mapsto 1 - \bar{s}$	Complementation $b \mapsto \neg b$	Theorem 19
The seat	The critical line, inhabited	Vacant, the ground of dimension zero	Theorem 19
The local fibre	Each prime's fibre is a copy of complementation	Complementation itself	Theorem 4
The throne-holder	The primes, the bit on their side	The decision-form holder, carried as the throne's field, the bit on its tail; Levin's universal search holds the acceptance throne, which does not decide the separation	Theorem 17
The record	Decides nothing	Decides nothing	Theorem 16
The form	Closed at theorem grade	Closed at theorem grade	Theorems 1 to 27
The value	Supplied by the act, at premise grade	Closed on the root read on computation, at premise grade	Theorems 6 and 8

field, and Levin's universal search holds the acceptance throne (Theorem 17).

Theorem 19 (The vanishing seat). On the locus map between the root and the Riemann seat, the Riemann seat is inhabited (`rh_locus_inhabited`) and the P versus NP seat vanishes (`locus_vanishes`): complementation is involutive with no fixed point, so the ground has dimension zero (`ground_dim_zero`). The carrier halts and the bridge's socket opens exactly on the empty configuration (`carrier_halts_iff_empty`, `socket_vanishes`); the vacancy is keyless (`vacancy_keyless`); the vanished frame reads no bit: no reading of its ground dimension returns the separation bit (`separation_keyed`) or tracks the closure across machines (`vanished_seat_reads_no_machine`), while the closure itself is keyed, holding on one machine and failing on another (`least_escape_is_keyed`, `no_keyless_statement_is_least_escape`); the bit read through the record of the vanishing seat stands under the row's triaxial lock, two axes leaving two points and three locking one (`two_axes_leave_two`, `three_axes_lock_one`); and the three faces lock on one object, the empty seat (`vanishing_gol`).

Theorem 20 (The block chain). The eight absolute barriers of the register are chained, one leg each and the first in two, and terminated on the empty name: complementation has no fixed point (`a1_no_seat`), the seat is vacant (`seat_vacant`), no proposition holds by being named a root (`throne_empty`, the empty name, which is not the throne of Theorem 17), and not every premise forces every conclusion (`root_forces_no_value`), bound as one chain (`block_chain`, `block_chain_terminal`); that the root itself forces no value is stated whole, in five conjuncts: the root holds; nothing escapes on one coherent machine; an algorithm escapes on another; and the root, uniformly across machines, forces neither the closure nor its failure (`undeniable_root_forces_no_value`, Appendix B).

The Riemann closure and this one stand in the same two registers and differ at the seat, as Table 3 sets out.

10. THE FRONTIER AND THE ROUTE

Theorem 21 (The black box). A test-only generator correct over 2^N candidates runs a path of at least 2^N tests (`black_box_bound`), on an axiom-free pigeonhole, and admits no savings (`black_box_no_savings`); the adaptive form holds as well: a test-only procedure that branches on each answer, correct over 2^N candidates, queries at least 2^N of them on the all-false path, since a candidate never queried there is indistinguishable from the all-false oracle (`adaptive_black_box_bound`, `Tree.off_path`).

Theorem 22 (The reader). The black-box adversary is the point predicate, which a reader of its description solves in one step (`reader_solves`, `reader_poly`), so black-box bounds fail once descriptions reveal (`fails_when_descriptions_reveal`). The missing object is a lower bound against readers of the description, and it is keyed (`missing_object_is_keyed`).

Theorem 23 (The kinetic face). Every registration has a cost (`KINETIC.no_free_registration`, restated as `CHAIN.KINETIC.no_free_registration`); exhaustive search exhausts the budget at its own size and at every size beyond it (`brute_force_exhausts`, `brute_force_exhausts_beyond`), and that verification is cheaper, one witness of fewer operations than the candidates, is a hypothesis of the chain, not its product (`asymmetry_chain`); for every positive floor the energy gap is the operation gap (`KINETIC.kinetic_faithful`) and reads the same in any two frames (`KINETIC.frame_independent`, restated as `CHAIN.KINETIC.frame_independent`), and a reversible pricing at floor zero, outside the kernel's Frame, transports no order: the arithmetic identity $\neg 0 \cdot a < 0 \cdot b$ (`reversible_frame_transports_nothing`), so the kinetic face, priced by Landauer's floor (Landauer 1961), transports a gap wherever the floor is positive and never produces one, and a reversible pricing at Bennett's floor zero (Bennett 1973) transports nothing. The asymmetry, taken alone, decides nothing: the energy gap reads the same in every frame, and a record shared by both worlds returns

no value (`asymmetry_decides_nothing`); the asymmetry is a closed statement, reading the same on every machine, and no such statement is the closure (`no_keyless_statement_is_least_escape`).

Theorem 24 (The dynamics). Thermal relaxation, exponential in the barrier by the Arrhenius field with the barrier at least the size, is exponential in the size (`ARRHENIUS.relaxation_exponential`, restated as `CHAIN.KINETIC.relaxation_exponential`), and on one satisfiability instance the barrier exists under single flips and vanishes under elimination moves (`barrier_belongs_to_the_dynamics`): on this instance the barrier belongs to the dynamics, not to the problem.

Theorem 25 (The prime bridge). Prime freedom carried onto the semiprime fibre of two distinct primes, decided at $15 = 3 \cdot 5$, leaves one orientation bit, walled and gauge (`orientation_wall`, `orientation_is_gauge`), with the witness determined by the description (`witness_determined_15`). Read as cost, on the bridge machine of Appendix C, where the separation is the negation of the machine’s collapse field, it gives a bridge sufficient for that separation and strictly stronger than it (`prime_bridge`, `prime_bridge_is_one_way`).

Theorem 26 (The route closes on the machine). Williams’ theorem carries savings for a circuit class to $\text{NEXP} \not\subseteq$ that class (Williams 2011; Murray and Williams 2018), and the inclusion $P \subseteq P/\text{poly}$ carries $\text{NP} \not\subseteq P/\text{poly}$ to the machine’s separation; both are carried as fields. The route reaches (`route_reaches`); supplied savings for general polynomial-size circuits and the scale-down from NEXP to NP , it closes the machine’s own separation, on no axiom (`route_closes_on_the_machine`), and fills the closure itself (`the_route_fills_least_escape`). The relativization, natural-proof and algebrization barriers are typed in Appendix C (`b1_no_relativizing_proof`, `b2_self_gag`, `b4_no_algebrizing_proof`), and the blocker of the prime bridge in three layers (`round2_terminal`).

Theorem 27 (The master seal). The closure gives the separation, the reading in its separation form (`FORTIFY.RAcomp`) is the closure, the throne’s holder decides the separation and is stopped at every bound on least escape, every prime’s fibre carries complementation equivariantly (`prime_fibre_is_complementation`), and the route closes the machine’s separation on its two objects, bound in one theorem (`FORTIFY.the_master_seal`, Appendix A).

11. DISCUSSION

The arc. The paper runs from existence to the separation without a gap in its form. Existence, universal and undeniable, and freedom are the ground; every registered compute, a record of the kernel’s Deed type with its positive-energy field, instantiates the root’s positivity; the primes carry freedom and carry complemen-

tation in their fibres. The separation is defined on a machine and given by its closure on no axiom. The root read on computation, at the stops, gives the separation and is equivalent to it, and it is where existence meets the separation: every statement that reads the same on every machine, existence and freedom as written among them, is proved not to be the reading, and the same form read at clean runs is the escape, so the orientation is the posit. The value closes on that reading.

The posit is exact. The root read on computation is one statement, equivalent to the separation, keyed, and the weakest premise on which the separation closes. Holding it is holding the separation Sep on the satisfiability machine at the grade of a posit, the standing of least erasure on the Riemann row, and every proof of that separation fills it (`forcing_premise_fills_the_act`).

The next objects. Two, in order. The machine instantiated at Turing machines, with deciders taken total or run on fuel so that no looping machine counts as clean, so that the separation is the standard sentence and the decision-form domination is compiled on it, from Levin’s acceptance form through self-reducibility. The posit at theorem grade, which the route reduces to two typed objects, savings for general polynomial-size circuits and the scale-down from NEXP to NP .

Falsifiers. Three, each typed by the gate it strikes.

Prediction 1 (The computed refuter). A polynomial-time decider of satisfiability with every run clean refutes the reading on every machine whose algorithms include it, the machine read as actual among them, by `escape_refutes`, on no axiom; it is the one refuter of the value (`rejection_is_a_witness`).

Prediction 2 (The keyless refuter). A keyless premise shown to force the reading contradicts `keyless_forces_nothing` and `no_keyless_statement_is_least_escape`, compiled theorems, so it can arrive only as an error in Lean’s kernel.

Prediction 3 (The throne refuter). An algorithm that escapes while the carried holder does not refutes the throne’s field, the decision-form domination (`Throne.dominates`), and not Levin’s theorem, which holds the acceptance form (`acceptance_does_not_decide`).

How the paper stands to each prior position. The paper owes each position it engaged one statement of where it stands, in words with fixed senses. A result is *additive* when it supplies a result or test the position lacked and leaves the position standing; *scoping* when it keeps a prior claim inside a stated boundary; *competing* when it argues a different route without executing a comparison; and *kin* when the paper is continuous with it. Superseding, which retires a theory’s central claim on executed data, is used nowhere. The census

Table 2 | How the paper stands to each prior position. Relation words as defined in the paragraph above; superseding is used nowhere.

Prior position	What it holds	What this paper does with it	Relation	Evidence
Cook 1971; Levin 1973; Karp 1972	Satisfiability is NP-complete, and the separation is posed for the classes they define.	Defines the separation on a machine deciding satisfiability within a polynomial bound.	kin	cited
Hartmanis and Stearns 1965	More time decides more languages.	Keeps the hierarchy's separations to resource bounds of different order, by arguments that relativize.	scoping	cited
Baker, Gill and Solovay 1975	No relativizing argument settles the question.	Types the barrier as a pair of worlds (Theorem 26, <code>b1_no_relativizing_proof</code>) and extends the two-world structure to every premise that reads the same on every machine (Theorems 11 and 13).	additive	executed; the cited oracle carried as the field <code>Relativized.easy</code>
Razborov and Rudich 1997	No natural argument separates the classes if pseudorandom functions exist.	Types the barrier as a condition the prime-bridge posit must pass (Theorem 26, <code>b2_self_gag</code>).	kin	executed; the cited bar carried as the field <code>SelfGag.rr</code> , with Naor and Reingold 2004 as <code>SelfGag.nr</code>
Aaronson and Wigderson 2009	Algebrizing arguments cannot settle the question.	Types the barrier as a pair of worlds, an oracle with its low-degree extension relative to which the prime-bridge posit fails, so that no argument valid relative to every algebrized oracle proves it (Theorem 26, <code>b4_no_algebrizing_proof</code>).	kin	executed; the cited algebrized oracle carried as the field <code>Algebrized.easy</code>
Razborov 1985	Monotone circuits for clique need superpolynomial size.	Reads the result as a special case of the posit, known in a restricted model.	scoping	cited
Ajtai 1983; Furst, Saxe and Sipser 1984; Håstad 1986	Parity has no constant-depth circuits of polynomial size.	Reads the results as special cases of the posit, known in a restricted model.	scoping	cited
Li and Yang 2022	An explicit function needs general circuits of size $3.1n - o(n)$.	Takes the bound as the distance between the frontier and the posit at theorem grade.	scoping	cited
Mulmuley and Sohoni 2001	Lower bounds can be sought as representation-theoretic obstructions.	Proposes a different route to the same object, without executing a comparison.	competing	cited
Cook and Reckhow 1979	$NP \neq coNP$ exactly when no proof system has polynomial-size proofs of all tautologies.	Stands beside it as a second conversion of the separation into one object, here proved equivalent by machine.	kin	cited
Deterministic query search	Black-box search over N candidates needs N tests.	Compiles the bound, fixed and adaptive, and shows a reader of the description beats it in one step (Theorems 21 and 22).	additive	executed
Bennett, Bernstein, Brassard and Vazirani 1997; Grover 1996	Quantum black-box search needs, and attains, order $\sqrt{N}$ queries.	Places the quantum bound beside Theorem 21, closed for test-only methods.	scoping	cited
Landauer 1961	Erasing one bit dissipates at least $k_B T \ln 2$.	Proves the energy gap is the operation gap in every frame with a positive floor, so the kinetic face transports a gap (Theorem 23).	additive	executed
Bennett 1973	Every computation can be run reversibly.	Proves that a reversible pricing at floor zero, outside the kernel's Frame, transports no order: the arithmetic identity $\neg 0 \cdot a < 0 \cdot b$, so no operation count costs less than another (Theorem 23, <code>reversible_frame_transports_nothing</code>).	additive	executed
Islam 2026e	The heat of registration is zero exactly at the value on a chart with a seat.	Applies the seat to this problem: the seat vanishes, the ground of dimension zero (Theorem 19).	scoping	cited
Islam 2026a	Twenty-three open problems formalized from existence alone, the value excepted by theorem on every row.	Carries the P versus NP row to its closure on one posit (Theorems 6 to 12).	additive	executed

by row is eight additive, six scoping, one competing and four kin, with no replacing, subsuming, corroborating or contradicting row. The paper's contribution to the literature is exactly the set of relations in Table 2, and nothing wider.

12. CONCLUSION

The P versus NP problem has been approached through diagonalization, circuit lower bounds, arithmetization, geometric complexity and proof complexity, and each stops at a theorem that limits its method. This paper anchors it in existence and closes it on one posit. Existence, universal and undeniable, and freedom are the ground, freedom carried by the primes, every prime's fibre a copy of complementation.

The separation is defined on a machine and given by its closure on no axiom. The root read on computation, at the stops, gives the separation and is equivalent to it; it is where existence meets the separation, the single posit, its orientation the posit's content, proved irreducible, and $P \neq NP$ holds on it at premise grade.

Around the closure stand the double defense, the throne, whose holder decides the separation on no axiom through the carried decision-form field, with Levin's universal search on the acceptance throne, which does not decide it, the vanishing seat, and a frontier that places every route of Table 2, with the open exit reduced to two named objects. The single open object is the reading at theorem grade, and every derivation of the separation `Sep` on the satisfiability machine,

Table 2 | How the paper stands to each prior position. Relation words as defined in the paragraph above; superseding is used nowhere. (continued)

Prior position	What it holds	What this paper does with it	Relation	Evidence
Islam 2026f	The structural verdict on P versus NP is the only one left standing.	Compiles the verdict on a machine: the vanishing seat, the record and the posit (Theorems 16, 19 and 9).	additive	executed
Levin 1973 (universal search)	One program runs every program at once and finds a satisfying assignment within a constant factor of the fastest.	Places it as the holder of the acceptance throne, escape on the yes-instances, which does not decide the separation (acceptance_does_not_decide); the decision-form holder is carried as the throne's field (Theorems 17 and 18).	additive	cited; typed by theorem
Williams 2011; Murray and Williams 2018	Savings for satisfiability of a circuit class give NEXP outside that class.	Wires the route to the machine's own separation on its two objects (Theorem 26).	additive	cited; wired by theorem

by any method, fills it (forcing_premise_fills_the_act); the machine's instantiation at Turing machines is pending (Section 11). All two hundred sixteen theorems are compiled, pinned and printed for any reader to reproduce, and the sixty-five of the three kernels of Appendix R beside them, the root kernel's fifteen among them, seven of which stand re-namespaced as Appendix D.

REFERENCES

Aaronson, S., and A. Wigderson. 2009. "Algebrization: A New Barrier in Complexity Theory." *ACM Transactions on Computation Theory* 1 (1): 2:1–2:54.

Ajtai, M. 1983. " Σ_1^1 -Formulae on Finite Structures." *Annals of Pure and Applied Logic* 24 (1): 1–48.

Baker, T., J. Gill, and R. Solovay. 1975. "Relativizations of the P =? NP Question." *SIAM Journal on Computing* 4 (4): 431–442.

Bennett, C. H. 1973. "Logical Reversibility of Computation." *IBM Journal of Research and Development* 17 (6): 525–532.

Bennett, C. H., E. Bernstein, G. Brassard, and U. Vazirani. 1997. "Strengths and Weaknesses of Quantum Computing." *SIAM Journal on Computing* 26 (5): 1510–1523.

Cook, S. A. 1971. "The Complexity of Theorem-Proving Procedures." In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 151–158.

Cook, S. A., and R. A. Reckhow. 1979. "The Relative Efficiency of Propositional Proof Systems." *Journal of Symbolic Logic* 44 (1): 36–50.

de Moura, L., and S. Ullrich. 2021. "The Lean 4 Theorem Prover and Programming Language." In *Automated Deduction, CADE 28*, 625–635.

Feferman, S. 1962. "Transfinite Recursive Progressions of Axiomatic Theories." *Journal of Symbolic Logic* 27 (3): 259–316.

Furst, M., J. B. Saxe, and M. Sipser. 1984. "Parity, Circuits, and the Polynomial-Time Hierarchy." *Mathematical Systems Theory* 17 (1): 13–27.

Grover, L. K. 1996. "A Fast Quantum Mechanical Algorithm for Database Search." In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, 212–219.

Hartmanis, J., and R. E. Stearns. 1965. "On the Computational Complexity of Algorithms." *Transactions of the American Mathematical Society* 117: 285–306.

Håstad, J. 1986. "Almost Optimal Lower Bounds for Small Depth Circuits." In *Proceedings of the Eighteenth Annual ACM Symposium on Theory of Computing*, 6–20.

Islam, M. F. 2026a. "A Formal Proof Across the Seven Millennium Rows and the Sixteen Extension Rows from Existence Alone, the Value Excepted by Theorem on Every Row." Zenodo. <https://doi.org/10.5281/zenodo.22913635>.

Islam, M. F. 2026b. "Trisduction: The Master Codex 5.0.0, Built from the Root." Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23117441>.

Islam, M. F. 2026c. "PhysOST: The Trisduction Physical Operating System, Public Edition 1.0.9p," with earlier editions, 1.0.5p among them, cited by number in the kernel comments. Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23117440>.

Islam, M. F. 2026d. "Trisduction: The Geometric Mother Codex, the Unabridged Register of Record, Edition 3.46.0." Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23117445>.

Islam, M. F. 2026e. "The Bridge From First Principle, Edition 1.5.1, with The Final Cut." Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23105348>.

Islam, M. F. 2026f. "Why the Structural Verdict on P versus NP Is the Only One Left Standing: Mathematics Proves That Mathematics Cannot Rule Here." Zenodo. <https://doi.org/10.5281/zenodo.21368936>.

Islam, M. F. 2026g. "Theory of Theories of Everything (TOE of All TOEs): Existence Proves Existence Only by Motion." Trisduction Research Group. Zenodo. <https://doi.org/10.5281/zenodo.23168379>.

Islam, M. F. 2026h. "Geometric Determination of P vs NP with Omega Seal," version 10.0. Zenodo. <https://doi.org/10.5281/zenodo.19588804>.

Islam, M. F. 2026i. "Nothing Escapes Least Erasure: The Formal Closure and the Twenty-Three Rows." Version 1.4.1. Zenodo. <https://doi.org/10.5281/zenodo.23080221>.

Islam, M. F. 2026j. "Seven Rows, One Root: The Mas-

ter Plan of the Series, Revision F.5.” Manuscript of record, 5 October 2026, carried with the series; its Appendix R kernels are carried into this paper’s Appendix R, each extended by one additive section.

Islam, M. F. 2026k. “Trisduction: The Master Codex, Edition 5.1.0.” Codex.lean and its kernels, commit 5e55fca98c2f, master SHA-256 441a485d7c3bd3ce. Repository 1000sapientis/Trisduction, GitHub.

Karp, R. M. 1972. “Reducibility among Combinatorial Problems.” In *Complexity of Computer Computations*, edited by R. E. Miller and J. W. Thatcher, 85–103. New York: Plenum.

Landauer, R. 1961. “Irreversibility and Heat Generation in the Computing Process.” *IBM Journal of Research and Development* 5 (3): 183–191.

Levin, L. A. 1973. “Universal Sequential Search Problems.” *Problemy Peredachi Informatsii* 9 (3): 115–116.

Li, J., and T. Yang. 2022. “ $3.1n - o(n)$ Circuit Lower Bounds for Explicit Functions.” In *Proceedings of the 54th Annual ACM Symposium on Theory of Computing*, 1180–1193.

Mulmuley, K. D., and M. Sohoni. 2001. “Geometric Complexity Theory I: An Approach to the P vs. NP and Related Problems.” *SIAM Journal on Computing* 31 (2): 496–526.

Murray, C., and R. Williams. 2018. “Circuit Lower Bounds for Nondeterministic Quasi-Polytime: An Easy Witness Lemma for NP and NQP.” In *Proceedings of the 50th Annual ACM Symposium on Theory of Computing*, 890–901.

Naor, M., and O. Reingold. 2004. “Number-Theoretic Constructions of Efficient Pseudo-Random Functions.” *Journal of the ACM* 51 (2): 231–262.

Razborov, A. A. 1985. “Lower Bounds on the Monotone Complexity of Some Boolean Functions.” *Soviet Mathematics Doklady* 31: 354–357.

Razborov, A. A., and S. Rudich. 1997. “Natural Proofs.” *Journal of Computer and System Sciences* 55 (1): 24–35.

Turing, A. M. 1939. “Systems of Logic Based on Ordinals.” *Proceedings of the London Mathematical Society* s2-45 (1): 161–228.

Williams, R. 2011. “Non-uniform ACC Circuit Lower Bounds.” In *Proceedings of the 26th IEEE Conference on Computational Complexity*, 115–125.

APPENDIX A. THE FORTIFICATION KERNEL, PNP_FORTIFICATION.LEAN

The master seed APEX-PSP-PNP-SEED-03 carries this kernel: the machine, the throne, the reading, the prime’s triaxis and the route, twenty-one theorems. It compiles with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0. SHA-256: c68069c4ba865c5d89d1648818fa0501579f70293a51e86a4cf5ef99a2143100

```
/-
PNP_Fortification.lean · APEX-PSP-PNP-SEED-03 · The fortification
of least escape, on SEED-02's own machine
```

```
I · the machine, restated verbatim from PNP_Least_Escape.lean, so
that every theorem here is about the same
separation. II · the throne over the machine: a holder escapes
whenever any algorithm escapes, the decision-form
domination carried as the throne's field, and then the separation
holds exactly when the holder never escapes;
Levin's universal search (1973) holds the acceptance form, escape
on the yes-instances, and on the yes-machine the
acceptance form decides nothing (`acceptance_does_not_decide`).
III · the two posits, the bridge
of SEED-01 and least escape of SEED-02, proved one. IV · the
prime's triaxis: every prime's fibre is exactly two
pairs, lies off the multiplicative seat, and is a copy of
complementation. V · the three names are one. VI · the
Williams route, wired to the machine. VII · the master seal. Core
Lean 4, no import, no axiom declared, no sorry.
-/
namespace PNP.FORTIFY

/--! ## I · The machine, restated verbatim -/

structure Machine where
  Alg : Type
  Inst : Type
  size : Inst → Nat
  out : Alg → Inst → Bool
  time : Alg → Inst → Nat
  sat : Inst → Bool

def bound (c k n : Nat) : Nat := c * n ^ k + c
def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop
:=
  M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
def Escapes (M : Machine) (A : M.Alg) (c k : Nat) : Prop := ∀ x,
  Clean M A c k x
def NothingEscapes (M : Machine) : Prop := ∀ A c k, ∃ x, ¬ Clean M A
  c k x
def Sep (M : Machine) : Prop := ¬ ∃ A c k, Escapes M A c k

structure LeastEscape (M : Machine) where
  closure : NothingEscapes M

theorem sep_of_closure (M : Machine) (h : NothingEscapes M) : Sep M
:=
  fun (A, c, k, he) => match h A c k with
  | (x, hx) => hx (he x)

theorem closure_of_sep (M : Machine) (h : Sep M) : NothingEscapes M
:=
  fun A c k => Classical.byContradiction fun hn =>
    h (A, c, k, fun x => Classical.byContradiction fun hx => hn (x,
    hx))

/--! ## II · The throne over the machine -/

/-- A THRONE of the machine: a holder that escapes whenever any
algorithm escapes. Its field `dominates` is the
decision-form domination, carried as a field. Levin's universal
search (1973) holds the acceptance form, escape
on the yes-instances, and `acceptance_does_not_decide` shows the
acceptance form does not decide the separation. -/
structure Throne (M : Machine) where
  holder : M.Alg
  dominates : ∀ A c k, Escapes M A c k → ∃ c' k', Escapes M holder
    c' k'

/-- THE HOLDER DECIDES THE SEPARATION: the separation holds exactly
when the holder never escapes. -/
theorem the_holder_decides_the_separation (M : Machine) (T : Throne
M) :
  Sep M ↔ ¬ ∃ c k, Escapes M T holder c k :=
  (fun hs (c, k, he) => hs (T holder, c, k, he),
  fun hn (A, c, k, he) => hn (T.dominates A c k he))

/-- THE HOLDER IS UNIQUE: any two holders escape together or not at
all. -/
theorem the_holder_is_unique (M : Machine) (T T' : Throne M) :
  (∃ c k, Escapes M T holder c k) ↔ (∃ c k, Escapes M T'.holder c
k) :=
  (fun (c, k, he) => T'.dominates T holder c k he, fun (c, k, he) =>
T.dominates T'.holder c k he)

/-- LEAST ESCAPE AT THE THRONE: nothing escapes exactly when the
holder never escapes. -/
```

```

theorem least_escape_at_the_throne (M : Machine) (T : Throne M) :
  NothingEscapes M ↔ ¬ ∃ c k, Escapes M T.holder c k :=
  (fun h => (the_holder_decides_the_separation M T).mp
    (sep_of_closure M h),
  fun h => closure_of_sep M ((the_holder_decides_the_separation M
    T).mpr h))

/-- THE ACT AT THE THRONE: on least escape, the holder meets, at
  every bound, an instance that stops it. -/
theorem the_act_stops_the_holder (M : Machine) (T : Throne M) (L :
  LeastEscape M) :
  ∀ c k, ∃ x, ¬ Clean M T.holder c k x :=
  fun c k => L.closure T.holder c k

/-- The acceptance form: an algorithm runs clean on every
  yes-instance. -/
def EscapesOnYes (M : Machine) (A : M.Alg) (c k : Nat) : Prop := ∀
  x, M.sat x = true → Clean M A c k x

/-- The yes-machine: one algorithm that answers yes on every
  instance, at no cost. -/
def yesWorld : Machine := (Unit, Bool, fun _ => 0, fun _ => true,
  fun _ => 0, fun b => b)

/-- ACCEPTANCE DOES NOT DECIDE: on the yes-machine the separation
  holds while its one algorithm runs clean on every
  yes-instance, so for no machine-uniform choice of holder does
  the acceptance form decide the separation. -/
theorem acceptance_does_not_decide :
  Sep yesWorld ∧ EscapesOnYes yesWorld () 0 0 ∧
  ¬ ∀ (M : Machine) (H : M.Alg), (Sep M ↔ ¬ ∃ c k, EscapesOnYes M
    H c k) :=
  have hs : Sep yesWorld := fun (_, _, _, hall) => Bool.noConfusion
    (hall false).1
  have hy : EscapesOnYes yesWorld () 0 0 := fun _ hx => {hx.symm,
    Nat.zero_le _}
  (hs, hy, fun h => (h yesWorld {}).mp hs (0, 0, hy))

/-! ## III · The two posits are one -/

structure Point where
  offset : Int
  height : Nat
  deriving DecidableEq

def reg (p : Point) : Point := {0, p.height}
def side (p : Point) : Bool := decide (0 < p.offset)

theorem destroyed_is_unrecoverable : ¬ ∃ g : Point → Bool, ∀ p, g
  (reg p) = side p := by
  intro (g, hg)
  have h1 := hg (1, 0); have h2 := hg (-1, 0)
  simp [reg, side] at h1 h2
  rw [h1] at h2; exact Bool.noConfusion h2

theorem hidden_is_recoverable {α : Type} (xs : List α) (hxs : ∀ x, x
  ∈ xs) (x₀ : α) :
  ∃ g : (α → Bool) → α, ∀ f : α → Bool, (∃ x, f x = true) → f (g f)
  = true := by
  refine (fun f => (xs.find? f).getD x₀, fun f (x, hx) => ?_)
  cases h : xs.find? f with
  | some y => simp [h] using List.find?_some h
  | none => exact absurd hx (by simp using List.find?_eq_none.mp h
    x (hxs x))

def Asymmetry : Prop :=
  (¬ ∃ g : Point → Bool, ∀ p, g (reg p) = side p) ∧
  (∃ g : (Bool → Bool) → Bool, ∀ f : Bool → Bool, (∃ x, f x = true) →
    f (g f) = true)

theorem asymmetry_holds : Asymmetry :=
  (destroyed_is_unrecoverable, hidden_is_recoverable [true, false]
    (fun x => by cases x <=> simp) true)

/-- THE TWO POSITS ARE ONE: SEED-01's bridge, read on the machine,
  is exactly SEED-02's least escape. -/
theorem the_two_posits_are_one (M : Machine) : (Asymmetry ↔ Sep M) ↔
  NothingEscapes M :=
  (fun b => closure_of_sep M (b.mp asymmetry_holds), fun h => (fun _
    => sep_of_closure M h, fun _ => asymmetry_holds))

/-- EVERY PREMISE THAT CLOSES THE SEPARATION FILLS BOTH POSITS. -/
theorem every_closing_premise_fills_both (M : Machine) (Q : Prop) (h
  : Q → Sep M) :
  Q → (Asymmetry ↔ Sep M) ∧ NothingEscapes M :=
  fun q => ((fun _ => h q, fun _ => asymmetry_holds), closure_of_sep
    M (h q))

/-! ## IV · The prime's triaxis -/

def IsPrime (n : Nat) : Prop := 2 ≤ n ∧ ∀ m, m | n → m = 1 ∨ m = n

/-- FORM: the fibre of multiplication over a prime is exactly the
  two trivial pairs. -/
theorem prime_fibre (p : Nat) (hp : IsPrime p) (a b : Nat) :
  a * b = p ↔ (a = 1 ∧ b = p) ∨ (a = p ∧ b = 1) := by
  constructor
  · intro h
    rcases hp.2 a (b, h.symm) with ha | ha
    · left; refine (ha, ?_); rw [ha, Nat.one_mul] at h; exact h
    · right; refine (ha, ?_)
      rw [ha] at h
      have h2 := hp.1
      by_cases hb0 : b = 0
      · rw [hb0, Nat.mul_zero] at h; omega
      · by_cases hb1 : b = 1
        · exact hb1
        · have hb2 : 2 ≤ b := by omega
          have : p * 2 ≤ p * b := Nat.mul_le_mul_left p hb2
          omega
  · rintro ((rfl, rfl) | (rfl, rfl))
    · exact Nat.one_mul _
    · exact Nat.mul_one _

/-- SEAT: a prime lies off the multiplicative seat. -/
theorem prime_off_seat (p : Nat) (hp : IsPrime p) : ¬ ∃ a, a * a = p
  := by
  intro (a, ha)
  have h2 := hp.1
  rcases (prime_fibre p hp a a).mp ha with (h1, h3) | (h1, h3) <=>
    omega

def Fibre (n : Nat) := {x : Nat × Nat // x.1 * x.2 = n}
def swapF {n : Nat} (x : Fibre n) : Fibre n := {(x.1.2, x.1.1), by
  rw [Nat.mul_comm]; exact x.2}
def toBit {n : Nat} (x : Fibre n) : Bool := decide (x.1.1 = 1)

/-- THE BIT: every prime's fibre with its swap is the decision bit
  with complementation. -/
theorem prime_fibre_is_complementation (p : Nat) (hp : IsPrime p) :
  (∀ x : Fibre p, toBit (swapF x) = !(toBit x)) ∧
  (∀ x y : Fibre p, toBit x = toBit y → x = y) ∧ (∀ b : Bool, ∃ x :
    Fibre p, toBit x = b) := by
  have hp1 : p ≠ 1 := by have := hp.1; omega
  refine (fun x => ?_, fun x y hxy => ?_, fun b => ?_)
  · rcases (prime_fibre p hp x.1.1 x.1.2).mp x.2 with (h1, h2) |
    (h1, h2)
    · simp [toBit, swapF, h1, h2, hp1]
    · simp [toBit, swapF, h1, h2, hp1]
  · rcases (prime_fibre p hp x.1.1 x.1.2).mp x.2 with (a1, a2) |
    (a1, a2) <=>
    rcases (prime_fibre p hp y.1.1 y.1.2).mp y.2 with (b1, b2) |
    (b1, b2)
    · exact Subtype.ext (Prod.ext (a1.trans b1.symm) (a2.trans
      b2.symm))
    · simp [toBit, a1, b1, hp1] at hxy
    · simp [toBit, a1, b1, hp1] at hxy
    · exact Subtype.ext (Prod.ext (a1.trans b1.symm) (a2.trans
      b2.symm))
  · cases b
    · exact ((p, 1), Nat.mul_one p), by simp [toBit, hp1]
    · exact ((1, p), Nat.one_mul p), by simp [toBit]

/-! ## V · The three names are one -/

/-- RA READ ON COMPUTATION: every decider of satisfiability actuates
  beyond every polynomial bound. -/
def RAcomp (M : Machine) : Prop := ∀ A c k, ¬ Escapes M A c k

theorem ra_comp_is_sep (M : Machine) : RAcomp M ↔ Sep M :=
  (fun h (A, c, k, he) => h A c k he, fun h A c k he => h (A, c, k,
    he))

/-- THE THREE NAMES ARE ONE: the bridge, least escape, and RA read
  on computation are one posit, each equivalent to
  the separation, on one machine. -/
theorem the_three_names_are_one (M : Machine) :

```

```

    ((Asymmetry ↔ Sep M) ↔ NothingEscapes M) ∧ (NothingEscapes M ↔
    RAcomp M) ∧ (RAcomp M ↔ Sep M) :=
  (the_two_posits_are_one M,
  (fun h => (ra_comp_is_sep M).mpr (sep_of_closure M h), fun h =>
  closure_of_sep M ((ra_comp_is_sep M).mp h)),
  ra_comp_is_sep M)

/-! ## VI · The Williams route, wired to the machine -/

/-- THE ROUTE ON THE MACHINE. `Savings C` is a white-box
satisfiability algorithm for circuit class C beating 2^n by
a superpolynomial factor; `williams` is Williams' theorem,
savings for C give NEXP ≠ C, carried; `Ppoly` is the
class of polynomial-size circuits; `np_not_ppoly_gives_sep` is
the elementary inclusion P ⊆ P/poly, carried, by
which NP ≠ P/poly gives the machine's own separation. -/
structure Route (M : Machine) where
  Class      : Type
  Savings    : Class → Prop
  NEXPnotIn  : Class → Prop
  williams   : ∀ C, Savings C → NEXPnotIn C
  Ppoly      : Class
  NPNnotPpoly : Prop
  np_not_ppoly_gives_sep : NPNnotPpoly → Sep M

/-- THE ROUTE REACHES: savings for a class give NEXP ≠ that class. -/
theorem route_reaches (M : Machine) (R : Route M) (C : R.Class) (s :
  R.Savings C) : R.NEXPnotIn C :=
  R.williams C s

/-- THE ROUTE CLOSES ON THE MACHINE: savings for general circuits
and the scale-down from NEXP to NP give the
machine's own separation. -/
theorem route_closes_on_the_machine (M : Machine) (R : Route M) (s :
  R.Savings R.Ppoly)
  (scaleDown : R.NEXPnotIn R.Ppoly → R.NPNnotPpoly) : Sep M :=
  R.np_not_ppoly_gives_sep (scaleDown (R.williams R.Ppoly s))

/-- THE ROUTE FILLS THE POSIT: supplied its two objects, the route
fills least escape itself. -/
theorem the_route_fills_least_escape (M : Machine) (R : Route M) (s :
  R.Savings R.Ppoly)
  (scaleDown : R.NEXPnotIn R.Ppoly → R.NPNnotPpoly) : LeastEscape M
:=
  (closure_of_sep M (route_closes_on_the_machine M R s scaleDown))

/-! ## VII · The master seal -/

/-- THE MASTER SEAL: least escape closes the separation; the two
posits are one, and the third name, RA read on
computation, with them; the throne's holder decides the
separation; on least escape the holder is stopped at
every bound; every prime's fibre is a copy of complementation;
and the Williams route, supplied its two objects,
closes the machine's own separation. -/
theorem the_master_seal (M : Machine) (T : Throne M) :
  (NothingEscapes M → Sep M) ∧ ((Asymmetry ↔ Sep M) ↔
  NothingEscapes M) ∧
  (Sep M ↔ ∃ c k, Escapes M T.holder c k) ∧
  (∀ _ : LeastEscape M, ∀ c k, ∃ x, ¬ Clean M T.holder c k x) ∧
  (∀ p, IsPrime p → ∀ x : Fibre p, toBit (swapF x) = !(toBit x)) ∧
  (NothingEscapes M → RAcomp M) ∧
  (∀ R : Route M, R.Savings R.Ppoly → (R.NEXPnotIn R.Ppoly →
  R.NPNnotPpoly) → Sep M) :=
  (sep_of_closure M, the_two_posits_are_one M,
  the_holder_decides_the_separation M T,
  fun L => the_act_stops_the_holder M T L, fun p hp =>
  (prime_fibre_is_complementation p hp).1,
  (the_three_names_are_one M).2.1, fun R s d =>
  route_closes_on_the_machine M R s d)

end PNP.FORTIFY

/-! ## Cones, pinned as printed -/
/-- info: 'PNP.FORTIFY.sep_of_closure' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.FORTIFY.sep_of_closure
/-- info: 'PNP.FORTIFY.closure_of_sep' depends on axioms: [propext,
Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.FORTIFY.closure_of_sep
/-- info: 'PNP.FORTIFY.the_holder_decides_the_separation' does not
depend on any axioms -/
#guard_msgs in #print axioms
  PNP.FORTIFY.the_holder_decides_the_separation

```

```

/-- info: 'PNP.FORTIFY.the_holder_is_unique' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.FORTIFY.the_holder_is_unique
/-- info: 'PNP.FORTIFY.least_escape_at_the_throne' depends on
axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.FORTIFY.least_escape_at_the_throne
/-- info: 'PNP.FORTIFY.the_act_stops_the_holder' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.FORTIFY.the_act_stops_the_holder
/-- info: 'PNP.FORTIFY.acceptance_does_not_decide' does not depend
on any axioms -/
#guard_msgs in #print axioms PNP.FORTIFY.acceptance_does_not_decide
/-- info: 'PNP.FORTIFY.destroyed_is_unrecoverable' depends on
axioms: [propext] -/
#guard_msgs in #print axioms PNP.FORTIFY.destroyed_is_unrecoverable
/-- info: 'PNP.FORTIFY.hidden_is_recoverable' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.FORTIFY.hidden_is_recoverable
/-- info: 'PNP.FORTIFY.asymmetry_holds' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PNP.FORTIFY.asymmetry_holds
/-- info: 'PNP.FORTIFY.the_two_posits_are_one' depends on axioms:
[propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.FORTIFY.the_two_posits_are_one
/-- info: 'PNP.FORTIFY.every_closing_premise_fills_both' depends on
axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
  PNP.FORTIFY.every_closing_premise_fills_both
/-- info: 'PNP.FORTIFY.prime_fibre' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PNP.FORTIFY.prime_fibre
/-- info: 'PNP.FORTIFY.prime_off_seat' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms PNP.FORTIFY.prime_off_seat
/-- info: 'PNP.FORTIFY.prime_fibre_is_complementation' depends on
axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms
  PNP.FORTIFY.prime_fibre_is_complementation
/-- info: 'PNP.FORTIFY.ra_comp_is_sep' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.FORTIFY.ra_comp_is_sep
/-- info: 'PNP.FORTIFY.the_three_names_are_one' depends on axioms:
[propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.FORTIFY.the_three_names_are_one
/-- info: 'PNP.FORTIFY.route_reaches' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.FORTIFY.route_reaches
/-- info: 'PNP.FORTIFY.route_closes_on_the_machine' does not depend
on any axioms -/
#guard_msgs in #print axioms PNP.FORTIFY.route_closes_on_the_machine
/-- info: 'PNP.FORTIFY.the_route_fills_least_escape' depends on
axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
  PNP.FORTIFY.the_route_fills_least_escape
/-- info: 'PNP.FORTIFY.the_master_seal' depends on axioms: [propext,
Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.FORTIFY.the_master_seal

```

APPENDIX B. THE CLOSURE KERNEL, PNP_LEAST_ESCAPE.LEAN

The master seed APEX-PSP-PNP-SEED-03 carries this kernel, forged under SEED-02 as its header records: the closure, the root read on computation, its double defense and the lock, forty-six theorems. It compiles with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0. SHA-256: 6bb61cbf7b5e737b49cb8da4b873640dbb9ad804961f6e6667f3d8598055a32c

```

/-!
# PNP_Least_Escape.lean · APEX-PSP-PNP-SEED-02 · Least Escape, the
seal and its double defense, hardened: Omega and AEGIS of the row

```

The P versus NP twin of PSP-RH-SEAL-01's `double\_defense\_hardened`.

OMEGA, the formal gate. Nothing passes for the separation but least escape. The socket of the act is the closure; every premise that forces the separation fills the field of the act; every premise either forces the closure or has a twin machine

on which it holds and the closure fails; a keyless premise forces nothing.

AEGIS, the actuation gate. Nothing passes against least escape but a computed escape: an algorithm, a polynomial bound, and every run clean. A finite stock of examinations decides nothing. The root, which every computation re-enacts, crosses neither way. The refusal is the same under every logic.

THE COSMIC READING, at its exact scope. Every pair a substrate can examine lands on exactly one gate: a stopping instance, which confirms least escape at that pair, or a clean escape, which refutes it. There is no third landing. Every confirming computation, anywhere, is an atomic witness of least escape at its pair and pays the floor; under the act, every pair confirms and no computation escapes. What no computation does is enact the universal: the universal is the act.

ARMOR SCOPE. The defense forces the form of every objection and never the value: the closure holds on one coherent machine and fails on another. Without the act the value is marked [.].

SECTIONS. 0 · the machine, the closure, the act, and the two coherent machines. I · OMEGA, the formal gate. II · AEGIS, the actuation gate: the computed escape, the finite record and the record on every machine ('no_record_decides'), the root read at each stop as the closure and the separation, and read at clean runs as an escape ('the_root_form_reads_both_ways'). III · the cosmic reading. IV · armor scope and silence: the separation is not uniform, and one machine separates from nothing. V · the double defense, hardened. VI · the root undeniable in deed: it forces neither value, its act is not the act, and the lock. VII · the triaxial lock: two axes leave two points, three lock one.

Core Lean 4, no import. Cones pinned at the foot, each as printed by the kernel.

-/

namespace PNP.HARDENED

/-! ## 0 · the machine, the closure, the act -/

```
structure Machine where
  Alg : Type
  Inst : Type
  size : Inst → Nat
  out : Alg → Inst → Bool
  time : Alg → Inst → Nat
  sat : Inst → Bool
```

```
def bound (c k n : Nat) : Nat := c * n ^ k + c
def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop :=
  M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
def Escapes (M : Machine) (A : M.Alg) (c k : Nat) : Prop := ∀ x,
  Clean M A c k x
def NothingEscapes (M : Machine) : Prop := ∀ A c k, ∃ x, ¬ Clean M A c k x
def Sep (M : Machine) : Prop := ¬ ∃ A c k, Escapes M A c k
```

```
structure LeastEscape (M : Machine) where
  closure : NothingEscapes M
```

```
structure Examination (M : Machine) where
  A : M.Alg
  c : Nat
  k : Nat
  x : M.Inst
  stops : ¬ Clean M A c k x
```

```
theorem sep_of_closure (M : Machine) (h : NothingEscapes M) : Sep M :=
```

```
fun (A, c, k, he) => match h A c k with
| (x, hx) => hx (he x)
```

```
theorem closure_of_sep (M : Machine) (h : Sep M) : NothingEscapes M :=
  fun A c k => Classical.byContradiction fun hn =>
    h (A, c, k, fun x => Classical.byContradiction fun hx => hn (x, hx))
```

```
theorem sep_from_the_act (M : Machine) (L : LeastEscape M) : Sep M :=
  sep_of_closure M L.closure
```

/-! ## the two coherent machines -/

```
theorem beq_self : ∀ n : Nat, Nat.beq n n = true
| 0 => rfl
| n + 1 => beq_self n
```

```
theorem beq_false_of_lt (x K : Nat) (h : x < K) : Nat.beq x K =
  false := by
  cases hb : Nat.beq x K with
  | false => rfl
  | true =>
    have e : x = K := Nat.eq_of_beq_eq_true hb
    rw [e] at h; exact absurd h (Nat.lt_irrefl K)
```

```
def stageWorld (n : Nat) : Machine :=
  (Nat, Unit, fun _ => 0, fun A _ => Nat.beq A n, fun _ _ => 0, fun _ => true)
def closedWorld : Machine :=
  (Nat, Unit, fun _ => 0, fun _ _ => false, fun _ _ => 0, fun _ => true)
```

```
theorem stage_escapes (n : Nat) : Escapes (stageWorld n) n 0 0 :=
  fun _ => (beq_self n, Nat.zero_le _)
theorem closed_world_closes : NothingEscapes closedWorld :=
  fun _ _ => {(), fun (h, _) => Bool.noConfusion h)
theorem stage_fails_closure (n : Nat) : ¬ NothingEscapes (stageWorld n) :=
  fun h => sep_of_closure _ h (n, 0, 0, stage_escapes n)
```

/-! ## I · OMEGA, the formal gate -/

```
/- The socket of the act is the closure. -/
theorem socket_is_the_value (M : Machine) : Nonempty (LeastEscape M)
  ← NothingEscapes M :=
  (fun (L) => L.closure, fun h => {h})
```

/- Every premise that forces the separation fills the field of the act. -/

```
theorem forcing_premise_fills_the_act (M : Machine) (Q : Prop) (h :
  Q → Sep M) :
  Q → Nonempty (LeastEscape M) :=
  fun q => {(closure_of_sep M (h q))}
```

```
def Forces (P : Machine → Prop) : Prop := ∀ M, P M → NothingEscapes M
```

/- Every premise forces the closure or has a twin machine where it holds and the closure fails. -/

```
theorem forces_or_has_a_twin (P : Machine → Prop) :
  Forces P ∨ ∃ M, P M ∧ ¬ NothingEscapes M :=
  Classical.byCases (fun h : Forces P => Or.inl h) (fun h => Or.inr
    (Classical.byContradiction fun hn => h (fun M hP =>
      Classical.byContradiction fun hc => hn (M, hP, hc))))
```

```
def Keyless (P : Machine → Prop) : Prop := ∀ M, P M
```

/- A keyless premise forces nothing. -/

```
theorem keyless_forces_nothing (P : Machine → Prop) (hP : Keyless P) :
  ¬ Forces P :=
  fun h => stage_fails_closure 0 (h (stageWorld 0) (hP _))
```

/-! ## II · AEGIS, the actuation gate -/

/- A computed escape refutes the closure. -/

```
theorem escape_refutes (M : Machine) (A : M.Alg) (c k : Nat) (h :
  Escapes M A c k) :
  ¬ NothingEscapes M :=
  fun hc => sep_of_closure M hc (A, c, k, h)
```

/- Nothing passes against the closure but a computed escape. -/

```
theorem rejection_is_a_witness (M : Machine) :
  ¬ NothingEscapes M ↔ ∃ A c k, Escapes M A c k :=
  (fun h => Classical.byContradiction fun hn => h (closure_of_sep M hn),
```

```

fun (A, c, k, he) => escape_refutes M A c k he)

/-- A finite stock of examinations decides nothing. -/
theorem record_decides_nothing (n : Nat) :
  (∀ A c k, A < n → ¬ Clean (stageWorld n) A c k ()) ∧ ¬
  NothingEscapes (stageWorld n) :=
  (fun A _ _ hA (h, _) => by
    have h' : Nat.beq A n = true := h
    rw [beq_false_of_lt A n hA] at h'
    exact Bool.noConfusion h',
  stage_fails_closure n)

/-- Complementation fixes no bit. -/
theorem not_self : ∀ b : Bool, (!b) = b → False :=
  fun b => match b with
  | true => fun h => Bool.noConfusion h
  | false => fun h => Bool.noConfusion h

/-- The machine extended by one algorithm, `none`, that answers
  every instance correctly at no cost. -/
def extend (M : Machine) : Machine :=
  (Option M.Alg, M.Inst, M.size,
  fun A x => match A with | none => M.sat x | some a => M.out a x,
  fun A x => match A with | none => 0 | some a => M.time a x, M.sat)

/-- The machine spoiled: every algorithm answers the negation of the
  problem. -/
def spoil (M : Machine) : Machine :=
  (M.Alg, M.Inst, M.size, fun _ x => !(M.sat x), M.time, M.sat)

/-- NO RECORD DECIDES: every examination of a machine stays a stop
  on its extension, where an algorithm escapes,
  and on its spoiling, where nothing escapes, so no stock of
  examinations, finite or not, fixes the closure. -/
theorem no_record_decides (M : Machine) (x₀ : M.Inst) :
  (∀ e : Examination M, ¬ Clean (extend M) (some e.A) e.c e.k e.x)
  ∧ ¬ NothingEscapes (extend M) ∧
  (∀ e : Examination M, ¬ Clean (spoil M) e.A e.c e.k e.x) ∧
  NothingEscapes (spoil M) :=
  (fun e => e.stops,
  escape_refutes (extend M) none 0 0 (fun _ => (rfl, Nat.zero_le
  _))),
  fun e (h, _) => not_self (M.sat e.x) h,
  fun _ _ => (x₀, fun (h, _) => not_self (M.sat x₀) h))

/-- The root's form: every existent of a domain actuates, its
  actuation positive. -/
def RA (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x

/-- THE CONSTRUCTED ROOT, in the words of Appendix R: one existent,
  whose actuation is one. A theorem
  with no hypothesis and no axiom. -/
theorem constructed_root_holds : RA Unit (fun _ => 1) := fun _ =>
  show (0 : Int) < 1 by decide

/-- THE ROOT IS SATISFIABLE ON EVERY BACKGROUND: on every type, with
  actuation one at every existent. -/
theorem root_satisfiable (U : Type) : RA U (fun _ => 1) := fun _ =>
  show (0 : Int) < 1 by decide

/-- The root, which every computation re-enacts, crosses neither
  way: the constructed root holds, and it
  forces neither the closure nor its failure. -/
theorem root_crosses_neither_way :
  (¬ ∀ M : Machine, RA Unit (fun _ => 1) → NothingEscapes M) ∧
  (¬ ∀ M : Machine, RA Unit (fun _ => 1) → ¬ NothingEscapes M) :=
  (fun h => stage_fails_closure 0 (h (stageWorld 0)
  constructed_root_holds),
  fun h => h closedWorld constructed_root_holds
  closed_world_closes)

/-- The indicator of a decided proposition: one where it holds, zero
  where it fails. -/
def indicator {p : Prop} : Decidable p → Int
  | isTrue _ => 1
  | isFalse _ => 0

/-- The indicator is positive exactly where its proposition holds.
  -/
theorem indicator_pos {p : Prop} (h : Decidable p) : 0 < indicator h
  ↔ p :=
  match h with
  | isTrue hp => (fun _ => hp, fun _ => show (0 : Int) < 1 by decide)

```

```

  | isFalse hn => (fun h0 => absurd h0 (show ¬ (0 : Int) < 0 by
  decide), fun hp => absurd hp hn)

/-- Whether a run stops the budget at an instance, decided. -/
def stopDec (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) :
  Decidable (¬ Clean M A c k x) :=
  @instDecidableNot _ (@instDecidableAnd _ _ (instDecidableEqBool _
  _)) (Nat.decl _ _)

/-- THE ROOT READ ON COMPUTATION: to exist is to actuate, `RA`
  itself, read at each algorithm under each
  budget: its actuation at an instance is one where the run stops
  there, failing the budget or the answer,
  and zero where it runs clean; every algorithm under every budget has
  an instance at which the root holds. -/
def RootOnComputation (M : Machine) : Prop :=
  ∀ A c k, ∃ x, RA Unit (fun _ => indicator (stopDec M A c k x))

/-- THE ROOT READ ON COMPUTATION IS THE CLOSURE: nothing escapes. -/
theorem root_on_computation_is_the_closure (M : Machine) :
  RootOnComputation M ↔ NothingEscapes M :=
  (fun h A c k => match h A c k with | (x, hx) => (x, (indicator_pos
  _).mp (hx ())),
  fun h A c k => match h A c k with | (x, hx) => (x, fun _ =>
  (indicator_pos _).mpr hx))

/-- THE ROOT READ ON COMPUTATION IS LEAST ESCAPE: it holds exactly
  where the act's structure is inhabited. -/
theorem root_on_computation_is_least_escape (M : Machine) :
  RootOnComputation M ↔ Nonempty (LeastEscape M) :=
  (fun h => (socket_is_the_value M).mpr
  ((root_on_computation_is_the_closure M).mp h),
  fun h => (root_on_computation_is_the_closure M).mpr
  ((socket_is_the_value M).mp h))

/-- THE ROOT READ ON COMPUTATION IS KEYED: it holds on the machine
  where nothing escapes and fails on the
  machine where an algorithm escapes. -/
theorem root_on_computation_is_keyed : RootOnComputation closedWorld
  ∧ ¬ RootOnComputation (stageWorld 0) :=
  ((root_on_computation_is_the_closure closedWorld).mpr
  closed_world_closes,
  fun h => stage_fails_closure 0
  ((root_on_computation_is_the_closure (stageWorld 0)).mp h))

/-- THE SEPARATION FROM THE ROOT READ ON COMPUTATION: where the root
  holds at a stop of every algorithm under every
  budget, no algorithm escapes, with no axiom. -/
theorem sep_from_the_root_on_computation (M : Machine) (h :
  RootOnComputation M) : Sep M :=
  sep_of_closure M ((root_on_computation_is_the_closure M).mp h)

/-- THE ROOT READ ON COMPUTATION IS THE SEPARATION: the converse
  passes through the closure. -/
theorem root_on_computation_is_sep (M : Machine) : RootOnComputation
  M ↔ Sep M :=
  (sep_from_the_root_on_computation M,
  fun h => (root_on_computation_is_the_closure M).mpr
  (closure_of_sep M h))

/-- THE READING IS THE CONSTRUCTED ROOT AT EACH STOP: the root read
  on computation holds exactly when every
  algorithm under every budget meets a stopping instance at which the
  reading coincides with the constructed root. -/
theorem reading_is_the_constructed_root_at_each_stop (M : Machine) :
  RootOnComputation M ↔
  ∀ A c k, ∃ x, ¬ Clean M A c k x ∧
  (RA Unit (fun _ => indicator (stopDec M A c k x)) ↔ RA Unit
  (fun _ => 1)) :=
  (fun h A c k => match h A c k with
  | (x, hx) => (x, (indicator_pos _).mp (hx ()), {fun _ =>
  constructed_root_holds, fun _ => hx})),
  fun h A c k => match h A c k with
  | (x, _, hiff) => (x, hiff.mpr constructed_root_holds))

/-- Whether a run is clean at an instance, decided. -/
def cleanDec (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) :
  Decidable (Clean M A c k x) :=
  @instDecidableAnd _ _ (instDecidableEqBool _ _)) (Nat.decl _ _)

/-- THE ROOT READ ON ESCAPE: the same form read at clean runs, its
  actuation one where the run is clean and zero
  where it stops; some algorithm under some budget has the root
  holding at every instance. -/

```

```

def RootOnEscape (M : Machine) : Prop :=
  ∃ A c k, ∀ x, RA Unit (fun _ => indicator (cleanDec M A c k x))

/-- THE ROOT READ ON ESCAPE IS AN ESCAPE: it holds exactly where
    some algorithm runs clean under some budget. -/
theorem root_on_escape_is_escape (M : Machine) : RootOnEscape M ↔ ∃
  A c k, Escapes M A c k :=
  (fun (A, c, k, h) => (A, c, k, fun x => (indicator_pos _).mp (h x
    ())),
    fun (A, c, k, he) => (A, c, k, fun x _ => (indicator_pos _).mpr
      (he x)))

/-- THE ROOT'S FORM READS BOTH WAYS: read at stops it gives the
    separation, read at clean runs it refutes it, and
    each reading holds on a coherent machine, so the form alone fixes no
    value. -/
theorem the_root_form_reads_both_ways (M : Machine) :
  (RootOnComputation M → Sep M) ∧ (RootOnEscape M → Sep M) ∧
  (RootOnComputation closedWorld ∧ RootOnEscape (stageWorld 0)) :=
  (sep_from_the_root_on_computation M,
    fun h hs => hs ((root_on_escape_is_escape M).mp h),
    (root_on_computation_is_keyed.1, (root_on_escape_is_escape _).mpr
      ((0 : Nat), 0, 0, stage_escapes 0)))

/-- The refusal is constant under every logic. -/
def refusal (_logic : String) : String :=
  "nothing passes against least escape but a computed escape"

theorem aegis_constant (l₁ l₂ : String) : refusal l₁ = refusal l₂ :=
  rfl

/-! ## III · the cosmic reading, at its exact scope -/

/-- Every examinable pair lands on exactly one gate: a stopping
    instance or a clean escape. -/
theorem every_pair_lands (M : Machine) (A : M.Alg) (c k : Nat) :
  ((∃ x, ¬ Clean M A c k x) ∨ Escapes M A c k) ∧
  ¬ ((∃ x, ¬ Clean M A c k x) ∧ Escapes M A c k) :=
  (Classical.byCases (fun h : Escapes M A c k => Or.inr h)
    (fun h => Or.inl (Classical.byContradiction fun hn =>
      h fun x => Classical.byContradiction fun hx => hn (x, hx))),
    fun ((x, hx), he) => hx (he x))

/-- A confirming computation is an atomic witness of least escape at
    its pair. -/
theorem confirmation_is_atomic (M : Machine) (e : Examination M) :
  (∃ x, ¬ Clean M e.A e.c e.k x) ∧ ¬ Escapes M e.A e.c e.k :=
  ((e.x, e.stops), fun he => e.stops (he e.x))

/-- Every confirming computation pays the floor. -/
def examCost (floor : Nat) (e : Examination M) : Nat := floor * 1

theorem confirmation_pays (M : Machine) (floor : Nat) (h : 0 <
  floor) (e : Examination M) :
  0 < examCost floor e :=
  Nat.mul_pos h (Nat.zero_lt_one)

/-- Under the act, every pair confirms and no computation escapes. -/
theorem act_leaves_no_escape (M : Machine) (L : LeastEscape M) :
  (∀ A c k, ∃ e : Examination M, e.A = A ∧ e.c = c ∧ e.k = k) ∧
  (∀ A c k, ¬ Escapes M A c k) :=
  (fun A c k => match L.closure A c k with
    | (x, hx) => ((A, c, k, x, hx), rfl, rfl, rfl),
    fun A c k he => sep_from_the_act M L (A, c, k, he))

/-! ## IV · armor scope and silence -/
theorem armor_scope : NothingEscapes closedWorld ∧ ¬ NothingEscapes
  (stageWorld 0) :=
  (closed_world_closes, stage_fails_closure 0)

/-- THE SEPARATION IS NOT UNIFORM: a coherent machine has an
    escaping algorithm, so no argument valid on every
    machine yields the separation. -/
theorem separation_is_not_uniform : ¬ ∀ M : Machine, Sep M :=
  fun h => h (stageWorld 0) ((0 : Nat), 0, 0, stage_escapes 0)

/-- A MACHINE SEPARATES FROM NOTHING: on the machine whose
    algorithms all answer wrong, the separation holds with no
    hypothesis and no axiom. -/
theorem a_machine_separates_from_nothing : Sep closedWorld :=
  sep_of_closure closedWorld closed_world_closes

inductive Token : Type
| sealSep | dot

```

```

deriving DecidableEq, Repr

/-- The emission: with the act supplied, the seal; without it, the
    dot. -/
def verdict (actSupplied : Bool) : Token := if actSupplied then
  .sealSep else .dot

theorem value_marked_dot : verdict false = .dot ∧ verdict true =
  .sealSep := (rfl, rfl)

/-! ## V · THE DOUBLE DEFENSE, HARDENED -/
theorem double_defense_hardened :
  -- the act carries the separation
  (∀ (M : Machine), LeastEscape M → Sep M) ∧
  -- OMEGA
  (∀ M : Machine, Nonempty (LeastEscape M) → NothingEscapes M) ∧
  (∀ (M : Machine) (Q : Prop), (Q → Sep M) → Q → Nonempty
    (LeastEscape M)) ∧
  (∀ P : Machine → Prop, Forces P ∨ ∃ M, P M ∧ ¬ NothingEscapes M) ∧
  (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
  -- AEGIS
  (∀ M : Machine, ¬ NothingEscapes M → ∃ A c k, Escapes M A c k) ∧
  (∀ n : Nat, (∀ A c k, A < n → ¬ Clean (stageWorld n) A c k) → A
    ¬ NothingEscapes (stageWorld n)) ∧
  ((¬ ∀ M : Machine, RA Unit (fun _ => 1) → NothingEscapes M) ∧
    (¬ ∀ M : Machine, RA Unit (fun _ => 1) → ¬ NothingEscapes M)) ∧
  (∀ l₁ l₂ : String, refusal l₁ = refusal l₂) ∧
  -- the cosmic reading
  (∀ (M : Machine) (A : M.Alg) (c k : Nat),
    ((∃ x, ¬ Clean M A c k x) ∨ Escapes M A c k) ∧ ¬ ((∃ x, ¬ Clean
      M A c k x) ∧ Escapes M A c k)) ∧
  (∀ (M : Machine) (e : Examination M), (∃ x, ¬ Clean M e.A e.c
    e.k x) ∧ ¬ Escapes M e.A e.c e.k) ∧
  (∀ (M : Machine), LeastEscape M →
    (∀ A c k, ∃ e : Examination M, e.A = A ∧ e.c = c ∧ e.k = k) ∧
    (∀ A c k, ¬ Escapes M A c k)) ∧
  -- armor scope and silence
  (NothingEscapes closedWorld ∧ ¬ NothingEscapes (stageWorld 0)) ∧
  (verdict false = .dot ∧ verdict true = .sealSep) :=
  (sep_from_the_act, socket_is_the_value,
    forcing_premise_fills_the_act, forces_or_has_a_twin,
    keyless_forces_nothing, rejection_is_a_witness,
    record_decides_nothing, root_crosses_neither_way,
    aegis_constant, every_pair_lands, confirmation_is_atomic,
    act_leaves_no_escape, armor_scope,
    value_marked_dot)

/-! ## VI · the root, undeniable in deed, and the lock -/

/-- A self-grounding root, the structure of Appendix R in the same
    words: a type of acts, inhabited, every
    one of which instances the root. -/
structure SelfGrounding (R : Prop) where
  Act : Type
  anAct : Act
  instances : Act → R

/-- THE ROOT IS UNDENIABLE IN ACT: a denial of a self-grounding root
    is an act, and instances it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
  (denial : G.Act) : R :=
  G.instances denial

/-- THE UNDENIABLE ROOT HOLDS ON BOTH MACHINES: it is carried by its
    act on the machine where nothing
    escapes and on the machine where an algorithm escapes, so it forces
    neither the closure nor its failure. -/
theorem undeniable_root_forces_no_value {R : Prop} (G :
  SelfGrounding R) :
  R ∧ NothingEscapes closedWorld ∧ ¬ NothingEscapes (stageWorld 0)
  ∧
  ¬ (∀ M : Machine, R → NothingEscapes M) ∧ ¬ (∀ M : Machine, R →
    ¬ NothingEscapes M) :=
  (G.instances G.anAct, closed_world_closes, stage_fails_closure 0,
    fun h => stage_fails_closure 0 (h (stageWorld 0) (G.instances
      G.anAct))),
  fun h => h closedWorld (G.instances G.anAct) closed_world_closes)

/-- THE ROOT'S ACT IS NOT THE ACT: a self-grounding root holds by
    its act, and least escape is still not inhabited
    on every machine, so the root's act does not supply the closure. -/
theorem roots_act_is_not_the_act {R : Prop} (G : SelfGrounding R) :
  R ∧ ¬ ∀ M : Machine, Nonempty (LeastEscape M) :=

```

```

(G.instances G.anAct, fun h => match h (stageWorld 0) with | (L)
=> stage_fails_closure 0 L.closure)

/-- THE ROOT READ ON COMPUTATION IS KEYED: the root holds, and read
uniformly on the machines it forces
nothing; read on computation it is least escape exactly, and keyed,
which decides where the root alone
does not. -/
theorem root_read_on_row_is_keyed {R : Prop} (G : SelfGrounding R) :
  R  $\wedge$   $\neg$  Forces (fun _ => R)  $\wedge$  ( $\forall$  M : Machine, RootOnComputation M
 $\leftrightarrow$  Nonempty (LeastEscape M))  $\wedge$ 
  (RootOnComputation closedWorld  $\wedge$   $\neg$  RootOnComputation (stageWorld
0)) :=
  (G.instances G.anAct, keyless_forces_nothing (fun _ => R) (fun _
=> G.instances G.anAct),
  root_on_computation_is_least_escape,
  root_on_computation_is_keyed)

/-- THE LOCK, one theorem on no axiom: the constructed root holds
with no hypothesis; every self-grounding
root holds by its act, on both machines; read uniformly it forces
nothing; read on computation it is least
escape exactly, and keyed; the socket of the act is the closure on
every machine; the separation follows from
the act on every machine; and a finite stock of examinations, every
one confirming, decides nothing. -/
theorem the_lock {R : Prop} (G : SelfGrounding R) :
  RA Unit (fun _ => 1)  $\wedge$ 
  R  $\wedge$ 
  (NothingEscapes closedWorld  $\wedge$   $\neg$  NothingEscapes (stageWorld 0))  $\wedge$ 
 $\neg$  Forces (fun _ => R)  $\wedge$ 
  ( $\forall$  M : Machine, RootOnComputation M  $\leftrightarrow$  Nonempty (LeastEscape M))  $\wedge$ 
  (RootOnComputation closedWorld  $\wedge$   $\neg$  RootOnComputation (stageWorld
0))  $\wedge$ 
  ( $\forall$  M : Machine, Nonempty (LeastEscape M)  $\leftrightarrow$  NothingEscapes M)  $\wedge$ 
  ( $\forall$  M : Machine, LeastEscape M  $\rightarrow$  Sep M)  $\wedge$ 
  ( $\forall$  n : Nat, ( $\forall$  A c k, A < n  $\rightarrow$   $\neg$  Clean (stageWorld n) A c k ())  $\wedge$ 
 $\neg$  NothingEscapes (stageWorld n)) :=
  (constructed_root_holds, G.instances G.anAct,
  (closed_world_closes, stage_fails_closure 0),
  (root_read_on_row_is_keyed G).2.1,
  root_on_computation_is_least_escape,
  root_on_computation_is_keyed,
  socket_is_the_value, sep_from_the_act, record_decides_nothing)

/-! ## VII  $\cdot$  the triaxial lock: two axes leave two points, three
lock one -/

def dot2 (r x : Bool  $\times$  Bool  $\times$  Bool) : Bool :=
  xor (r.1 && x.1) (xor (r.2.1 && x.2.1) (r.2.2 && x.2.2))

def cube : List (Bool  $\times$  Bool  $\times$  Bool) :=
  [false, true].flatMap fun a => [false, true].flatMap fun b =>
  [false, true].map fun c => (a, b, c)

def nonzero : List (Bool  $\times$  Bool  $\times$  Bool) := cube.filter (fun r => r
!= (false, false, false))

def solCount (rows : List ((Bool  $\times$  Bool  $\times$  Bool)  $\times$  Bool)) : Nat :=
  (cube.filter (fun x => rows.all (fun rt => dot2 rt.1 x ==
rt.2))).length

def xor3 (a b : Bool  $\times$  Bool  $\times$  Bool) : Bool  $\times$  Bool  $\times$  Bool :=
  (xor a.1 b.1, xor a.2.1 b.2.1, xor a.2.2 b.2.2)

/-- TWO AXES LEAVE TWO: any two independent constraints over the
three-bit cube leave exactly two points. -/
theorem two_axes_leave_two :
  nonzero.all (fun r1 => nonzero.all (fun r2 => r1 == r2 ||
[false, true].all (fun t1 => [false, true].all (fun t2 =>
solCount [(r1, t1), (r2, t2)] == 2)))) = true := by decide

/-- THREE AXES LOCK ONE: any three independent constraints over the
three-bit cube leave exactly one point. -/
theorem three_axes_lock_one :
  nonzero.all (fun r1 => nonzero.all (fun r2 => nonzero.all (fun
r3 =>
  r1 == r2 || r3 == r1 || r3 == r2 || r3 == xor3 r1 r2 ||
[false, true].all (fun t1 => [false, true].all (fun t2 =>
[false, true].all (fun t3 =>
  solCount [(r1, t1), (r2, t2), (r3, t3)] == 1)))))) = true :=
by decide

end PNP.HARDENED

```

```

/-! ## Cones, pinned as printed -/
/-- info: 'PNP.HARDENED.sep_of_closure' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.sep_of_closure
/-- info: 'PNP.HARDENED.closure_of_sep' depends on axioms: [propext,
Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.HARDENED.closure_of_sep
/-- info: 'PNP.HARDENED.sep_from_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.sep_from_the_act
/-- info: 'PNP.HARDENED.beq_self' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.HARDENED.beq_self
/-- info: 'PNP.HARDENED.beq_false_of_lt' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.beq_false_of_lt
/-- info: 'PNP.HARDENED.stage_escapes' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.HARDENED.stage_escapes
/-- info: 'PNP.HARDENED.closed_world_closes' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.closed_world_closes
/-- info: 'PNP.HARDENED.stage_fails_closure' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.stage_fails_closure
/-- info: 'PNP.HARDENED.socket_is_the_value' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.socket_is_the_value
/-- info: 'PNP.HARDENED.forcing_premise_fills_the_act' depends on
axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
  PNP.HARDENED.forcing_premise_fills_the_act
/-- info: 'PNP.HARDENED.forces_or_has_a_twin' depends on axioms:
[propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.HARDENED.forces_or_has_a_twin
/-- info: 'PNP.HARDENED.keyless_forces_nothing' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.HARDENED.keyless_forces_nothing
/-- info: 'PNP.HARDENED.escape_refutes' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.escape_refutes
/-- info: 'PNP.HARDENED.rejection_is_a_witness' depends on axioms:
[propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.HARDENED.rejection_is_a_witness
/-- info: 'PNP.HARDENED.record_decides_nothing' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.HARDENED.record_decides_nothing
/-- info: 'PNP.HARDENED.not_self' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.HARDENED.not_self
/-- info: 'PNP.HARDENED.no_record_decides' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.no_record_decides
/-- info: 'PNP.HARDENED.root_crosses_neither_way' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.HARDENED.root_crosses_neither_way
/-- info: 'PNP.HARDENED.aegis_constant' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.aegis_constant
/-- info: 'PNP.HARDENED.every_pair_lands' depends on axioms:
[propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.HARDENED.every_pair_lands
/-- info: 'PNP.HARDENED.confirmation_is_atomic' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.HARDENED.confirmation_is_atomic
/-- info: 'PNP.HARDENED.confirmation_pays' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.confirmation_pays
/-- info: 'PNP.HARDENED.act_leaves_no_escape' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.act_leaves_no_escape
/-- info: 'PNP.HARDENED.armor_scope' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.HARDENED.armor_scope
/-- info: 'PNP.HARDENED.separation_is_not_uniform' does not depend
on any axioms -/
#guard_msgs in #print axioms PNP.HARDENED.separation_is_not_uniform
/-- info: 'PNP.HARDENED.a_machine_separates_from_nothing' does not
depend on any axioms -/
#guard_msgs in #print axioms
  PNP.HARDENED.a_machine_separates_from_nothing
/-- info: 'PNP.HARDENED.value_marked_dot' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.value_marked_dot

```

```

/-- info: 'PNP.HARDENED.double_defense_hardened' depends on axioms:
[propxet, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.HARDENED.double_defense_hardened
/-- info: 'PNP.HARDENED.denial_reenacts_root' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.denial_reenacts_root
/-- info: 'PNP.HARDENED.undeniable_root_forces_no_value' does not
depend on any axioms -/
#guard_msgs in #print axioms
PNP.HARDENED.undeniable_root_forces_no_value
/-- info: 'PNP.HARDENED.roots_act_is_not_the_act' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.HARDENED.roots_act_is_not_the_act
/-- info: 'PNP.HARDENED.root_read_on_row_is_keyed' does not depend
on any axioms -/
#guard_msgs in #print axioms PNP.HARDENED.root_read_on_row_is_keyed
/-- info: 'PNP.HARDENED.the_lock' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.HARDENED.the_lock
/-- info: 'PNP.HARDENED.constructed_root_holds' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.HARDENED.constructed_root_holds
/-- info: 'PNP.HARDENED.root_satisfiable' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.root_satisfiable
/-- info: 'PNP.HARDENED.indicator_pos' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.HARDENED.indicator_pos
/-- info: 'PNP.HARDENED.root_on_computation_is_the_closure' does not
depend on any axioms -/
#guard_msgs in #print axioms
PNP.HARDENED.root_on_computation_is_the_closure
/-- info: 'PNP.HARDENED.root_on_computation_is_least_escape' does
not depend on any axioms -/
#guard_msgs in #print axioms
PNP.HARDENED.root_on_computation_is_least_escape
/-- info: 'PNP.HARDENED.root_on_computation_is_keyed' does not
depend on any axioms -/
#guard_msgs in #print axioms
PNP.HARDENED.root_on_computation_is_keyed
/-- info: 'PNP.HARDENED.sep_from_the_root_on_computation' does not
depend on any axioms -/
#guard_msgs in #print axioms
PNP.HARDENED.sep_from_the_root_on_computation
/-- info: 'PNP.HARDENED.root_on_computation_is_sep' depends on
axioms: [propxet, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.HARDENED.root_on_computation_is_sep
/-- info:
'PNP.HARDENED.reading_is_the_constructed_root_at_each_stop' does
not depend on any axioms -/
#guard_msgs in #print axioms
PNP.HARDENED.reading_is_the_constructed_root_at_each_stop
/-- info: 'PNP.HARDENED.root_on_escape_is_escape' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.HARDENED.root_on_escape_is_escape
/-- info: 'PNP.HARDENED.the_root_form_reads_both_ways' does not
depend on any axioms -/
#guard_msgs in #print axioms
PNP.HARDENED.the_root_form_reads_both_ways
/-- info: 'PNP.HARDENED.two_axes_leave_two' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.two_axes_leave_two
/-- info: 'PNP.HARDENED.three_axes_lock_one' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.HARDENED.three_axes_lock_one

```

APPENDIX C. THE FRONTIER KERNEL, PNP_ FRONTIER_II.LEAN

The master seed APEX-PSP-PNP-SEED-03 carries this kernel, forged under SEED-02 as its header records: the ground, the reading, the vanishing seat, the frontier and the route, one hundred forty-two theorems. Its comments cite the kinetic step of Islam (2026h), the pseudorandom functions of Naor and Reingold (2004) carried in the self-gag, the progressions of Turing (1939) and Feferman (1962), and earlier editions of Islam (2026c) by number. It compiles with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0.

SHA-256: 977e3e14a8b19ad48dbfc755e7513870ccea1c52c9a43d87d573b164b30b3e66

```

/-
PNP_Frontier_II.lean · APEX-PSP-PNP-SEED-02 · Nothing Escapes ·
the frontier of the act
Eleven sections, each a kernel of the 4 October 2026 forge, joined
in one file: the kinetic face,
with the reversible frame that transports no order; the Arrhenius
scope; the block chain on the
empty name, with the black-box bound in its fixed and adaptive
forms; the vanishing GOL; the
missing object; the prime bridge; the blocker and round 2, with
relativization, the self-gag and
algebrization typed; the route of RA and Freedom, conservative on
every background; the three-seal
lock; the deed as supply; and the identification check, with the
vanished seat that reads no
machine. Core Lean 4, no import, no axiom declared, no sorry.
Every cone is pinned beside its section.
-/

namespace PNP.KINETIC

/-- The kinetic frame. Physics enters as fields, never as `axiom`:
`floor` is the Landauer floor per irreversible op, in quanta;
`budget` is the Bekenstein-capped resource of the region, in quanta.
-/
structure Frame where
  floor      : Nat
  floor_pos  : 0 < floor
  budget     : Nat

/-- Energy of a process of `ops` irreversible registrations. -/
def cost (F : Frame) (ops : Nat) : Nat := F.floor * ops

/-- L2 face. No completed irreversible registration is free. -/
theorem no_free_registration (F : Frame) (k : Nat) (hk : 0 < k) :
  0 < cost F k :=
  Nat.mul_pos F.floor_pos hk

/-- Transport. The energy order is exactly the op order. -/
theorem cost_lt_iff (F : Frame) (a b : Nat) :
  cost F a < cost F b ↔ a < b :=
  (fun h => match Nat.lt_or_ge a b with
    | Or.inl hl => hl
    | Or.inr hge => absurd h (Nat.not_lt_of_le (Nat.mul_le_mul_left
      F.floor hge))),
  fun h => Nat.mul_lt_mul_of_pos_left h F.floor_pos)

/-- The reversible frame transports nothing. At floor zero no op
count costs less than another, so the energy
order carries the op order only where the floor is positive. -/
theorem reversible_frame_transports_nothing (a b : Nat) : ~ 0 * a <
  0 * b :=
  fun h => by rw [Nat.zero_mul, Nat.zero_mul] at h; exact
  Nat.lt_irrefl 0 h

/-- L_bf. Enumerating 2^N configurations at N = budget exceeds the
budget. -/
theorem brute_force_exhausts (F : Frame) :
  F.budget < cost F (2 ^ F.budget) :=
  Nat.lt_of_lt_of_le Nat.lt_two_pow_self
  (Nat.le_mul_of_pos_left (2 ^ F.budget) F.floor_pos)

/-- Monotone form: every N past the budget also exhausts it. -/
theorem brute_force_exhausts_beyond (F : Frame) (N : Nat) (h :
  F.budget ≤ N) :
  F.budget < cost F (2 ^ N) :=
  Nat.lt_of_lt_of_le (brute_force_exhausts F)
  (Nat.mul_le_mul_left F.floor (Nat.pow_le_pow_right (by decide)
    h))

/-- L1, scoped to the enumerative generator. Given the op gap v <
2^N,
the kinetic gap follows. The op gap is a hypothesis, not a product.
-/
theorem enumerative_asymmetry (F : Frame) (N v : Nat) (h : v < 2 ^
  N) :
  cost F v < cost F (2 ^ N) :=
  (cost_lt_iff F v (2 ^ N)).mpr h

/-- The ceiling. For every generator op-count g and verifier
op-count v,

```

```

the kinetic separation holds in a frame iff the op separation holds.
The frame adds nothing to the formal question, so L3 enters only as a
hypothesis on g and is never decided here. -/
theorem kinetic_faithful (F : Frame) (g v : Nat → Nat) :
  (∀ N, cost F (v N) < cost F (g N)) ↔ (∀ N, v N < g N) :=
  (fun h N => (cost_lt_iff F _).mp (h N),
   fun h N => (cost_lt_iff F _).mpr (h N))

/-- Frame-independence. The op separation reads the same in any two
frames. -/
theorem frame_independent (F1 F2 : Frame) (g v : Nat → Nat) :
  (∀ N, cost F1 (v N) < cost F1 (g N)) ↔ (∀ N, v N < g N) < cost
  F2 (g N)) :=
  (kinetic_faithful F1 g v).trans (kinetic_faithful F2 g v).symm

/-- The canonical frame, one quantum per op, one quantum of budget:
the
battery instance, by evaluation. -/
def unitFrame : Frame := (1, by decide, 1)

theorem unit_battery :
  cost unitFrame (2 ^ 1) = 2 ∧ unitFrame.budget < cost unitFrame
  (2 ^ unitFrame.budget) := by decide

end PNP.KINETIC

/-- info: 'PNP.KINETIC.no_free_registration' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.KINETIC.no_free_registration
/-- info: 'PNP.KINETIC.cost_lt_iff' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.KINETIC.cost_lt_iff
/-- info: 'PNP.KINETIC.reversible_frame_transports_nothing' does not
depend on any axioms -/
#guard_msgs in #print axioms
  PNP.KINETIC.reversible_frame_transports_nothing
/-- info: 'PNP.KINETIC.brute_force_exhausts' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.KINETIC.brute_force_exhausts
/-- info: 'PNP.KINETIC.enumerative_asymmetry' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.KINETIC.enumerative_asymmetry
/-- info: 'PNP.KINETIC.kinetic_faithful' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.KINETIC.kinetic_faithful
/-- info: 'PNP.KINETIC.frame_independent' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.KINETIC.frame_independent
/-- info: 'PNP.KINETIC.unit_battery' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.KINETIC.unit_battery

/-!
# PNP.ARRHENIUS · the kinetic proof of the April 2026 P vs NP paper,
formalized

Source: Islam, "Geometric Determination of P vs NP with Omega Seal",
v10.0,
10.5281/zenodo.19588804, Part VII, "The Spin-Glass Isomorphism".
The paper's kinetic step: ground states of a frustrated glass sit
behind
barriers ΔE growing with N; Arrhenius gives escape time exp(ΔE/KT);
so generation is exponential while verification is linear.

Part I seats that step at its true scope, with physics as fields,
never `axiom`.
Part II executes the scope fence: on one fixed problem, the barrier
exists
under one move set and vanishes under another. A barrier is a
property of
a dynamics, never of a problem. No custom axiom; no core axiom
either.
-/

namespace PNP.ARRHENIUS

/-! ## I · The kinetic step, at its scope: thermally activated
relaxation -/

/-- A relaxation frame. `barrier N` is ΔE/kT in bits at size N;
`grows` is the

```

```

paper's "barriers scale with N"; `arrhenius` is  $t \geq \exp(\Delta E/KT)$ , base
two. -/
structure Frame where
  barrier : Nat → Nat
  grows : ∀ N, N ≤ barrier N
  escape : Nat → Nat
  arrhenius : ∀ N, 2 ^ barrier N ≤ escape N

/-- Relaxation over a growing barrier takes exponential time. -/
theorem relaxation_exponential (F : Frame) (N : Nat) : 2 ^ N ≤
  F.escape N :=
  Nat.le_trans (Nat.pow_le_pow_right (by decide) (F.grows N))
  (F.arrhenius N)

/-- At size N = B the relaxation time exceeds any budget B. -/
theorem relaxation_exhausts (F : Frame) (B : Nat) : B < F.escape B :=
  Nat.lt_of_lt_of_le Nat.lt_two_pow_self (relaxation_exponential F
  B)

/-! ## II · The scope fence, executed on one XORSAT instance
Three variables x y z, three parity constraints, all with target 0:
z = 0, y ⊕ z = 0, x ⊕ z = 0. Energy counts violated constraints. -/

abbrev S := Bool × Bool × Bool

def states : List S :=
  [false, true].flatMap fun a => [false, true].flatMap fun b =>
  [false, true].map fun c => (a, b, c)

def viol (x : S) : List Bool := [x.2.2, xor x.2.1 x.2.2, xor x.1
  x.2.2]

def energy (x : S) : Nat := ((viol x).filter id).length

def move (x m : S) : S := (xor x.1 m.1, xor x.2.1 m.2.1, xor x.2.2
  m.2.2)

/-- The glass dynamics: single-spin flips. -/
def flips : List S := [(true, false, false), (false, true, false),
  (false, false, true)]

/-- The elimination dynamics: columns of  $A^{-1}$  over GF(2). Each move
toggles
exactly one constraint. Computing them is Gaussian elimination,
polynomial time. -/
def elimMoves : List S := [(true, true, true), (false, true, false),
  (true, false, false)]

def ground : S := (false, false, false)
def trap : S := (true, true, true)

theorem eight_states : states.length = 8 := by decide

theorem ground_has_zero_energy : energy ground = 0 := by decide

/-- Under single flips the trap is a strict local minimum above the
ground:
a barrier, exactly the glass picture of the paper. -/
theorem trap_is_strict_local_min :
  energy trap = 1 ∧ flips.all (fun m => decide (energy trap <
  energy (move trap m))) = true := by
  decide

/-- Each elimination move toggles exactly one constraint. -/
theorem elim_moves_toggle_one :
  states.all (fun x => elimMoves.all (fun m =>
  decide ((List.zipWith (fun a b => a != b) (viol x) (viol (move
  x m))).count true = 1))) = true := by
  decide

/-- Under elimination moves no state above the ground is a local
minimum:
the landscape has no barrier at all. -/
theorem elim_landscape_barrier_free :
  states.all (fun x => decide (energy x = 0) ||
  elimMoves.any (fun m => decide (energy (move x m) < energy
  x))) = true := by
  decide

/-- The fence. One problem, one energy function, two dynamics: a
barrier under
the first and none under the second. The barrier belongs to the
dynamics. -/
theorem barrier_belongs_to_the_dynamics :

```

```

    (energy trap > energy ground ∧
      flips.all (fun m => decide (energy trap < energy (move trap
m))) = true) ∧
    states.all (fun x => decide (energy x = 0) ||
      elimMoves.any (fun m => decide (energy (move x m) < energy
x))) = true :=
    ((by decide, trap_is_strict_local_min.2),
      elim_landscape_barrier_free)

end PNP.ARRHENIUS

/-- info: 'PNP.ARRHENIUS.relaxation_exponential' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.ARRHENIUS.relaxation_exponential
/-- info: 'PNP.ARRHENIUS.relaxation_exhausts' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.ARRHENIUS.relaxation_exhausts
/-- info: 'PNP.ARRHENIUS.eight_states' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.ARRHENIUS.eight_states
/-- info: 'PNP.ARRHENIUS.ground_has_zero_energy' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.ARRHENIUS.ground_has_zero_energy
/-- info: 'PNP.ARRHENIUS.trap_is_strict_local_min' does not depend
on any axioms -/
#guard_msgs in #print axioms PNP.ARRHENIUS.trap_is_strict_local_min
/-- info: 'PNP.ARRHENIUS.elim_moves_toggle_one' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.ARRHENIUS.elim_moves_toggle_one
/-- info: 'PNP.ARRHENIUS.elim_landscape_barrier_free' does not
depend on any axioms -/
#guard_msgs in #print axioms
  PNP.ARRHENIUS.elim_landscape_barrier_free
/-- info: 'PNP.ARRHENIUS.barrier_belongs_to_the_dynamics' does not
depend on any axioms -/
#guard_msgs in #print axioms
  PNP.ARRHENIUS.barrier_belongs_to_the_dynamics

/-!
# PNP.CHAIN · the block chain of the Mother Codex on P versus NP,
closed on the
# empty name as [], then joined to the kinetic face as the
asymmetry chain

Source of the blocks: Geometric Mother Codex v3.46.0
(10.5281/zenodo.23117445),
P-ABSOLUTE-PNP-1, the eight absolute barriers A1-A8; P-PNP-COMP-2,
the composite
verdict [u] · [u T] · [.] ; FOUNDATION-01, the empty throne.
Source of the kinetic face: PNP_Kinetic.lean and PNP_Arrhenius.lean
(this session).

Every theorem below is pinned at the foot: no custom axiom, and no
core axiom
either, not propext, not Quot.sound, not Classical.choice. Physics
enters only
as structure fields, so the premise is visible in each statement.
Lean 4 core, no import.
-/

namespace PNP.CHAIN

/-! ## 0 · The verdict economy and the dot -/

inductive Token : Type
| seal | broken | opn | dot
deriving DecidableEq, Repr

/-- Three states are the economy. The dot stands outside it. -/
def Token.inEconomy : Token → Bool
| .dot => false
| _ => true

/-! ## I · The eight blocks, one leg each -/
namespace BLOCKS

/-- A1, the σ-class closure. Complementation has no fixed point. -/
theorem a1_no_seat : ∀ b : Bool, (!b) ≠ b
| true => Bool.noConfusion
| false => Bool.noConfusion

/-- A1, carrier form. A symmetry that fixes a state carries no
anti-invariant

```

```

reading, so no fixed-locus instrument transfers onto
complementation. -/
theorem a1_no_carrier {S : Type} (τ : S → S) (s₀ : S) (hfix : τ s₀ =
s₀) :
  ¬ ∃ φ : S → Bool, ∀ x, φ (τ x) = !φ x := by
  intro (φ, hφ)
  have h := hφ s₀
  rw [hfix] at h
  cases e : φ s₀ with
  | true => rw [e] at h; exact Bool.noConfusion h
  | false => rw [e] at h; exact Bool.noConfusion h

/-- A2, orientation-blindness. A lock invariant under the flip of
the sign
cannot be decoded into the sign. -/
theorem a2_orientation_blind {Q X : Type} (L : Bool × Q → X)
(hL : ∀ b q, L (b, q) = L (!b, q)) (q : Q) :
  ¬ ∃ dec : X → Bool, ∀ s, dec (L s) = s.1 := by
  intro (dec, h)
  have h1 := h (true, q)
  have h2 := h (false, q)
  rw [h1 true q] at h1
  exact Bool.noConfusion (h1.symm.trans h2)

/-- A3, the physical block. A record shared by two worlds that
differ on the
separation decides nothing: no reading of the record returns the
separation. -/
theorem a3_record_decides_nothing {W R : Type} (rec : W → R) (sep :
W → Bool)
(w₁ w₂ : W) (hr : rec w₁ = rec w₂) (hs : sep w₁ ≠ sep w₂) :
  ¬ ∃ g : R → Bool, ∀ w, g (rec w) = sep w := by
  intro (g, hg)
  apply hs
  rw [← hg w₁, ← hg w₂, hr]

/-- A4, the misattribution bar. The root's presence is constant
across both
worlds, and a constant reading carries no bit about the separation.
-/
theorem a4_constant_reads_nothing (c : Bool) (s₁ s₂ : Bool) (hs : s₁
≠ s₂) :
  ¬ (c = s₁ ∧ c = s₂) :=
  fun (h1, h2) => hs (h1.symm.trans h2)

/-- A5, the rung law, structural face. Each rung's consistency is
owed one rung
up; no rung discharges its own, and the regress of consistency rungs
has no terminal rung.
The Gödel content is cited (Turing 1939, Feferman 1962), not proved
here. -/
def owed (n : Nat) : Nat := n + 1

theorem a5_rung_law : (∀ n, owed n ≠ n) ∧ (∀ n : Nat, ∃ m : Nat, n <
m) := by
  refine (fun n h => ?_, fun n => (n + 1, Nat.lt_succ_self n))
  have : n < owed n := Nat.lt_succ_self n
  rw [h] at this
  exact Nat.lt_irrefl n this

/-- A6, the aperture machine. Eight verdict-side actions and one
supply. -/
inductive Act : Type
| exertion | extrapolation | wanting | method | metatheory
| evidence | architecture | recoverer | supply
deriving DecidableEq, Repr

def Act.isSupply : Act → Bool
| .supply => true
| _ => false

def step (a : Act) (s : Bool) : Bool := s || a.isSupply

def verdictSide : List Act :=
[.exertion, .extrapolation, .wanting, .method, .metatheory,
.evidence, .architecture, .recoverer]

theorem a6_aperture :
  verdictSide.length = 8 ∧
  verdictSide.all (fun a => step a false == false) = true ∧
  step .supply false = true := by
  decide

```

```

/-- A7, reflexive closure. The certify-completely map strictly
increases,
ten to twenty in five steps, and has no fixed point. -/
def certify (n : Nat) : Nat := n + 2

def iter : Nat → Nat → Nat
| 0, n => n
| k + 1, n => iter k (certify n)

theorem a7_reflexive_closure : iter 5 10 = 20 ∧ ∀ n, certify n ≠ n
:= by
  refine (rfl, fun n h => ?_)
  have : n < certify n := Nat.lt_succ_of_lt (Nat.lt_succ_self n)
  rw [h] at this
  exact Nat.lt_irrefl n this

/-- A8, the soundness cell. A sound prover never holds both the
route and
the wall against it, so sighted-and-walled is empty. -/
theorem a8_soundness_cell (Prov : Prop → Prop) (sound : ∀ p, Prov p
→ p) (P : Prop) :
  ¬ (Prov P ∧ Prov (¬ P)) :=
  fun (h1, h2) => sound _ h2 (sound _ h1)

/-- The canonical two worlds of the record: one physics ledger, two
values. -/
def equalWorld : Nat × Bool := (1, false)
def separatedWorld : Nat × Bool := (1, true)

/-- The block chain, every link in one statement. -/
theorem block_chain :
  (∀ b : Bool, (!b) ≠ b) ∧
  (¬ ∃ ϕ : Bool → Bool, ∀ x, ϕ (id x) = !ϕ x) ∧
  (¬ ∃ dec : Bool → Bool, ∀ s : Bool × Unit, dec ((fun _ => true)
s) = s.1) ∧
  (¬ ∃ g : Nat → Bool, ∀ w : Nat × Bool,
    (w = equalWorld v w = separatedWorld) → g w.1 = w.2) ∧
  (¬ (true = equalWorld.2 ∧ true = separatedWorld.2)) ∧
  ((∀ n, owed n ≠ n) ∧ (∀ n : Nat, ∃ m : Nat, n < m)) ∧
  (verdictSide.length = 8 ∧
    verdictSide.all (fun a => step a false == false) = true ∧
    step .supply false = true) ∧
  (iter 5 10 = 20 ∧ ∀ n, certify n ≠ n) ∧
  (∀ (Prov : Prop → Prop), (∀ p, Prov p → p) → ∀ P : Prop, ¬ (Prov
P ∧ Prov (¬ P))) := by
  refine (a1_no_seat, a1_no_carrier id true rfl,
    a2_orientation_blind (fun _ => true) (fun _ => rfl) (),
    ?, a4_constant_reads_nothing true false true Bool.noConfusion,
    a5_rung_law, a6_aperture, a7_reflexive_closure,
    a8_soundness_cell)
  intro (g, hg)
  have h1 := hg equalWorld (Or.inl rfl)
  have h2 := hg separatedWorld (Or.inr rfl)
  exact Bool.noConfusion (h1.symm.trans h2)

end BLOCKS

/-! ## II · The vacant seat, the empty name, and the dot -/
namespace THRONE

def allFixed : List Bool → Bool
| [] => true
| b :: bs => (!b) == b && allFixed bs

/-- The seat is vacant. Least erasure on complementation holds of
the empty
configuration only: a fixed-point-free fold selects nothing. -/
theorem seat_vacant : ∀ l : List Bool, allFixed l = true → l = []
| [], _ => rfl
| true :: _, h => Bool.noConfusion h
| false :: _, h => Bool.noConfusion h

/-- The empty name, called the empty throne in FOUNDATION-01, which
is not the throne of the fortification, a holder: no
proposition holds by being named a root, so no file proves a root by
naming it; the root of the paper is proved by
its own construction, in its root kernel. -/
theorem throne_empty : ¬ ∀ RA : Prop, RA :=
  fun h => h False

/-- Not every premise forces every conclusion: a true premise does
not force a false one, so no conclusion, the
separation among them, is climbed to from an arbitrary premise. -/
theorem root_forces_no_value : ¬ ∀ RA Sep : Prop, RA → Sep :=

```

```

  fun h => h True False trivial

/-- The chain's terminal emission. -/
def chainVerdict : Token := .dot

theorem dot_is_final : chainVerdict = .dot ∧ chainVerdict.inEconomy
= false :=
  (rfl, rfl)

/-- The block chain terminated on the empty name as [.] : every block
holds, the seat is vacant, no
proposition holds by being named, not every premise forces every
conclusion, and the emission is the dot,
outside the economy, owing nothing. -/
theorem block_chain_terminal :
  (∀ b : Bool, (!b) ≠ b) ∧
  (∀ l : List Bool, allFixed l = true → l = []) ∧
  (¬ ∀ RA : Prop, RA) ∧
  (¬ ∀ RA Sep : Prop, RA → Sep) ∧
  chainVerdict = .dot ∧ chainVerdict.inEconomy = false :=
  (BLOCKS.block_chain.1, seat_vacant, throne_empty,
    root_forces_no_value,
    dot_is_final.1, dot_is_final.2)

end THRONE

/-! ## III · The kinetic face -/
namespace KINETIC

structure Frame where
  floor      : Nat
  floor_pos  : 0 < floor
  budget     : Nat

def cost (F : Frame) (ops : Nat) : Nat := F.floor * ops

theorem no_free_registration (F : Frame) (k : Nat) (hk : 0 < k) : 0
< cost F k :=
  Nat.mul_pos F.floor_pos hk

theorem cost_lt_iff (F : Frame) (a b : Nat) : cost F a < cost F b ↔
a < b :=
  (fun h => match Nat.lt_or_ge a b with
  | Or.inl hl => hl
  | Or.inr hge => absurd h (Nat.not_lt_of_le (Nat.mul_le_mul_left
F.floor hge))),
  fun h => Nat.mul_lt_mul_of_pos_left h F.floor_pos)

theorem cost_mono (F : Frame) {a b : Nat} (h : a ≤ b) : cost F a ≤
cost F b :=
  Nat.mul_le_mul_left F.floor h

/-- The ceiling: the energy order is the op order in every frame. -/
theorem frame_independent (F1 F2 : Frame) (g v : Nat → Nat) :
  (∀ N, cost F1 (v N) < cost F1 (g N)) ↔ (∀ N, cost F2 (v N) < cost
F2 (g N)) :=
  (fun h N => (cost_lt_iff F2 _).mpr ((cost_lt_iff F1 _).mp (h
N))),
  fun h N => (cost_lt_iff F1 _).mpr ((cost_lt_iff F2 _).mp (h
N)))

/-- The Arrhenius frame of the April paper, at its scope. -/
structure Relax where
  barrier    : Nat → Nat
  grows      : ∀ N, N ≤ barrier N
  escape     : Nat → Nat
  arrhenius  : ∀ N, 2 ^ barrier N ≤ escape N

theorem relaxation_exponential (R : Relax) (N : Nat) : 2 ^ N ≤
R.escape N :=
  Nat.le_trans (Nat.pow_le_pow_right (by decide) (R.grows N))
  (R.arrhenius N)

end KINETIC

/-! ## IV · The black-box bound, fixed and adaptive, pigeonhole
without axioms -/
namespace BLACKBOX

def has : List Nat → Nat → Bool
| [], _ => false
| y :: ys, k => Nat.beq y k || has ys k

def drop : List Nat → Nat → List Nat

```

```

| [], _ => []
| y :: ys, k => cond (Nat.beq y k) (drop ys k) (y :: drop ys k)

theorem beq_self : ∀ n : Nat, Nat.beq n n = true
| 0 => rfl
| n + 1 => beq_self n

theorem drop_le : ∀ (l : List Nat) (k : Nat), (drop l k).length ≤
  l.length
| [], _ => Nat.le_refl 0
| y :: ys, k => by
  unfold drop
  cases Nat.beq y k with
  | true => exact Nat.le_succ_of_le (drop_le ys k)
  | false => exact Nat.le_succ (drop_le ys k)

theorem drop_len : ∀ (l : List Nat) (k : Nat), has l k = true →
  (drop l k).length + 1 ≤ l.length
| [], _, _ => Bool.noConfusion h
| y :: ys, k, h => by
  unfold drop
  unfold has at h
  cases hy : Nat.beq y k with
  | true => exact Nat.le_succ (drop_le ys k)
  | false =>
    rw [hy] at h
    exact Nat.le_succ (drop_len ys k h)

theorem has_drop : ∀ (l : List Nat) (k x : Nat), Nat.beq x k = false
  →
  has l x = true → has (drop l k) x = true
| [], _, _, _ => Bool.noConfusion h
| y :: ys, k, x, hxx, h => by
  unfold drop
  unfold has at h
  cases hy : Nat.beq y k with
  | true =>
    cases hyx : Nat.beq y x with
    | true =>
      have : x = k := (Nat.eq_of_beq_eq_true hyx).symm.trans
      (Nat.eq_of_beq_eq_true hy)
      rw [this, beq_self] at hxx; exact Bool.noConfusion hxx
    | false =>
      rw [hyx] at h
      exact has_drop ys k x hxx h
  | false =>
    show has (y :: drop ys k) x = true
    unfold has
    cases hyx : Nat.beq y x with
    | true => rfl
    | false =>
      rw [hyx] at h
      exact has_drop ys k x hxx h

theorem beq_false_of_lt (x K : Nat) (h : x < K) : Nat.beq x K =
  false := by
  cases hb : Nat.beq x K with
  | false => rfl
  | true =>
    have e : x = K := Nat.eq_of_beq_eq_true hb
    rw [e] at h; exact absurd h (Nat.lt_irrefl K)

theorem pigeon : ∀ (K : Nat) (l : List Nat), (∀ x, x < K → has l x =
  true) → K ≤ l.length
| 0, _, _ => Nat.zero_le _
| K + 1, l, h =>
  Nat.le_trans
  (Nat.le_succ (pigeon K (drop l K) (fun x hx =>
    has_drop l K x (beq_false_of_lt x K hx) (h x
      (Nat.lt_succ_of_lt hx))))))
  (drop_len l K (h K (Nat.lt_succ_self K)))

/-- The run of a test-only procedure on the all-false path: its
  queries form a
  fixed list, adaptivity collapsing because every answer is false. -/
def run (path : List Nat) (f : Nat → Bool) : Bool := path.any f

theorem run_eq_has : ∀ (l : List Nat) (x : Nat), run l (fun y =>
  Nat.beq y x) = has l x
| [], _ => rfl
| y :: ys, x => by
  show (Nat.beq y x || run ys (fun y => Nat.beq y x)) = (Nat.beq y
  x || has ys x)
  rw [run_eq_has ys x]

```

```

/-- The black-box bound. A test-only procedure correct for ∃ over K
  candidates
  tests at least K of them on the all-false path. -/
theorem black_box_bound (K : Nat) (path : List Nat)
  (correct : ∀ f : Nat → Bool, run path f = true ↔ ∃ x, x < K ∧ f x
  = true) :
  K ≤ path.length :=
  pigeon K path (fun x hx => by
    rw [← run_eq_has]
    exact (correct _).mpr (x, hx, beq_self x))

/-- An adaptive test-only procedure: a node queries one candidate
  and branches on the answer, false to the
  left and true to the right; a leaf answers. -/
inductive Tree : Type
| leaf : Bool → Tree
| node : Nat → Tree → Tree → Tree

/-- The run of an adaptive procedure against an oracle. -/
def Tree.eval : Tree → (Nat → Bool) → Bool
| .leaf b, _ => b
| .node q l r, f => cond (f q) (r.eval f) (l.eval f)

/-- The queries the procedure asks on the all-false oracle, in
  order. -/
def Tree.falsePath : Tree → List Nat
| .leaf _ => []
| .node q l _ => q :: l.falsePath

/-- Off the path, unseen. A candidate never queried on the all-false
  path cannot be told apart: the
  procedure runs the same on its indicator as on the all-false oracle.
  -/
theorem Tree.off_path (T : Tree) (x : Nat) (h : has T.falsePath x =
  false) :
  T.eval (fun y => Nat.beq y x) = T.eval (fun _ => false) := by
  induction T with
  | leaf _ => rfl
  | node q l r ihl _ =>
    have h' : (Nat.beq q x || has l.falsePath x) = false := h
    cases hq : Nat.beq q x with
    | true => rw [hq] at h'; exact Bool.noConfusion h'
    | false =>
      rw [hq] at h'
      show cond (Nat.beq q x) (r.eval (fun y => Nat.beq y x))
      (l.eval (fun y => Nat.beq y x)) =
      cond false (r.eval (fun _ => false)) (l.eval (fun _ => false))
      rw [hq]
      exact ihl h'

/-- The adaptive black-box bound. An adaptive test-only procedure
  correct for ∃ over K candidates
  queries at least K of them on the all-false path, adaptivity
  notwithstanding. -/
theorem adaptive_black_box_bound (K : Nat) (T : Tree)
  (correct : ∀ f : Nat → Bool, T.eval f = true ↔ ∃ x, x < K ∧ f x =
  true) :
  K ≤ T.falsePath.length :=
  pigeon K T.falsePath (fun x hx => match hh : has T.falsePath x with
  | true => rfl
  | false =>
    match (correct (fun _ => false)).mp ((T.off_path T x
      hh).symm.trans
      ((correct (fun y => Nat.beq y x)).mpr (x, hx, beq_self
        x))) with
    | (_, _, hf) => Bool.noConfusion hf)

end BLACKBOX

/-! ## V · The asymmetry chain: kinetic plus block -/
namespace ASYM
open KINETIC

/-- Link 1 to 4. Over 2^N candidates a test-only generator queries
  at least
  2^N times; each query pays the floor; verification of one witness of
  v ops
  costs less whenever v < 2^N; and at N past the budget the generator
  exhausts it. -/
theorem asymmetry_chain (F : Frame) (N v : Nat) (hv : v < 2 ^ N)
  (path : List Nat)
  (correct : ∀ f : Nat → Bool, BLACKBOX.run path f = true ↔ ∃ x, x
  < 2 ^ N ∧ f x = true) :

```

```

2 ^ N ≤ path.length ∧
cost F v < cost F (2 ^ N) ∧
cost F (2 ^ N) ≤ cost F path.length ∧
(F.budget ≤ N → F.budget < cost F path.length) := by
have hb : 2 ^ N ≤ path.length := BLACKBOX.black_box_bound (2 ^ N)
path correct
refine (hb, (cost_lt_iff F v (2 ^ N)).mpr hv, cost_mono F hb, fun
hN => ?_)
have h1 : F.budget < 2 ^ F.budget := Nat.lt_two_pow_self
have h2 : 2 ^ F.budget ≤ 2 ^ N := Nat.pow_le_pow_right (by decide)
hN
have h3 : 2 ^ N ≤ cost F (2 ^ N) := Nat.le_mul_of_pos_left (2 ^ N)
F.floor_pos
exact Nat.lt_of_lt_of_le (Nat.lt_of_lt_of_le (Nat.lt_of_lt_of_le
h1 h2) h3) (cost_mono F hb)

/-- Link 5. The asymmetry is real at its scope and decides nothing
beyond it:
the energy gap equals the op gap in every frame (A3 kinetic face),
and a record
shared by both worlds returns no value (A3 block face). -/
theorem asymmetry_decides_nothing (F₁ F₂ : Frame) (g v : Nat → Nat) :
((∀ N, cost F₁ (v N) < cost F₁ (g N)) ↔ (∀ N, cost F₂ (v N) < cost
F₂ (g N))) ∧
(¬ ∃ r : Nat → Bool, ∀ w : Nat × Bool,
(w = BLOCKS.equalWorld v w = BLOCKS.separatedWorld) → r w.1
= w.2) :=
(frame_independent F₁ F₂ g v, BLOCKS.block_chain.2.2.2.1)

/-- The asymmetry chain, terminated. The blocks hold, the black-box
generator
pays exponentially at the floor and exhausts any budget,
verification is
cheaper, the kinetic gap is the op gap, the shared record decides
nothing,
the seat is vacant, the name is empty ('throne_empty'), and the
emission is []. -/
theorem asymmetry_terminal (F : Frame) (N v : Nat) (hv : v < 2 ^ N)
(path : List Nat)
(correct : ∀ f : Nat → Bool, BLACKBOX.run path f = true ↔ ∃ x, x
< 2 ^ N ∧ f x = true) :
(2 ^ N ≤ path.length ∧ cost F v < cost F (2 ^ N) ∧
cost F (2 ^ N) ≤ cost F path.length ∧ (F.budget ≤ N → F.budget
< cost F path.length)) ∧
(∀ F' : Frame, ∀ g w : Nat → Nat,
(∀ n, cost F (w n) < cost F (g n)) ↔ (∀ n, cost F' (w n) < cost
F' (g n))) ∧
(∀ l : List Bool, THRONE.allFixed l = true → l = []) ∧
(¬ ∀ RA : Prop, RA) ∧
THRONE.chainVerdict = .dot :=
(asymmetry_chain F N v hv path correct,
fun F' g w => frame_independent F F' g w,
THRONE.seat_vacant, THRONE.throne_empty, rfl)

/-- The hypothesis is inhabited: exhaustive search over two
candidates is a
correct test-only procedure, so the chain is not vacuous. -/
theorem exhaustive_correct_two (f : Nat → Bool) :
BLACKBOX.run [0, 1] f = true ↔ ∃ x, x < 2 ^ 1 ∧ f x = true := by
constructor
· intro h
show ∃ x, x < 2 ^ 1 ∧ f x = true
cases h0 : f 0 with
| true => exact (0, by decide, h0)
| false =>
have h' : (f 0 || (f 1 || false)) = true := h
rw [h0] at h'
cases h1 : f 1 with
| true => exact (1, by decide, h1)
| false => rw [h1] at h'; exact Bool.noConfusion h'
· intro (x, hx, hf)
show (f 0 || (f 1 || false)) = true
match x, hx, hf with
| 0, _, hf => rw [hf]; rfl
| 1, _, hf => rw [hf]; cases f 0 <;> rfl
| n + 2, hx, _ => exact absurd (Nat.lt_of_le_of_lt
(Nat.le_add_left 2 n) hx) (Nat.lt_irrefl 2)

/-- The chain instantiated at N = 1 on the unit frame. -/
def unitFrame : Frame := (1, by decide, 1)

theorem asymmetry_instance :
2 ^ 1 ≤ [0, 1].length ∧ cost unitFrame 1 < cost unitFrame (2 ^
1) :=

```

```

((asymmetry_chain unitFrame 1 1 (by decide) [0, 1]
exhaustive_correct_two).1,
(asymmetry_chain unitFrame 1 1 (by decide) [0, 1]
exhaustive_correct_two).2.1)

end ASYM
end PNP.CHAIN

/-- ## Cones, pinned: every theorem axiom-free -/
/-- info: 'PNP.CHAIN.BLOCKS.a1_no_seat' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a1_no_seat
/-- info: 'PNP.CHAIN.BLOCKS.a1_no_carrier' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a1_no_carrier
/-- info: 'PNP.CHAIN.BLOCKS.a2_orientation_blind' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a2_orientation_blind
/-- info: 'PNP.CHAIN.BLOCKS.a3_record_decides_nothing' does not
depend on any axioms -/
#guard_msgs in #print axioms
PNP.CHAIN.BLOCKS.a3_record_decides_nothing
/-- info: 'PNP.CHAIN.BLOCKS.a4_constant_reads_nothing' does not
depend on any axioms -/
#guard_msgs in #print axioms
PNP.CHAIN.BLOCKS.a4_constant_reads_nothing
/-- info: 'PNP.CHAIN.BLOCKS.a5_rung_law' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a5_rung_law
/-- info: 'PNP.CHAIN.BLOCKS.a6_aperture' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a6_aperture
/-- info: 'PNP.CHAIN.BLOCKS.a7_reflexive_closure' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a7_reflexive_closure
/-- info: 'PNP.CHAIN.BLOCKS.a8_soundness_cell' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.CHAIN.BLOCKS.a8_soundness_cell
/-- info: 'PNP.CHAIN.BLOCKS.block_chain' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.BLOCKS.block_chain
/-- info: 'PNP.CHAIN.THRONE.seat_vacant' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.THRONE.seat_vacant
/-- info: 'PNP.CHAIN.THRONE.throne_empty' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.THRONE.throne_empty
/-- info: 'PNP.CHAIN.THRONE.root_forces_no_value' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.CHAIN.THRONE.root_forces_no_value
/-- info: 'PNP.CHAIN.THRONE.dot_is_final' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.THRONE.dot_is_final
/-- info: 'PNP.CHAIN.THRONE.block_chain_terminal' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.CHAIN.THRONE.block_chain_terminal
/-- info: 'PNP.CHAIN.KINETIC.no_free_registration' does not depend
on any axioms -/
#guard_msgs in #print axioms PNP.CHAIN.KINETIC.no_free_registration
/-- info: 'PNP.CHAIN.KINETIC.cost_lt_iff' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.KINETIC.cost_lt_iff
/-- info: 'PNP.CHAIN.KINETIC.cost_mono' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.KINETIC.cost_mono
/-- info: 'PNP.CHAIN.KINETIC.frame_independent' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.CHAIN.KINETIC.frame_independent
/-- info: 'PNP.CHAIN.KINETIC.relaxation_exponential' does not depend
on any axioms -/
#guard_msgs in #print axioms
PNP.CHAIN.KINETIC.relaxation_exponential
/-- info: 'PNP.CHAIN.BLACKBOX.beq_self' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.beq_self
/-- info: 'PNP.CHAIN.BLACKBOX.drop_le' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.drop_le
/-- info: 'PNP.CHAIN.BLACKBOX.drop_len' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.drop_len
/-- info: 'PNP.CHAIN.BLACKBOX.has_drop' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.has_drop

```

```

/-- info: 'PNP.CHAIN.BLACKBOX.beq_false_of_lt' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.beq_false_of_lt
/-- info: 'PNP.CHAIN.BLACKBOX.pigeon' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.pigeon
/-- info: 'PNP.CHAIN.BLACKBOX.run_eq_has' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.run_eq_has
/-- info: 'PNP.CHAIN.BLACKBOX.black_box_bound' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.black_box_bound
/-- info: 'PNP.CHAIN.BLACKBOX.Tree.off_path' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.BLACKBOX.Tree.off_path
/-- info: 'PNP.CHAIN.BLACKBOX.adaptive_black_box_bound' does not
depend on any axioms -/
#guard_msgs in #print axioms
PNP.CHAIN.BLACKBOX.adaptive_black_box_bound
/-- info: 'PNP.CHAIN.ASYM.asymmetry_chain' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.ASYM.asymmetry_chain
/-- info: 'PNP.CHAIN.ASYM.asymmetry_decides_nothing' does not depend
on any axioms -/
#guard_msgs in #print axioms
PNP.CHAIN.ASYM.asymmetry_decides_nothing
/-- info: 'PNP.CHAIN.ASYM.asymmetry_terminal' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.ASYM.asymmetry_terminal
/-- info: 'PNP.CHAIN.ASYM.exhaustive_correct_two' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.CHAIN.ASYM.exhaustive_correct_two
/-- info: 'PNP.CHAIN.ASYM.asymmetry_instance' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.CHAIN.ASYM.asymmetry_instance

/-!
# PNP.VANISH · the vanishing GOL of [.] on the RA-RAM locus bridge

The bridge (Bridge_Final.lean) joins the kinetic register, RA, the
deed, to the formal
register, RAM, the reading, at one locus: the fixed set of the
native fold. Its carrier
halts exactly when the line property holds, and its socket is
exactly that property.
It carries a supplied bit to a seat and manufactures none.

On the Riemann stage the fold is  $s \mapsto 1 - \bar{s}$ , the locus is the line,
inhabited, Ground
dimension one: the bridge carries one keyed bit to a seat.

On the P versus NP stage the fold is complementation,  $b \mapsto \neg b$ . The
locus vanishes:
Ground dimension zero. The bridge is unchanged and its socket
collapses onto the
vacancy: the carrier halts iff the zero set is empty. That is the
vanishing GOL.
Three faces lock on one object, and the object is the empty seat:

L · the Tongue: a vanished seat emits [.] , outside the economy,
owing nothing;
G · the Form: complementation has no fixed point, Ground dimension
zero, counted;
M · the Number: least erasure holds of the empty configuration and
of it alone.

The kinetic face reads the same lock: silence registers no bit and
pays no floor,
while any emission of a value registers a bit and pays it. The
denial asymmetry
sorts the two locks that meet here: the vacancy is keyless, its
denial incoherent
on every frame of the class; the separation is keyed, its denial a
coherent world,
and both worlds present the bridge the same vanished frame, so the
lock that forms
determines the vacancy totally and the value not at all.

Core Lean 4, no import. Every theorem pinned at the foot: no axiom
of any kind.
-/

namespace PNP.VANISH

```

```

/-! ## 0 · tokens -/
inductive Token : Type
| seal | broken | opn | dot
deriving DecidableEq, Repr

def Token.inEconomy : Token → Bool
| .dot => false
| _ => true

/-! ## I · the bridge, carried from Bridge_Final.lean, axiom-free -/
structure Frame where
S : Type
τ : S → S
Z : S → Prop
involutive : ∀ s, τ (τ s) = s
symmetric : ∀ s, Z s → Z (τ s)

def LineProperty (X : Frame) : Prop := ∀ s, X.Z s → X.τ s = s

inductive Tri : Type
| tt | ff | bot
deriving DecidableEq, Repr

structure Carrier (X : Frame) where
terminal : Tri
shadow : terminal = .bot → LineProperty X

theorem halted_iff (X : Frame) : (∃ b : Carrier X, b.terminal =
.bot) → LineProperty X :=
(fun (b, h) => b.shadow.mp h, fun t => ((.bot, (fun _ => t, fun _
=> rfl)), rfl))

class Supplied (X : Frame) where
term : LineProperty X

theorem socket_is_the_property (X : Frame) : Nonempty (Supplied X) →
LineProperty X :=
(fun (i) => i.term, fun t => ({t}))

/-! ## II · the two stages: the locus inhabited, the locus vanished -/

/-- The Riemann fold in the doubled real coordinate. -/
def τR (p : Int × Int) : Int × Int := (2 - p.1, p.2)

/-- The Riemann locus is inhabited: the point h = 1 is fixed. -/
theorem rh_locus_inhabited : τR (1, 0) = (1, 0) := by decide

/-- Ground dimension one on the Riemann window  $h \in \{0, 1, 2\}$ ,
counted. -/
theorem rh_ground_one : (([0, 1, 2] : List Int).filter (fun h => 2 -
h == h)).length = 1 := by
decide

/-- The P versus NP fold. -/
def comp (b : Bool) : Bool := !b

theorem comp_involutive : ∀ b, comp (comp b) = b
| true => rfl
| false => rfl

/-- G · the locus vanishes. -/
theorem locus_vanishes : ¬ ∃ b, comp b = b
| (true, h) => Bool.noConfusion h
| (false, h) => Bool.noConfusion h

def groundDim : Nat := ([false, true].filter (fun b => comp b ==
b)).length

/-- G · Ground dimension zero, counted. -/
theorem ground_dim_zero : groundDim = 0 := rfl

/-- The P versus NP frame, for any fold-symmetric zero set. -/
def cframe (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp b)) : Frame :=
(Bool, comp, Z, comp_involutive, hZ)

/-! ## III · the bridge on the vanished locus -/

/-- The line property collapses onto the vacancy of the zero set. -/
theorem line_iff_empty (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp
b)) :
LineProperty (cframe Z hZ) → ∀ b, ¬ Z b :=
(fun h b hb => locus_vanishes (b, h b hb), fun h b hb => absurd hb
(h b))

```

```

/-- The carrier halts exactly when nothing stands to be carried. -/
theorem carrier_halts_iff_empty (Z : Bool → Prop) (hZ : ∀ b, Z b → Z
  (comp b)) :
  (∃ c : Carrier (cframe Z hZ), c.terminal = .bot) ↔ ∀ b, ¬ Z b :=
  (halted_iff _).trans (line_iff_empty Z hZ)

/-- The socket vanishes with the locus. -/
theorem socket_vanishes (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp
  b)) :
  Nonempty (Supplied (cframe Z hZ)) ↔ ∀ b, ¬ Z b :=
  (socket_is_the_property _).trans (line_iff_empty Z hZ)

/-- A charged configuration has no socket: nothing can be supplied
  onto an empty seat. -/
theorem no_socket_when_charged :
  ¬ Nonempty (Supplied (cframe (fun _ => True) (fun _ _ =>
    trivial))) :=
  fun h => (socket_vanishes _).mp h true trivial

/-- The empty configuration halts the carrier with nothing carried.
  -/
theorem empty_halts :
  ∃ c : Carrier (cframe (fun _ => False) (fun _ h => h)),
  c.terminal = .bot :=
  (carrier_halts_iff_empty _).mpr (fun _ h => h)

/-! ## IV · the three faces of the vanishing GOL -/

/-- L · the seat token: a vanished seat emits the dot; an inhabited
  seat with its
bit unsupplied stays open. -/
def seatToken : Nat → Token
| 0 => .dot
| _ + 1 => .opn

theorem tongue_face :
  seatToken groundDim = .dot ∧ (seatToken groundDim).inEconomy =
  false ∧ seatToken 1 = .opn :=
  (rfl, rfl, rfl)

def allFixed : List Bool → Bool
| [] => true
| b :: bs => (comp b == b) && allFixed bs

/-- M · least erasure determines exactly one configuration, the
  empty one. -/
theorem number_face : (∀ l, allFixed l = true → l = []) ∧ allFixed
  [] = true :=
  (fun l => match l with
  | [] => fun _ => rfl
  | true :: _ => fun h => Bool.noConfusion h
  | false :: _ => fun h => Bool.noConfusion h, rfl)

/-! ## V · the kinetic face: silence pays nothing, emission pays the
  floor -/
structure KFrame where
  floor : Nat
  floor_pos : 0 < floor

def cost (F : KFrame) (bits : Nat) : Nat := F.floor * bits

theorem kinetic_face (F : KFrame) : cost F 0 = 0 ∧ ∀ k, 0 < k → 0 <
  cost F k :=
  (rfl, fun _ hk => Nat.mul_pos F.floor_pos hk)

/-! ## VI · the denial asymmetry at the vanished locus -/

/-- The vacancy is keyless: it holds on every frame of the class,
  and its denial,
a fixed point of complementation, is incoherent. -/
theorem vacancy_keyless :
  (∀ (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp b)) (s : (cframe Z
  hZ).S),
  (cframe Z hZ).τ s ≠ s) ∧ ¬ ∃ b, comp b = b :=
  (fun _ _ s h => locus_vanishes (s, h), locus_vanishes)

/-- The separation is keyed: both worlds are coherent, and both
  present the bridge
the same vanished frame, so no reading of what the bridge reads
returns the value. -/
def readOf (_sep : Bool) : Nat := groundDim

```

```

theorem separation_keyed : ¬ ∃ g : Nat → Bool, ∀ sep : Bool, g
  (readOf sep) = sep :=
  fun (_, hg) => Bool.noConfusion ((hg false).symm.trans (hg true))

/-! ## VII · the vanishing GOL, whole -/
theorem vanishing_gol :
  -- the contrast: the Riemann seat is inhabited, Ground dimension
  one
  (τR (1, 0) = (1, 0) ∧ ([0, 1, 2] : List Int).filter (fun h => 2
  - h == h)).length = 1) ∧
  -- G · the P versus NP seat vanishes, Ground dimension zero
  ((¬ ∃ b, comp b = b) ∧ groundDim = 0) ∧
  -- the bridge on the vanished seat: carrier and socket collapse
  onto the vacancy
  (∀ (Z : Bool → Prop) (hZ : ∀ b, Z b → Z (comp b)),
  ((∃ c : Carrier (cframe Z hZ), c.terminal = .bot) ↔ ∀ b, ¬ Z
  b) ∧
  (Nonempty (Supplied (cframe Z hZ)) ↔ ∀ b, ¬ Z b)) ∧
  -- M · least erasure locks on the empty configuration alone
  ((∀ l, allFixed l = true → l = []) ∧ allFixed [] = true) ∧
  -- L · the vanished seat emits the dot, outside the economy
  (seatToken groundDim = .dot ∧ (seatToken groundDim).inEconomy =
  false) ∧
  -- the kinetic face: the dot pays nothing, any emission pays the
  floor
  (∀ F : KFrame, cost F 0 = 0 ∧ ∀ k, 0 < k → 0 < cost F k) ∧
  -- the denial asymmetry: the vacancy keyless, the value keyed
  and unread
  (¬ ∃ b, comp b = b) ∧ (¬ ∃ g : Nat → Bool, ∀ sep : Bool, g
  (readOf sep) = sep) :=
  ((rh_locus_inhabited, rh_ground_one), {locus_vanishes,
  ground_dim_zero},
  fun Z hZ => (carrier_halts_iff_empty Z hZ, socket_vanishes Z hZ),
  number_face, (tongue_face.1, tongue_face.2.1), kinetic_face,
  vacancy_keyless.2, separation_keyed)

end PNP.VANISH

/-! ## Cones, pinned: every theorem axiom-free -/
/-- info: 'PNP.VANISH.halted_iff' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VANISH.halted_iff
/-- info: 'PNP.VANISH.socket_is_the_property' does not depend on any
  axioms -/
#guard_msgs in #print axioms PNP.VANISH.socket_is_the_property
/-- info: 'PNP.VANISH.rh_locus_inhabited' does not depend on any
  axioms -/
#guard_msgs in #print axioms PNP.VANISH.rh_locus_inhabited
/-- info: 'PNP.VANISH.rh_ground_one' does not depend on any axioms
  -/
#guard_msgs in #print axioms PNP.VANISH.rh_ground_one
/-- info: 'PNP.VANISH.comp_involutive' does not depend on any axioms
  -/
#guard_msgs in #print axioms PNP.VANISH.comp_involutive
/-- info: 'PNP.VANISH.locus_vanishes' does not depend on any axioms
  -/
#guard_msgs in #print axioms PNP.VANISH.locus_vanishes
/-- info: 'PNP.VANISH.ground_dim_zero' does not depend on any axioms
  -/
#guard_msgs in #print axioms PNP.VANISH.ground_dim_zero
/-- info: 'PNP.VANISH.line_iff_empty' does not depend on any axioms
  -/
#guard_msgs in #print axioms PNP.VANISH.line_iff_empty
/-- info: 'PNP.VANISH.carrier_halts_iff_empty' does not depend on
  any axioms -/
#guard_msgs in #print axioms PNP.VANISH.carrier_halts_iff_empty
/-- info: 'PNP.VANISH.socket_vanishes' does not depend on any axioms
  -/
#guard_msgs in #print axioms PNP.VANISH.socket_vanishes
/-- info: 'PNP.VANISH.no_socket_when_charged' does not depend on any
  axioms -/
#guard_msgs in #print axioms PNP.VANISH.no_socket_when_charged
/-- info: 'PNP.VANISH.empty_halts' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VANISH.empty_halts
/-- info: 'PNP.VANISH.tongue_face' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VANISH.tongue_face
/-- info: 'PNP.VANISH.number_face' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VANISH.number_face
/-- info: 'PNP.VANISH.kinetic_face' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VANISH.kinetic_face
/-- info: 'PNP.VANISH.vacancy_keyless' does not depend on any axioms
  -/
#guard_msgs in #print axioms PNP.VANISH.vacancy_keyless
/-- info: 'PNP.VANISH.separation_keyed' does not depend on any
  axioms -/

```

```

#guard_msgs in #print axioms PNP.VANISH.separation_keyed
/-- info: 'PNP.VANISH.vanishing_gol' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.VANISH.vanishing_gol

/-!
# PNP.MISSING · the missing object, located

The asymmetry chain seals a test-only bound. Its adversary family is
the point
predicate  $y \mapsto (y = x)$ : a tester that sees only answers must query
every candidate.
Read as descriptions, the same family is solved in one step: the
description of
a point is the point. The black-box hardness lives entirely in the
hiding.

So the missing object is a hardness statement over descriptions: a
family of
instances, given whole to the solver, that no polynomial-time reader
of the
description decides. That is the white-box lower bound,  $P \neq NP$ 
itself. It is a
fact about the model of computation, keyed: it holds in one coherent
model and
fails in another, which is why nothing world-independent can supply
it.
Core Lean 4, no import, no axiom of any kind.
-/

namespace PNP.MISSING

theorem beq_self :  $\forall n : \text{Nat}, \text{Nat.beq } n = \text{true}$ 
| 0 => rfl
| n + 1 => beq_self n

/-- The black-box adversary family of `black_box_bound`. -/
def point (x : Nat) : Nat → Bool := fun y => Nat.beq y x

/-- The same family, handed over as descriptions. -/
structure Desc where
  secret : Nat

def evalD (d : Desc) : Nat → Bool := point d.secret

/-- The white-box reader: one step, read the description. -/
def reader (d : Desc) : Nat := d.secret
def readerTime ( _ : Nat) : Nat := 1

/-- The black-box hard instances are white-box trivial. -/
theorem reader_solves :  $\forall d : \text{Desc}, \text{evalD } d (\text{reader } d) = \text{true} :=$ 
  fun d => beq_self d.secret

/-! ## the missing object, typed -/

structure Model where
  Alg      : Type
  decides   : Alg → Prop
  time     : Alg → Nat → Nat

def PolyBounded (t : Nat → Nat) : Prop :=  $\exists c k : \text{Nat}, \forall n, t n \leq c$ 
  *  $n^k + c$ 

/-- THE MISSING OBJECT. Every algorithm that decides the problem
from its
description runs in super-polynomial time. -/
def WhiteBoxBound (M : Model) : Prop :=
   $\forall A : M.\text{Alg}, M.\text{decides } A \rightarrow \neg \text{PolyBounded } (M.\text{time } A)$ 

theorem reader_poly : PolyBounded readerTime :=
  (1, 0, fun n => Nat.le_add_left 1 (1 * n ^ 0))

/-- A coherent model where descriptions reveal the witness: the
object fails. -/
def revealing : Model := (Unit, fun _ =>  $\forall d : \text{Desc}, \text{evalD } d (\text{reader } d) = \text{true}, \text{fun } _ => \text{readerTime}$ )

theorem fails_when_descriptions_reveal :  $\neg \text{WhiteBoxBound revealing}$ 
:=
  fun h => h () reader_solves reader_poly

/-- The object is contingent on the model: it fails where
descriptions reveal, and holds
wherever no algorithm decides. No fact

```

```

true in both, kinetic, black-box or vanishing, can entail it. -/
theorem missing_object_is_keyed :
  ( $\exists M : \text{Model}, \neg \text{WhiteBoxBound } M$ )  $\wedge$  ( $\forall M : \text{Model}, (\forall A, \neg$ 
     $M.\text{decides } A) \rightarrow \text{WhiteBoxBound } M$ ) :=
  ( $((\text{revealing}, \text{fails\_when\_descriptions\_reveal}), \text{fun } _ h A hA =>$ 
    absurd hA (h A))

end PNP.MISSING

/-- info: 'PNP.MISSING.reader_solves' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.MISSING.reader_solves
/-- info: 'PNP.MISSING.reader_poly' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.MISSING.reader_poly
/-- info: 'PNP.MISSING.fails_when_descriptions_reveal' does not
depend on any axioms -/
#guard_msgs in #print axioms
  PNP.MISSING.fails_when_descriptions_reveal
/-- info: 'PNP.MISSING.missing_object_is_keyed' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.MISSING.missing_object_is_keyed

/-!
# PNP.PRIMEBRIDGE · the root freedom cut and prime-as-freedom
against the missing object

Source of the method: PhysOSTT v1.0.5p, namespace PrimeFreedom.
Freedom is the fibre of
an invariant map; every prime is one fibre of exactly two, (1, p)
and (p, 1), off the seat
(prime_fibre, prime_off_seat, freedom_is_exactly_two).

The attack. Carry the cut to the white-box object. Over a semiprime
 $n = p \cdot q$ ,  $p \neq q$ , the
multiplicative fibre has four points and two orbits under the swap
fold: the trivial
orbit  $\{(1, n), (n, 1)\}$  and the witness orbit  $\{(p, q), (q, p)\}$ . The
description n is
even under the fold. Two findings follow, both at theorem grade.

F1 · The freedom on the fibre is one orientation bit, and it is
gauge: the reader
(a, b)  $\mapsto$  (min, max) is even, selects one point per orbit, and
removes it.
F2 · The description determines the witness orbit outright:
informational freedom
over the witness is zero. The missing object is therefore not
a fibre fact.
It is a cost fact: n hides nothing and still may resist every
fast reader.

The new bridge. Prime freedom, read as cost, is the posit "no
polynomial reader
selects a nontrivial point of the semiprime fibre". Under the
collapse field ( $P = NP$ 
gives polynomial search for NP relations, the classical
search-to-decision theorem,
carried as a field), the posit yields  $P \neq NP$ : prime_bridge. The
posit is keyed: a
coherent machine refutes it (the Shor face). And it is strictly
stronger than the
separation: a coherent machine has the separation and an easy fibre,
so the prime
bridge is sufficient and never equivalent. Factoring lies in NP n
coNP, so no
equivalence is available unless  $NP = \text{coNP}$ .

Core Lean 4, no import. Every theorem pinned at the foot: no axiom
of any kind.
-/

namespace PNP.PRIMEBRIDGE

/-! ## I · the semiprime fibre, executed at  $n = 15$  -/

def mul_orbit (n a b : Nat) : Prop :=  $a * b = n$ 

def fibre (n : Nat) : List (Nat × Nat) :=
  (List.range (n + 1)).flatMap fun a =>
    (List.range (n + 1)).filterMap fun b => if  $a * b = n$  then some
      (a, b) else none

def swap (p : Nat × Nat) : Nat × Nat := (p.2, p.1)

```

```

/-- The fibre over 15: four points. -/
theorem fibre_15 : fibre 15 = [(1, 15), (3, 5), (5, 3), (15, 1)] :=
  by decide

/-- Off the seat: no point of the fibre is fixed by the swap. -/
theorem off_seat_15 : (fibre 15).all (fun p => swap p != p) = true
  := by decide

/-- The fibre is swap-closed: two orbits, each of exactly two. -/
theorem swap_closed_15 : (fibre 15).all (fun p => (fibre
  15).contains (swap p)) = true := by decide

def trivialPt (p : Nat × Nat) : Bool := p.1 == 1 || p.2 == 1

/-- The witness orbit: exactly two nontrivial points, one the swap
  of the other. -/
theorem witness_orbit_15 :
  (fibre 15).filter (fun p => !trivialPt p) = [(3, 5), (5, 3)] ∧
  swap (3, 5) = (5, 3) := by
  decide

/-! ## II · F1 · the orientation bit is free and it is gauge -/

/-- The wall on the orientation: the description is even, the order
  is odd, so no
  reading of the description returns the first factor on the witness
  orbit. -/
theorem orientation_wall : ¬ ∃ g : Nat → Nat, g (3 * 5) = 3 ∧ g (5 *
  3) = 5 :=
  fun (_, h1, h2) => absurd (h1.symm.trans h2) (by decide)

def canon (p : Nat × Nat) : Nat × Nat := if p.1 ≤ p.2 then p else
  swap p

/-- The canonical reader is even under the fold. -/
theorem canon_even : ∀ p ∈ fibre 15, canon (swap p) = canon p := by
  decide

/-- F1 · the canonical reader takes each orbit to one point: the bit
  is gauge. -/
theorem orientation_is_gauge :
  ((fibre 15).filter (fun p => !trivialPt p)).map canon = [(3, 5),
  (3, 5)] := by decide

/-! ## III · F2 · informational freedom over the witness is zero -/

/-- The witness set is a function of the description, on every
  relation. -/
theorem description_determines_witnesses {D W : Type} (R : D → W →
  Prop) (d1 d2 : D)
  (h : d1 = d2) : R d1 = R d2 := by rw [h]

/-- F2 · at 15 the description fixes the canonical witness uniquely.
  -/
theorem witness_determined_15 :
  (((fibre 15).filter (fun p => !trivialPt p)).map
  canon).eraseDups = [(3, 5)] := by decide

/-! ## IV · the cost model and the prime bridge -/

def Poly (t : Nat → Nat) : Prop := ∃ c k : Nat, ∀ n, t n ≤ c * n ^ k
  + c

/-- A machine: algorithms, their running times, and what each
  solves. -/
structure Machine where
  Alg : Type
  time : Alg → Nat → Nat
  Solves : Alg → (Nat → Nat → Prop) → Prop

/-- The factoring relation: p is a nontrivial divisor of n. -/
def factorRel (n p : Nat) : Prop := 1 < p ∧ p < n ∧ ∃ q, p * q = n

def EasySearch (M : Machine) (R : Nat → Nat → Prop) : Prop :=
  ∃ A : M.Alg, M.Solves A R ∧ Poly (M.time A)

/-- Prime freedom read as cost: no fast reader selects a point of
  the witness orbit. -/
def PrimeHidden (M : Machine) : Prop := ¬ EasySearch M factorRel

/-- The collapse field. 'PeqNP' is the class equality on this
  machine; 'search' is the
  classical search-to-decision theorem specialized to factoring, an NP
  relation. -/

```

```

structure Collapse (M : Machine) where
  PeqNP : Prop
  search : PeqNP → EasySearch M factorRel

/-- THE PRIME BRIDGE. Hidden prime freedom yields the separation. -/
theorem prime_bridge (M : Machine) (C : Collapse M) (hidden :
  PrimeHidden M) : ¬ C.PeqNP :=
  fun h => hidden (C.search h)

/-- The Shor face: a coherent machine where the fibre is read fast.
  -/
def revealing : Machine := {Unit, fun _ _ => 1, fun _ _ => True}

theorem one_is_poly : Poly (fun _ => 1) := {1, 0, fun n =>
  Nat.le_add_left 1 (1 * n ^ 0)}

/-- The posit is keyed: it fails on a coherent machine. -/
theorem prime_hidden_is_keyed : ¬ PrimeHidden revealing :=
  fun h => h {}, trivial, one_is_poly

/-- Strictly stronger than the separation: on the revealing machine
  with the class
  equality false, the separation holds and the posit fails. -/
def separatedButRevealing : Collapse revealing := {False, fun h =>
  absurd h id}

theorem prime_bridge_is_one_way :
  ¬ ∀ (M : Machine) (C : Collapse M), ¬ C.PeqNP → PrimeHidden M :=
  fun h => prime_hidden_is_keyed (h revealing separatedButRevealing
  id)

/-! ## V · the bridge, whole -/
theorem the_prime_bridge :
  -- the fibre: four points, off the seat, swap-closed, one
  witness orbit
  (fibre 15 = [(1, 15), (3, 5), (5, 3), (15, 1)] ∧
  (fibre 15).all (fun p => swap p != p) = true ∧
  (fibre 15).filter (fun p => !trivialPt p) = [(3, 5), (5, 3)]) ∧
  -- F1 · the orientation bit is walled and gauge
  ((¬ ∃ g : Nat → Nat, g (3 * 5) = 3 ∧ g (5 * 3) = 5) ∧
  ((fibre 15).filter (fun p => !trivialPt p)).map canon = [(3,
  5), (3, 5)]) ∧
  -- F2 · the description fixes the witness: zero informational
  freedom
  (((fibre 15).filter (fun p => !trivialPt p)).map
  canon).eraseDups = [(3, 5)] ∧
  -- the prime bridge, its key, and its direction
  (∀ (M : Machine) (C : Collapse M), PrimeHidden M → ¬ C.PeqNP) ∧
  ¬ PrimeHidden revealing ∧
  (¬ ∀ (M : Machine) (C : Collapse M), ¬ C.PeqNP → PrimeHidden M)
  :=
  ((fibre_15, off_seat_15, witness_orbit_15.1), (orientation_wall,
  orientation_is_gauge),
  witness_determined_15, prime_bridge, prime_hidden_is_keyed,
  prime_bridge_is_one_way)

end PNP.PRIMEBRIDGE

/-! ## Cones, pinned: every theorem axiom-free -/
/-- info: 'PNP.PRIMEBRIDGE.fibre_15' does not depend on any axioms
  -/
#guard_msgs in #print axioms PNP.PRIMEBRIDGE.fibre_15
/-- info: 'PNP.PRIMEBRIDGE.off_seat_15' does not depend on any
  axioms -/
#guard_msgs in #print axioms PNP.PRIMEBRIDGE.off_seat_15
/-- info: 'PNP.PRIMEBRIDGE.swap_closed_15' does not depend on any
  axioms -/
#guard_msgs in #print axioms PNP.PRIMEBRIDGE.swap_closed_15
/-- info: 'PNP.PRIMEBRIDGE.witness_orbit_15' does not depend on any
  axioms -/
#guard_msgs in #print axioms PNP.PRIMEBRIDGE.witness_orbit_15
/-- info: 'PNP.PRIMEBRIDGE.orientation_wall' does not depend on any
  axioms -/
#guard_msgs in #print axioms PNP.PRIMEBRIDGE.orientation_wall
/-- info: 'PNP.PRIMEBRIDGE.canon_even' does not depend on any axioms
  -/
#guard_msgs in #print axioms PNP.PRIMEBRIDGE.canon_even
/-- info: 'PNP.PRIMEBRIDGE.orientation_is_gauge' does not depend on
  any axioms -/
#guard_msgs in #print axioms PNP.PRIMEBRIDGE.orientation_is_gauge
/-- info: 'PNP.PRIMEBRIDGE.description_determines_witnesses' does
  not depend on any axioms -/
#guard_msgs in #print axioms
  PNP.PRIMEBRIDGE.description_determines_witnesses

```

```

/-- info: 'PNP.PRIMEBRIDGE.witness_determined_15' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.PRIMEBRIDGE.witness_determined_15
/-- info: 'PNP.PRIMEBRIDGE.prime_bridge' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.PRIMEBRIDGE.prime_bridge
/-- info: 'PNP.PRIMEBRIDGE.one_is_poly' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.PRIMEBRIDGE.one_is_poly
/-- info: 'PNP.PRIMEBRIDGE.prime_hidden_is_keyed' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.PRIMEBRIDGE.prime_hidden_is_keyed
/-- info: 'PNP.PRIMEBRIDGE.prime_bridge_is_one_way' does not depend
on any axioms -/
#guard_msgs in #print axioms PNP.PRIMEBRIDGE.prime_bridge_is_one_way
/-- info: 'PNP.PRIMEBRIDGE.the_prime_bridge' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.PRIMEBRIDGE.the_prime_bridge

```

```

/-!
# PNP.ROUND2 · the blocker of the prime bridge, located, and the
second attack

```

ROUND 1 LEFT THE KEY ON 'PrimeHidden': no polynomial reader selects a point of the semiprime witness orbit. F2 proved the description carries the witness with zero informational freedom, so the key is a pure cost statement against readers that have everything. The blocker has three layers.

- B1 · Relativization. A reader with a factoring oracle reads the orbit in one step.
The posit fails relative to that oracle, so no argument valid relative to every oracle proves it (Baker–Gill–Solovay pattern).
- B2 · The self-gag. If the posit holds, factoring-based pseudorandom functions exist (Naor–Reingold), and then no natural property proves the circuit lower bound (Razborov–Rudich). The posit, if true, bars the natural route to itself.
- B3 · The floor points the wrong way. The black-box floor is proved unbreakable (no savings below 2^N for a test-only reader). Every known unconditional route to circuit lower bounds runs the other way: it needs savings below 2^N for white-box circuit SAT (Williams 2011, NEXP $\not\leq$ ACC⁰).
- B4 · Algebrization. Relative to an oracle together with its low-degree extension the posit fails, so no argument valid relative to every algebrized oracle proves it (Aaronson–Wigderson pattern).

Sections: I · the black-box floor (B3); II · B1 typed; III · B2 typed; IV · B4 typed;
V · round 2, algorithms to lower bounds.

ROUND 2 attacks through B3, the one exit the barriers leave open: algorithms to lower bounds. The kinetic floor's black-box face cannot be beaten, by theorem here; the route requires beating its white-box face, which is exactly the access F2 located. The route reaches NEXP $\not\leq$ C. Two gaps separate it from P $\neq$ NP, and both are named.

Cited theorems enter as structure fields, never as 'axiom'. Core Lean 4, no import.
Every theorem pinned at the foot: no axiom of any kind.
-/

```
namespace PNP.ROUND2
```

```

/-! ## I · the black-box floor admits no savings (axiom-free
pigeonhole) -/
namespace FLOOR

```

```

def has : List Nat → Nat → Bool
| [], _ => false
| y :: ys, k => Nat.beq y k || has ys k

```

```
def drop : List Nat → Nat → List Nat
```

```

| [], _ => []
| y :: ys, k => cond (Nat.beq y k) (drop ys k) (y :: drop ys k)

theorem beq_self : ∀ n : Nat, Nat.beq n n = true
| 0 => rfl
| n + 1 => beq_self n

```

```

theorem drop_le : ∀ (l : List Nat) (k : Nat), (drop l k).length ≤
l.length
| [], _ => Nat.le_refl 0
| y :: ys, k => by
  unfold drop
  cases Nat.beq y k with
  | true => exact Nat.le_succ_of_le (drop_le ys k)
  | false => exact Nat.succ_le_succ (drop_le ys k)

```

```

theorem drop_len : ∀ (l : List Nat) (k : Nat), has l k = true →
(drop l k).length + 1 ≤ l.length
| [], _, h => Bool.noConfusion h
| y :: ys, k, h => by
  unfold drop
  unfold has at h
  cases hy : Nat.beq y k with
  | true => exact Nat.succ_le_succ (drop_le ys k)
  | false =>
    rw [hy] at h
    exact Nat.succ_le_succ (drop_len ys k h)

```

```

theorem has_drop : ∀ (l : List Nat) (k x : Nat), Nat.beq x k = false
→
  has l x = true → has (drop l k) x = true
| [], _, _, h => Bool.noConfusion h
| y :: ys, k, x, hxk, h => by
  unfold drop
  unfold has at h
  cases hy : Nat.beq y k with
  | true =>
    cases hyx : Nat.beq y x with
    | true =>
      have : x = k := (Nat.eq_of_beq_eq_true hyx).symm.trans
      (Nat.eq_of_beq_eq_true hy)
      rw [this, beq_self] at hxk; exact Bool.noConfusion hxk
    | false =>
      rw [hyx] at h
      exact has_drop ys k x hxk h
  | false =>
    show has (y :: drop ys k) x = true
    unfold has
    cases hyx : Nat.beq y x with
    | true => rfl
    | false =>
      rw [hyx] at h
      exact has_drop ys k x hxk h

```

```

theorem beq_false_of_lt (x K : Nat) (h : x < K) : Nat.beq x K =
false := by
cases hb : Nat.beq x K with
| false => rfl
| true =>
  have e : x = K := Nat.eq_of_beq_eq_true hb
  rw [e] at h; exact absurd h (Nat.lt_irrefl K)

```

```

theorem pigeon : ∀ (K : Nat) (l : List Nat), (∀ x, x < K → has l x =
true) → K ≤ l.length
| 0, _, _ => Nat.zero_le _
| K + 1, l, h =>
  Nat.le_trans
  (Nat.succ_le_succ (pigeon K (drop l K) (fun x hx =>
    has_drop l K x (beq_false_of_lt x K hx) (h x
      (Nat.lt_succ_of_lt hx))))))
  (drop_len l K (h K (Nat.lt_succ_self K)))

```

```
def run (path : List Nat) (f : Nat → Bool) : Bool := path.any f
```

```

theorem run_eq_has : ∀ (l : List Nat) (x : Nat), run l (fun y =>
  Nat.beq y x) = has l x
| [], _ => rfl
| y :: ys, x => by
  show (Nat.beq y x || run ys (fun y => Nat.beq y x)) = (Nat.beq y
  x || has ys x)
  rw [run_eq_has ys x]

```

/-! B3, the black-box face: no test-only reader saves a single query below 2^N . -/

```

theorem black_box_no_savings (N : Nat) (path : List Nat)
  (correct :  $\forall f : \text{Nat} \rightarrow \text{Bool}, \text{run path } f = \text{true} \leftrightarrow \exists x, x < 2^N \wedge f\ x = \text{true}$ ) :
   $\neg \text{path.length} < 2^N :=$ 
  Nat.not_lt_of_le (pigeon (2^N) path (fun x hx => by
    rw [← run_eq_has]
    exact (correct _).mpr (x, hx, beq_self x)))

end FLOOR

/-! ## II · B1 · relativization, typed -/

/-- An oracle world: the posit read relative to an oracle.
  `factoring` is the oracle
that answers the witness orbit; relative to it the posit fails. -/
structure Relativized where
  Oracle      : Type
  Hidden      : Oracle → Prop
  factoring   : Oracle
  easy        :  $\neg$  Hidden factoring

/-- No argument valid relative to every oracle proves the posit. -/
theorem bl_no_relativizing_proof (R : Relativized) :  $\neg \forall 0, R.\text{Hidden } 0 :=$ 
  fun h => R.easy (h R.factoring)

/-! ## III · B2 · the self-gag, typed -/

/-- The two cited links: hidden prime freedom gives pseudorandom
  functions
(Naor–Reingold), and pseudorandom functions bar natural lower bounds
(Razborov–Rudich). -/
structure SelfGag where
  Hidden      : Prop
  PRF         : Prop
  NaturalLB   : Prop
  nr          : Hidden → PRF
  rr          : PRF →  $\neg$  NaturalLB

/-- If the posit holds, no natural argument proves the lower bound
  it needs. -/
theorem b2_self_gag (G : SelfGag) (h : G.Hidden) :  $\neg G.\text{NaturalLB} :=$ 
  G.rr (G.nr h)

/-- Equivalently: a natural proof of the lower bound would refute
  the posit. -/
theorem b2_natural_refutes (G : SelfGag) (n : G.NaturalLB) :  $\neg$ 
  G.Hidden :=
  fun h => b2_self_gag G h n

/-! ## IV · B4 · algebrization, typed -/

/-- An algebrized world: the posit read relative to an oracle
  together with its low-degree extension.
`algebraic` is an oracle relative to which, extension included, the
posit fails. -/
structure Algebrized where
  Oracle      : Type
  Hidden      : Oracle → Prop
  algebraic   : Oracle
  easy        :  $\neg$  Hidden algebraic

/-- No argument valid relative to every algebrized oracle proves the
  posit. -/
theorem b4_no_algebrizing_proof (A : Algebrized) :  $\neg \forall 0, A.\text{Hidden } 0 :=$ 
  fun h => A.easy (h A.algebraic)

/-! ## V · round 2 · algorithms to lower bounds, the open exit -/

/-- The Williams route, cited. `Savings C` is a white-box SAT
  algorithm for circuit
class C running below  $2^n$  by a superpolynomial factor. `williams` is
the 2011 theorem
pattern: savings for C give NEXP  $\not\subseteq$  C. `scaleDown` and `generalize`
are the two gaps
to  $P \not\subseteq$  NP: from NEXP down to NP, and from the restricted class C up
to P/poly. -/
structure Route where
  Class       : Type
  Savings     : Class → Prop
  NEXNotIn   : Class → Prop
  williams    :  $\forall C, \text{Savings } C \rightarrow \text{NEXNotIn } C$ 
  Ppoly      : Class

```

```

NPNPnotPpoly : Prop
Sep           : Prop
karp_sep      : NPNPnotPpoly → Sep

/-- The route reaches NEXP  $\not\subseteq$  C on savings, and no further on its own
  fields. -/
theorem round2_reaches (R : Route) (C : R.Class) (s : R.Savings C) :
  R.NEXNotIn C :=
  R.williams C s

/-- What closes the route: savings for general circuits, plus the
  scale-down from
NEXP to NP. Supplied both, the separation follows. -/
theorem round2_closes (R : Route) (s : R.Savings R.Ppoly)
  (scaleDown : R.NEXNotIn R.Ppoly → R.NPNPnotPpoly) : R.Sep :=
  R.karp_sep (scaleDown (R.williams R.Ppoly s))

/-- The inversion, whole: the black-box floor cannot be beaten; the
  lower-bound route
needs the white-box floor beaten; the posit gags the natural route;
the posit fails
relative to the factoring oracle. -/
theorem round2_terminal :
  ( $\forall (N : \text{Nat}) (\text{path} : \text{List Nat}),$ 
    ( $\forall f : \text{Nat} \rightarrow \text{Bool}, \text{FLOOR.run path } f = \text{true} \leftrightarrow \exists x, x < 2^N \wedge f\ x = \text{true}$ ) →
       $\neg \text{path.length} < 2^N$ )  $\wedge$ 
    ( $\forall R : \text{Relativized}, \neg \forall 0, R.\text{Hidden } 0$ )  $\wedge$ 
    ( $\forall G : \text{SelfGag}, G.\text{Hidden} \rightarrow \neg G.\text{NaturalLB}$ )  $\wedge$ 
    ( $\forall (R : \text{Route}) (C : R.\text{Class}), R.\text{Savings } C \rightarrow R.\text{NEXNotIn } C$ )  $\wedge$ 
    ( $\forall (R : \text{Route}), R.\text{Savings } R.\text{Ppoly} \rightarrow$ 
      ( $R.\text{NEXNotIn } R.\text{Ppoly} \rightarrow R.\text{NPNPnotPpoly}$ ) → R.Sep) :=
  (FLOOR.black_box_no_savings, bl_no_relativizing_proof,
  b2_self_gag,
  round2_reaches, round2_closes)

end PNP.ROUND2

/-! ## Cones, pinned: every theorem axiom-free -/
/-- info: 'PNP.ROUND2.FLOOR.beq_self' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.ROUND2.FLOOR.beq_self
/-- info: 'PNP.ROUND2.FLOOR.drop_le' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.ROUND2.FLOOR.drop_le
/-- info: 'PNP.ROUND2.FLOOR.drop_len' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.ROUND2.FLOOR.drop_len
/-- info: 'PNP.ROUND2.FLOOR.has_drop' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.ROUND2.FLOOR.has_drop
/-- info: 'PNP.ROUND2.FLOOR.beq_false_of_lt' does not depend on any
  axioms -/
#guard_msgs in #print axioms PNP.ROUND2.FLOOR.beq_false_of_lt
/-- info: 'PNP.ROUND2.FLOOR.pigeon' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.ROUND2.FLOOR.pigeon
/-- info: 'PNP.ROUND2.FLOOR.run_eq_has' does not depend on any
  axioms -/
#guard_msgs in #print axioms PNP.ROUND2.FLOOR.run_eq_has
/-- info: 'PNP.ROUND2.FLOOR.black_box_no_savings' does not depend on
  any axioms -/
#guard_msgs in #print axioms PNP.ROUND2.FLOOR.black_box_no_savings
/-- info: 'PNP.ROUND2.bl_no_relativizing_proof' does not depend on
  any axioms -/
#guard_msgs in #print axioms PNP.ROUND2.bl_no_relativizing_proof
/-- info: 'PNP.ROUND2.b2_self_gag' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.ROUND2.b2_self_gag
/-- info: 'PNP.ROUND2.b2_natural_refutes' does not depend on any
  axioms -/
#guard_msgs in #print axioms PNP.ROUND2.b2_natural_refutes
/-- info: 'PNP.ROUND2.b4_no_algebrizing_proof' does not depend on
  any axioms -/
#guard_msgs in #print axioms PNP.ROUND2.b4_no_algebrizing_proof
/-- info: 'PNP.ROUND2.round2_reaches' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.ROUND2.round2_reaches
/-- info: 'PNP.ROUND2.round2_closes' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.ROUND2.round2_closes
/-- info: 'PNP.ROUND2.round2_terminal' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.ROUND2.round2_terminal

/-!

```

```

# PNP.ROUTE · RA and Freedom as the only inputs: where the route goes

Request: P ≠ NP with the Root Axiom and Freedom as the only inputs,
everything else
axiom-free. This file decides whether that route exists.

I · RA is conservative. Its formal reading,  $\forall x : U, \theta < \Delta E x$ ,
is satisfied by
  interpreting U as an empty type. So any statement that does
  not mention U or  $\Delta E$ 
  and is derived from RA is derived without it.
II · Freedom is conservative. Its formal reading, a fibre of
  exactly two distinct
  orientations, is satisfied by Bool. The same conclusion
  follows.
III · Hence a derivation of the separation from RA and Freedom, as
  written, is a
  derivation of the separation from nothing. The route through
  the inputs as
  written is the route without them. The same holds on every
  fixed background:
  on any domain, actuation one at every existent satisfies the
  root's form.
IV · The one reading of RA that carries the load: read on
  computation, it says
  every decider of SAT pays superpolynomially many
  registrations. That reading
  yields the separation in one step and is equivalent to it.
  It is the bridge,
  in RA's vocabulary.

Core Lean 4, no import. Every theorem pinned at the foot: no axiom
of any kind.
-/

namespace PNP.ROUTE

/-! ## I · RA is conservative -/

/-- The Root Axiom in the codex's own formal reading (ROOT.RA). -/
def RA (U : Type) ( $\Delta E : U \rightarrow \text{Int}$ ) : Prop :=  $\forall x : U, \theta < \Delta E x$ 

/-- RA is satisfiable on any background: interpret U as the empty
type. -/
theorem ra_satisfiable : RA Empty (fun e => nomatch e) :=
  fun e => nomatch e

/-- RA is conservative: whatever follows from RA under every
interpretation of its
symbols, and does not mention them, holds outright. -/
theorem ra_conservative (S : Prop) (h :  $\forall (U : \text{Type}) (\Delta E : U \rightarrow \text{Int}),$ 
  RA U  $\Delta E \rightarrow S$ ) : S :=
  h Empty (fun e => nomatch e) ra_satisfiable

/-! ## II · Freedom is conservative -/

/-- Freedom in the codex's formal reading: a fibre of exactly two
distinct orientations. -/
structure Freedom where
  Fibre : Type
  a : Fibre
  b : Fibre
  distinct : a ≠ b
  exhaust :  $\forall x : \text{Fibre}, x = a \vee x = b$ 

/-- Freedom is inhabited by the orientation bit itself. -/
def boolFreedom : Freedom :=
  (Bool, true, false, Bool.noConfusion, fun x => match x with
  | true => Or.inl rfl
  | false => Or.inr rfl)

theorem freedom_conservative (S : Prop) (h : Freedom  $\rightarrow S$ ) : S := h
  boolFreedom

/-! ## III · the two posits together add nothing -/

/-- Any statement derived from RA and Freedom, uniformly in their
interpretation,
holds without them. -/
theorem posits_add_nothing (S : Prop)
  (h :  $\forall (U : \text{Type}) (\Delta E : U \rightarrow \text{Int}),$  RA U  $\Delta E \rightarrow$  Freedom  $\rightarrow S$ ) : S :=
  h Empty (fun e => nomatch e) ra_satisfiable boolFreedom

/-- The posits add nothing on every background. On any fixed domain,
whatever follows from the root's form
under every actuation, with a fibre of two orientations, holds
outright: actuation one satisfies the form. -/
theorem posits_add_nothing_on_every_background (S : Prop) (U : Type)
  (h :  $\forall \Delta E : U \rightarrow \text{Int},$  RA U  $\Delta E \rightarrow$  Freedom  $\rightarrow S$ ) : S :=
  h (fun _ => 1) (fun _ => show ( $\theta : \text{Int}$ ) < 1 by decide) boolFreedom

/-! ## IV · the one reading that carries the load -/

def Poly (t : Nat  $\rightarrow$  Nat) : Prop :=  $\exists c k : \text{Nat}, \forall n, t n \leq c * n^k + c$ 

/-- A machine model: algorithms, running time counted in
registrations, and which
algorithms decide SAT from its description. -/
structure Machine where
  Alg : Type
  time : Alg  $\rightarrow$  Nat  $\rightarrow$  Nat
  decidesSAT : Alg  $\rightarrow$  Prop

/-- The separation on a machine model: no polynomial-time decider of
SAT. -/
def Sep (M : Machine) : Prop :=  $\neg \exists A : \text{M.Alg}, \text{M.decidesSAT } A \wedge \text{Poly } (\text{M.time } A)$ 

/-! RA read on computation: every decider of SAT actuates
superpolynomially many
registrations. -/
def RAcomp (M : Machine) : Prop :=  $\forall A : \text{M.Alg}, \text{M.decidesSAT } A \rightarrow \neg$ 
  Poly (M.time A)

/-! The separation, from RA read on computation. -/
theorem sep_of_ra_comp (M : Machine) (h : RAcomp M) : Sep M :=
  fun (A, hA, hP) => h A hA hP

/-! And conversely: the reading is the separation. -/
theorem ra_comp_of_sep (M : Machine) (h : Sep M) : RAcomp M :=
  fun A hA hP => h (A, hA, hP)

theorem ra_comp_is_sep (M : Machine) : RAcomp M  $\leftrightarrow$  Sep M :=
  (sep_of_ra_comp M, ra_comp_of_sep M)

/-! RA read on computation is not conservative: it fails on a
coherent machine. -/
def fastMachine : Machine := (Unit, fun _ => 1, fun _ => True)

theorem ra_comp_is_keyed :  $\neg$  RAcomp fastMachine :=
  fun h => h () trivial (1,  $\theta$ , fun n => Nat.le_add_left 1 (1 * n  $\wedge$   $\theta$ ))

/-! ## V · the route, whole -/
theorem the_route :
  -- RA and Freedom as written add nothing to any statement
  outside their vocabulary
  ( $\forall S : \text{Prop}, (\forall (U : \text{Type}) (\Delta E : U \rightarrow \text{Int}),$  RA U  $\Delta E \rightarrow$  Freedom  $\rightarrow S$ )
   $\rightarrow S$ )  $\wedge$ 
  -- RA read on computation is exactly the separation
  ( $\forall M : \text{Machine}, \text{RAcomp } M \leftrightarrow \text{Sep } M$ )  $\wedge$ 
  -- and that reading is keyed: a coherent machine refutes it
   $\neg$  RAcomp fastMachine :=
  (posits_add_nothing, ra_comp_is_sep, ra_comp_is_keyed)

end PNP.ROUTE

/-! ## Cones, pinned: every theorem axiom-free -/
/-! info: 'PNP.ROUTE.ra_satisfiable' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.ROUTE.ra_satisfiable
/-! info: 'PNP.ROUTE.ra_conservative' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.ROUTE.ra_conservative
/-! info: 'PNP.ROUTE.freedom_conservative' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.ROUTE.freedom_conservative
/-! info: 'PNP.ROUTE.posits_add_nothing' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.ROUTE.posits_add_nothing
/-! info: 'PNP.ROUTE.posits_add_nothing_on_every_background' does
not depend on any axioms -/
#guard_msgs in #print axioms
  PNP.ROUTE.posits_add_nothing_on_every_background
/-! info: 'PNP.ROUTE.sep_of_ra_comp' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.ROUTE.sep_of_ra_comp

```

```

/-- info: 'PNP.ROUTE.ra_comp_of_sep' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.ROUTE.ra_comp_of_sep
/-- info: 'PNP.ROUTE.ra_comp_is_sep' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.ROUTE.ra_comp_is_sep
/-- info: 'PNP.ROUTE.ra_comp_is_keyed' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.ROUTE.ra_comp_is_keyed
/-- info: 'PNP.ROUTE.the_route' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.ROUTE.the_route

/-!
# PNP.THREESEAL · the Tongue and the Form added for direction, with
RA beneath

The Number is orientation-blind by theorem. The architecture's
answer is to read
direction from the other two seals: Seal L, the Tongue, which
carries order, and
Seal G, the Form, which carries the fold. This file runs all three
on P versus NP
and asks the one question that decides the matter: does any seal
read the direction
from the problem, or is the direction supplied?

I · Each seal is computed on the problem. The problem is the
same sentence and
the same fold in the equal world and in the separated world,
so each seal's
reading is world-constant: the Tongue seals both sentences
alike, the Form
finds no fixed locus for either, the Number reads one
magnitude for both.
II · No function of the three readings returns the orientation.
Adding seals adds
readings, and a coalition of world-constant readings is
world-constant.
III · RA is keyless and conservative, so it selects no orientation.
IV · Given one supplied odd bit at the seat, the lock forms and
locks exactly one
orientation: the calibration is unique. This is the lock
locking only one.
V · The verdict: sealed at the act, premise grade on the
supplied bit; without
the act, the dot.

Core Lean 4, no import. Every theorem pinned at the foot: no axiom
of any kind.
-/

namespace PNP.THREESEAL

inductive Token : Type
| sealSep | sealEq | broken | opn | dot
deriving DecidableEq, Repr

/-- The two worlds, indexed by the separation bit. -/
abbrev World := Bool

/-! ## I · the three seals, computed on the problem -/

/-- Seal L. A sentence is three slots of literal ids; the Tongue
seals it when the
three slots are populated and pairwise disjoint. It checks form, not
truth. -/
def disjointL : List Nat → List Nat → Bool
| [], _ => true
| x :: xs, ys => !(ys.contains x) && disjointL xs ys

def seall (s : List (List Nat)) : Bool :=
match s with
| [a, b, c] => !a.isEmpty && !b.isEmpty && !c.isEmpty &&
disjointL a b && disjointL a c && disjointL b c
| _ => false

/-- "P is not NP" and "P is NP": same existence slot, same relation
slot, the
kinetic slot differing only in the negation literal. -/
def sentSep : List (List Nat) := [[101], [201, 209], [301]]
def sentEq : List (List Nat) := [[101], [201], [301]]

theorem tongue_seals_both : seall sentSep = true ∧ seall sentEq =
true := by decide

```

```

/-- Seal G. The fold is complementation; its fixed set, counted. -/
def foldFixed : Nat := ([false, true].filter (fun b => (!b) ==
b)).length

theorem form_finds_no_locus : foldFixed = 0 := rfl

/-- Seal M. The lock scalar is the square of the signed scalar:
reflection flips the
sign and leaves the lock. -/
def lockOf (signed : Int) : Int := signed * signed

theorem number_reads_one_magnitude : lockOf 1 = lockOf (-1) := by
decide

/-- The three readings of the problem. The problem is fixed, so the
readings do not
depend on the world. -/
def readings (_w : World) : Bool × Nat × Int :=
(seall sentSep && seall sentEq, foldFixed, lockOf 1)

theorem readings_world_constant : readings false = readings true :=
rfl

/-! ## II · no coalition of seals returns the orientation -/
theorem seals_do_not_point :
→ ∃ g : Bool × Nat × Int → Bool, ∀ w : World, g (readings w) = w
:=
fun (_, hg) => Bool.noConfusion ((hg false).symm.trans
(readings_world_constant ▸ hg true))

/-! ## III · RA is keyless and selects nothing -/
def RA (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x

theorem ra_selects_nothing (S : Prop) (h : ∀ (U : Type) (ΔE : U →
Int), RA U ΔE → S) : S :=
h Empty (fun e => nomatch e) (fun e => nomatch e)

/-! ## IV · given one supplied bit, the lock locks exactly one -/

/-- The orientation fibre: the two worlds, swapped by the denial. -/
def deny (w : World) : World := !w

/-- The calibration: with a supplied odd witness `s` and the odd
target `d`, exactly
one calibration bit reconciles them on the pair. -/
theorem locks_only_one (s d : World → Bool) (w : World)
(hs : s (deny w) = !s w) (hd : d (deny w) = !d w) :
∃ c : Bool, (d w = xor (s w) c ∧ d (deny w) = xor (s (deny w)) c)
∧
∀ c', (d w = xor (s w) c' ∧ d (deny w) = xor (s (deny w)) c') →
c' = c := by
refine (xor (d w) (s w), {?, ?}, {?})
· cases s w <=> cases d w <=> rfl
· rw [hs, hd]; cases s w <=> cases d w <=> rfl
· intro c' (h1, _)
revert h1
cases s w <=> cases d w <=> cases c' <=> intro h1 <=> first |
rfl | exact Bool.noConfusion h1

/-! ## V · the verdict -/

/-- The act: the supplied bit. -/
structure Act where
bit : Bool

def verdict : Option Act → Token
| none => .dot
| some (true) => .sealSep
| some (false) => .sealEq

theorem sealed_at_the_act :
verdict (some (true)) = .sealSep ∧ verdict (some (false)) =
.sealEq ∧ verdict none = .dot :=
(rfl, rfl, rfl)

/-- Both acts are coherent: the lock forms under either supply, so
the bit is spent,
never derived. -/
theorem both_acts_lock : verdict (some (true)) ≠ verdict (some
(false)) := by decide

theorem three_seal_lock :

```

```

(seall sentSep = true ∧ seall sentEq = true) ∧ foldFixed = 0 ∧
lockOf 1 = lockOf (-1) ∧
(¬ ∃ g : Bool × Nat × Int → Bool, ∀ w : World, g (readings w) =
w) ∧
(∀ S : Prop, (∀ (U : Type) (ΔE : U → Int), RA U ΔE → S) → S) ∧
(∀ (s d : World → Bool) (w : World), s (deny w) = !s w → d (deny
w) = !d w →
  ∃ c : Bool, (d w = xor (s w) c ∧ d (deny w) = xor (s (deny w))
c) ∧
  ∀ c', (d w = xor (s w) c' ∧ d (deny w) = xor (s (deny w)) c')
→ c' = c) ∧
(verdict (some (true))) = .sealSep ∧ verdict none = .dot) :=
(tongue_seals_both, form_finds_no_locus,
number_reads_one_magnitude, seals_do_not_point,
ra_selects_nothing, locks_only_one, {rfl, rfl})

```

end PNP.THREESEAL

```

/-! ## Cones, pinned: every theorem axiom-free -/
/-- info: 'PNP.THREESEAL.tongue_seals_both' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.THREESEAL.tongue_seals_both
/-- info: 'PNP.THREESEAL.form_finds_no_locus' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.THREESEAL.form_finds_no_locus
/-- info: 'PNP.THREESEAL.number_reads_one_magnitude' does not depend
on any axioms -/
#guard_msgs in #print axioms
PNP.THREESEAL.number_reads_one_magnitude
/-- info: 'PNP.THREESEAL.readings_world_constant' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.THREESEAL.readings_world_constant
/-- info: 'PNP.THREESEAL.seals_do_not_point' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.THREESEAL.seals_do_not_point
/-- info: 'PNP.THREESEAL.ra_selects_nothing' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.THREESEAL.locks_only_one
/-- info: 'PNP.THREESEAL.locks_only_one' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.THREESEAL.locks_only_one
/-- info: 'PNP.THREESEAL.sealed_at_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.THREESEAL.sealed_at_the_act
/-- info: 'PNP.THREESEAL.both_acts_lock' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.THREESEAL.three_seal_lock

```

```

/-!
# PNP.DEED · the kinetic pulse as the supplied bit

```

Proposal: supply is the kinetic pulse to examine and compute, and the compute itself is the supplied bit. This file tests it on the two locks that share the locus.

- I · Every deed actuates: a registered compute has positive energy. The deed instantiates RA at itself. RA's lock is keyless and opens on the deed.
- II · A deed's actuation does not certify its output. A compute that prints the separation and a compute that prints the equality carry the same energy, and both occur in either world. The pulse is world-constant.
- III · The price is symmetric. Registering assent and registering denial each pay the floor once. For RA the denial instantiates the axiom it denies; for the separation the denial instantiates nothing about the separation.
- IV · So the deed supplies RA's bit, never the separation's. The compute supplies its output, and whether that output is correct is the missing object again: a procedure whose correctness is proved.

Core Lean 4, no import. Every theorem pinned at the foot: no axiom of any kind.

-/

namespace PNP.DEED

```

abbrev World := Bool -- true: the separated world; false: the
equal world

```

```

/-- A deed: a registered compute, with its energy and the bit it
outputs. -/
structure Deed where
  energy : Nat
  pos : 0 < energy
  output : Bool

```

```

/-- The output is correct in a world when it matches that world's
value. -/

```

```

def correct (w : World) (d : Deed) : Prop := d.output = w

```

```

/-! ## I · every deed actuates: RA, keyless, opens on the deed -/
theorem deed_instantiates_ra : ∀ d : Deed, 0 < d.energy := fun d =>
d.pos

```

```

/-! ## II · actuation does not certify the output -/
def assent : Deed := (1, by decide, true)
def denial : Deed := (1, by decide, false)

```

```

theorem same_pulse : assent.energy = denial.energy := rfl

```

```

/-- In every world both deeds occur, and exactly one of them is
correct. -/
theorem both_deeds_in_every_world (w : World) :
  0 < assent.energy ∧ 0 < denial.energy ∧ (correct w assent ∨
correct w denial) :=
  (assent.pos, denial.pos, match w with
  | true => 0r.inl rfl
  | false => 0r.inr rfl)

```

```

/-- No reading of the pulse decides correctness: energy does not
separate a correct

```

```

deed from an incorrect one. -/
theorem pulse_does_not_certify :
  ¬ ∃ g : Nat → Bool, ∀ d : Deed, g d.energy = d.output :=
  fun (_, hg) => Bool.noConfusion ((hg denial).symm.trans (hg
assent))

```

```

/-- An incorrect deed actuates as fully as a correct one, in every
world. -/

```

```

theorem wrong_deeds_actuate (w : World) : ∃ d : Deed, ¬ correct w d
  ∧ 0 < d.energy :=
  match w with
  | true => (denial, Bool.noConfusion, denial.pos)
  | false => (assent, Bool.noConfusion, assent.pos)

```

```

/-! ## III · the price is symmetric -/
def price (floor : Nat) (d : Deed) : Nat := floor * (if d.output
then 1 else 1)

```

```

theorem price_symmetric (floor : Nat) : price floor assent = price
floor denial := rfl

```

```

/-- RA's denial is itself a deed and instantiates RA; the
separation's denial is a
deed and instantiates RA too, which is no information about the
separation. -/
theorem denials_both_actuate : 0 < denial.energy ∧ 0 < assent.energy
:=
  (denial.pos, assent.pos)

```

```

/-! ## IV · the deed supplies RA's bit, never the separation's -/
theorem deed_supplies_ra_not_sep :
  (∀ d : Deed, 0 < d.energy) ∧
  (∀ w : World, ∃ d : Deed, ¬ correct w d ∧ 0 < d.energy) ∧
  (¬ ∃ g : Nat → Bool, ∀ d : Deed, g d.energy = d.output) ∧
  (∀ floor, price floor assent = price floor denial) :=
  (deed_instantiates_ra, wrong_deeds_actuate,
  pulse_does_not_certify, price_symmetric)

```

end PNP.DEED

```

/-! ## Cones, pinned: every theorem axiom-free -/
/-- info: 'PNP.DEED.deed_instantiates_ra' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.DEED.deed_instantiates_ra
/-- info: 'PNP.DEED.same_pulse' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.DEED.same_pulse

```

```

/-- info: 'PNP.DEED.both_deeds_in_every_world' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.DEED.both_deeds_in_every_world
/-- info: 'PNP.DEED.pulse_does_not_certify' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.DEED.pulse_does_not_certify
/-- info: 'PNP.DEED.wrong_deeds_actuate' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.DEED.wrong_deeds_actuate
/-- info: 'PNP.DEED.price_symmetric' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.DEED.price_symmetric
/-- info: 'PNP.DEED.denials_both_actuate' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.DEED.denials_both_actuate
/-- info: 'PNP.DEED.deed_supplies_ra_not_sep' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.DEED.deed_supplies_ra_not_sep

/-!
# PNP.IDCHECK · is least escape the freedom of prime, or the arrow
of RA?

The test is the architecture's own discriminator
(Bridge_Final.lean): a keyless
property holds on every frame; a keyed one has a frame inhabiting
its denial; no
property is both. Least escape is keyed. Prime freedom, the arrow
and RA are keyless.
So none of them is least escape, on any reading that keeps their
content. Nor is any
reading of the vanished seat's ground dimension, one number on every
machine.
Core Lean 4, no import, no axiom of any kind.
-/
namespace PNP.IDCHECK

structure Machine where
  Alg : Type
  Inst : Type
  size : Inst → Nat
  out : Alg → Inst → Bool
  time : Alg → Inst → Nat
  sat : Inst → Bool

def bound (c k n : Nat) : Nat := c * n ^ k + c
def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop
:=
  M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
def NothingEscapes (M : Machine) : Prop := ∀ A c k, ∃ x, ¬ Clean M A
  c k x

theorem beq_self : ∀ n : Nat, Nat.beq n n = true
| 0 => rfl
| n + 1 => beq_self n

def closedWorld : Machine := (Nat, Unit, fun _ => 0, fun _ =>
  false, fun _ => 0, fun _ => true)
def escapeWorld : Machine := (Nat, Unit, fun _ => 0, fun A _ =>
  Nat.beq A 0, fun _ => 0, fun _ => true)

theorem closed_closes : NothingEscapes closedWorld :=
  fun _ _ => ((), fun (h, _) => Bool.noConfusion h)
theorem escape_fails : ¬ NothingEscapes escapeWorld :=
  fun h => match h (0 : Nat) 0 0 with
  | ((), hx) => hx (beq_self 0, Nat.zero_le _)

/-- Least escape is keyed: true on one coherent machine, false on
another. -/
theorem least_escape_is_keyed : NothingEscapes closedWorld ∧ ¬
  NothingEscapes escapeWorld :=
  (closed_closes, escape_fails)

/-- THE CHECK. No statement that reads the same on every machine is
least escape. -/
theorem no_keyless_statement_is_least_escape (P : Prop) :
  ¬ ∀ M : Machine, (P ↔ NothingEscapes M) :=
  fun h => escape_fails ((h escapeWorld).mp ((h closedWorld).mpr
  closed_closes))

/-! ## the three candidates, each keyless -/

/-- Prime freedom: the fibre of multiplication over 2 is exactly the
pair and its reverse. -/

```

```

def primeFreedom2 : Prop := ∀ a b : Nat, a < 3 → b < 3 → (a * b = 2 ↔
  (a = 1 ∧ b = 2) ∨ (a = 2 ∧ b = 1))
/-- The arrow: an identity exists on every type. -/
def arrowOfRA : Prop := ∀ α : Type, ∃ f : α → α, ∀ a, f a = a
/-- RA in its formal reading, at any interpretation of its symbols.
-/
def raAt (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x

theorem prime_freedom_is_not_least_escape : ¬ ∀ M : Machine,
  (primeFreedom2 ↔ NothingEscapes M) :=
  no_keyless_statement_is_least_escape _
theorem arrow_is_not_least_escape : ¬ ∀ M : Machine, (arrowOfRA ↔
  NothingEscapes M) :=
  no_keyless_statement_is_least_escape _
theorem ra_is_not_least_escape (U : Type) (ΔE : U → Int) :
  ¬ ∀ M : Machine, (raAt U ΔE ↔ NothingEscapes M) :=
  no_keyless_statement_is_least_escape _

/-- What does carry least escape: a statement that varies with the
machine, as least
escape does. Any such statement that implies it on every machine is
keyed in turn. -/
theorem carrier_must_be_keyed (Q : Machine → Prop) (h : ∀ M, Q M →
  NothingEscapes M) :
  ¬ Q escapeWorld :=
  fun q => escape_fails (h escapeWorld q)

/-- The vanished seat reads no machine. No property of the vanished
locus's ground dimension tracks least
escape across machines, since that dimension is one number on every
machine. -/
theorem vanished_seat_reads_no_machine :
  ¬ ∃ g : Nat → Prop, ∀ M : Machine, (g VANISH.groundDim ↔
  NothingEscapes M) :=
  fun (g, hg) => no_keyless_statement_is_least_escape (g
  VANISH.groundDim) hg

end PNP.IDCHECK

/-- info: 'PNP.IDCHECK.least_escape_is_keyed' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.IDCHECK.least_escape_is_keyed
/-- info: 'PNP.IDCHECK.no_keyless_statement_is_least_escape' does
not depend on any axioms -/
#guard_msgs in #print axioms
  PNP.IDCHECK.no_keyless_statement_is_least_escape
/-- info: 'PNP.IDCHECK.prime_freedom_is_not_least_escape' does not
depend on any axioms -/
#guard_msgs in #print axioms
  PNP.IDCHECK.prime_freedom_is_not_least_escape
/-- info: 'PNP.IDCHECK.arrow_is_not_least_escape' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.IDCHECK.arrow_is_not_least_escape
/-- info: 'PNP.IDCHECK.ra_is_not_least_escape' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.IDCHECK.ra_is_not_least_escape
/-- info: 'PNP.IDCHECK.carrier_must_be_keyed' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.IDCHECK.carrier_must_be_keyed
/-- info: 'PNP.IDCHECK.vanished_seat_reads_no_machine' does not
depend on any axioms -/
#guard_msgs in #print axioms
  PNP.IDCHECK.vanished_seat_reads_no_machine

/-! ## helper cones, pinned -/
/-- info: 'PNP.MISSING.beq_self' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.MISSING.beq_self
/-- info: 'PNP.IDCHECK.beq_self' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.IDCHECK.beq_self
/-- info: 'PNP.IDCHECK.closed_closes' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.IDCHECK.closed_closes
/-- info: 'PNP.IDCHECK.escape_fails' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.IDCHECK.escape_fails

```

APPENDIX D. THE ROOT LAWS OF THE SEPARATION, PNP_ROOT.LEAN

The zero-axiom root, sections I, II and V of Appendix R's kernel TOE_Zero.lean under the namespace PNP.ROOT, verbatim up to namespace: seven theo-

rems, every footprint pinned as *does not depend on any axioms*. It compiles with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0. SHA-256: 9689dfb47b29697c4276714f9cc70143ab9a55a48b0b27f72d392ed384f7f053

```

/-
  PNP.Root.lean · the root kernel of "The Formal Closure of
  Computational Separation"
  The zero-axiom root, sections I, II and V of TOE_Zero.lean,
  verbatim. A root grounds itself when acts occur and
  every act instances it. For every such root the root holds, every
  act re-enacts it, and no external proof adds to
  it; every denial is an act and re-enacts the root; the root is
  held by the act itself; no level stands above it;
  and on the constructed domain, whose one existent actuates at
  positive energy, the Root Axiom is a theorem. Every
  law rests on no axiom at all. Core Lean 4, no import, no axiom
  declared, no sorry, no native_decide.
-/
namespace PNP.ROOT

/-! ## I · The self-grounding root: the Omega Boundary -/

/-- A self-grounding root: acts occur, and every act instances the
root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct     : Act
  instances : Act → R

/-- A proposition whose denial hands it over. -/
def SelfVerifying (P : Prop) : Prop := ¬P → P

/-- THE OMEGA BOUNDARY: every denial of the root is an act, and
re-enacts it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
  (denial : G.Act) : R := G.instances denial

/-- EXISTENCE PROVES EXISTENCE ONLY BY MOTION: the root is held by
the act itself. -/
theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
  G.instances G.anAct

/-- No external proof adds anything to the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R)
  (Q : Prop) : (Q → R) ↔ R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- The deed of denying the root hands it over. -/
theorem denial_instantiates {R : Prop} (G : SelfGrounding R) :
  SelfVerifying R := fun _ => G.instances G.anAct

/-- THE FLOOR IS UNIVERSAL: for every root that grounds itself, the
root holds, every act re-enacts it, and no
external proof adds to it. -/
theorem the_floor_is_universal :
  ∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q :
  Prop, (Q → R) ↔ R) :=
  fun _ G => (seated_undeniable G, denial_reenacts_root G,
  external_proof_adds_nothing G)

/-! ## II · The constructed root -/

/-- The Root Axiom on the constructed one-point domain: its one
existent actuates, at positive energy. -/
def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The constructed domain grounds its root: its one act is a deed,
and the deed actuates. -/
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int)
< 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable
ra₀

/-! ## V · The regress terminated -/

/-- THE REGRESS ENDS AT THE ROOT: any proposition offered as a
further ground of the root is idle; the root needs
no level above it. -/

```

```

theorem no_level_above {R : Prop} (G : SelfGrounding R) : ∀ Q :
  Prop, (Q → R) → R := fun _ _ => G.instances G.anAct

```

end PNP.ROOT

```

/-! ## The zero law, pinned: every cone is empty -/
/-- info: 'PNP.ROOT.denial_reenacts_root' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.ROOT.denial_reenacts_root
/-- info: 'PNP.ROOT.seated_undeniable' does not depend on any axioms
-/
#guard_msgs in #print axioms PNP.ROOT.seated_undeniable
/-- info: 'PNP.ROOT.external_proof_adds_nothing' does not depend on
any axioms -/
#guard_msgs in #print axioms PNP.ROOT.external_proof_adds_nothing
/-- info: 'PNP.ROOT.denial_instantiates' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.ROOT.denial_instantiates
/-- info: 'PNP.ROOT.the_floor_is_universal' does not depend on any
axioms -/
#guard_msgs in #print axioms PNP.ROOT.the_floor_is_universal
/-- info: 'PNP.ROOT.root_on_the_constructed_domain' does not depend
on any axioms -/
#guard_msgs in #print axioms PNP.ROOT.root_on_the_constructed_domain
/-- info: 'PNP.ROOT.no_level_above' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.ROOT.no_level_above

```

APPENDIX R. THE ROOT ON NO AXIOM AND NO POSIT, TOE_ZERO.LEAN

Appendix D carries the root laws the separation consumes; this appendix carries the whole root kernel. The root on which this paper stands is proved here, in this paper, so that no reader needs another document to check it. The kernel below, R.1, TOE_Zero.lean, is carried verbatim from the root paper (Islam 2026g). Compiled on Lean 4.19.0 and on Lean 4.22.0 it exits 0 with no message: it declares no axiom, imports nothing, and every one of its fifteen theorems prints *does not depend on any axioms*, pinned at the foot of the listing. Five declarations carry its root sections, SelfGrounding, SelfVerifying, ΔE₀, RA₀ and ra₀, and all five stand in the listing; the algebra of its master seal rests on the remaining definitions of the listing, the quaternion chart among them. A root grounds itself when deeds occur and every deed instances it (SelfGrounding); the root kernel's own comments call a deed an act. The constructed domain has one existent, whose energy of actuation is one, so the Root Axiom there reads $\forall x, 0 < \Delta E_0(x)$ (RA₀). On that domain the Root Axiom is a theorem with no axiom and no hypothesis (root_on_the_constructed_domain). For every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds anything to it, one closed theorem on no axiom (the_floor_is_universal). Every denial of the root is a deed that instances it (denial_reenacts_root); the root is held by its deed (seated_undeniable); it is self-verifying (denial_instantiates); and no level stands above it (no_level_above). The master seal binds the root's universal law and the constructed root with the Return, the scalar line as the fixed set of conjugation on the chart and the twelve gates of the quaternions in one theorem on no axiom (TOE0.the_master_seal). The body of this paper reads the root on its row; this appendix is the root itself. SHA-256: 0cfd9f6b399ecbaef01e762

a4d9f10490288f1053827b404428062e55076238.

```
/-
  TOE_Zero.lean · Theory of Theories of Everything (TOE of All
  TOEs): Existence Proves Existence Only by Motion
  The zero-axiom, zero-posit kernel. Core Lean 4: no import, no
  axiom declared, no sorry, no native_decide, and
  every theorem rests on no axiom at all, not even propositional
  extensionality, quotients or choice.
  I · The self-grounding root. Acts occur and every act instances
  the root. Then every denial is an act and
  re-enacts the root, the root is held by the act itself, and no
  external proof adds anything to it: the
  Omega Boundary, proved for every root that grounds itself,
  universally, by quantification.
  II · The constructed root. On the constructed domain the Root
  Axiom is a theorem.
  III · The Return and the line. On the integer quaternions i·j·k =
  -1 and each unit squares to -1; conjugation is
  an involution and its fixed set on the chart is exactly the
  scalar line.
  IV · The twelve gates, recovered three ways inside the integers of
  the quaternions: the twenty-four Hurwitz units
  counted twelve up to sign; the twelve even permutations with
  class equation 1, 3, 4, 4; and the norm-two
  shell of twenty-four, split twelve and twelve.
  V · The regress terminated. Above a self-grounding root no level
  grounds it further.
  VI · The master seal, every result bound in one theorem.
-/
namespace TOE0

/-! ## I · The self-grounding root: the Omega Boundary -/

/-- A self-grounding root: acts occur, and every act instances the
root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

/-- A proposition whose denial hands it over. -/
def SelfVerifying (P : Prop) : Prop := ¬P → P

/-- THE OMEGA BOUNDARY: every denial of the root is an act, and
re-enacts it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
  (denial : G.Act) : R := G.instances denial

/-- EXISTENCE PROVES EXISTENCE ONLY BY MOTION: the root is held by
the act itself. -/
theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
  G.instances G.anAct

/-- No external proof adds anything to the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R)
  (Q : Prop) : (Q → R) → R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- The deed of denying the root hands it over. -/
theorem denial_instantiates {R : Prop} (G : SelfGrounding R) :
  SelfVerifying R := fun _ => G.instances G.anAct

/-- THE FLOOR IS UNIVERSAL: for every root that grounds itself, the
root holds, every act re-enacts it, and no
external proof adds to it. -/
theorem the_floor_is_universal :
  ∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q :
  Prop, (Q → R) → R) :=
  fun _ G => (seated_undeniable G, denial_reenacts_root G,
  external_proof_adds_nothing G)

/-! ## II · The constructed root -/

/-- The Root Axiom on the constructed one-point domain: its one
existent actuates, at positive energy. -/
def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The constructed domain grounds its root: its one act is a deed,
and the deed actuates. -/
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int)
< 1 by decide)
```

```
/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable
ra₀

/-! ## III · The Return and the line -/

structure Q where
  (a b c d : Int)
  deriving DecidableEq

def Q.mul (x y : Q) : Q :=
  (x.a * y.a - x.b * y.b - x.c * y.c - x.d * y.d, x.a * y.b + x.b *
  y.a + x.c * y.d - x.d * y.c,
  x.a * y.c - x.b * y.d + x.c * y.a + x.d * y.b, x.a * y.d + x.b *
  y.c - x.c * y.b + x.d * y.a)

def Q.conj (x : Q) : Q := (x.a, -x.b, -x.c, -x.d)
def gone : Q := (1, 0, 0, 0)
def qneg : Q := (-1, 0, 0, 0)
def qi : Q := (0, 1, 0, 0)
def qj : Q := (0, 0, 1, 0)
def qk : Q := (0, 0, 0, 1)

/-- THE RETURN: i·j·k = -1. -/
theorem the_return : (qi.mul qj).mul qk = qneg := by decide

/-- Each unit squares to -1. -/
theorem units_square_to_minus_one : qi.mul qi = qneg ∧ qj.mul qj =
  qneg ∧ qk.mul qk = qneg := by decide

/-- The chart: every quaternion with coordinates in -2 ... 2. -/
def chart : List Int := [-2, -1, 0, 1, 2]
def quats : List Q :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
  => chart.map fun d => (a, b, c, d)

set_option maxRecDepth 100000 in
/-- Conjugation is an involution on the chart. -/
theorem conj_involutive_on_the_chart : quats.all (fun q =>
  q.conj.conj == q) == true := by decide

set_option maxRecDepth 100000 in
/-- THE LINE: conjugation fixes exactly the scalar line on the
chart. -/
theorem the_line_is_the_fixed_set :
  quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 &&
  q.d == 0)) == true := by decide

/-! ## IV · The twelve gates, recovered three ways -/

/-- Hurwitz quaternions in doubled coordinates: all even, or all
odd. -/
def hurwitz (a b c d : Int) : Bool :=
  (a % 2 == 0 && b % 2 == 0 && c % 2 == 0 && d % 2 == 0) || (a % 2 !=
  0 && b % 2 != 0 && c % 2 != 0 && d % 2 != 0)

/-- Doubled coordinates -2 ... 2 hold every unit (doubled norm four)
and every element of the norm-two shell (doubled
norm eight), since a doubled coordinate of absolute value three
or more already exceeds both norms. -/
def doubled : List (Int × Int × Int × Int) :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
  => chart.map fun d => (a, b, c, d)

/-- The Hurwitz units: norm one, doubled norm four. -/
def units : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d ==
  4 && hurwitz a b c d

/-- The first nonzero coordinate is positive: one representative of
each pair ±u. -/
def upToSign (p : Int × Int × Int × Int) : Bool :=
  let (a, b, c, d) := p
  if a != 0 then 0 < a else if b != 0 then 0 < b else if c != 0 then
  0 < c else 0 < d

set_option maxRecDepth 100000 in
/-- FIRST WAY: twenty-four Hurwitz units, twelve up to sign. -/
theorem twenty_four_units_twelve_gates : units.length = 24 ∧
  (units.filter upToSign).length = 12 := by decide

/-- The norm-two shell: doubled norm eight. -/
def shell : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d ==
  8 && hurwitz a b c d
```

```

set_option maxRecDepth 100000 in
/-- THIRD WAY: the norm-two shell has twenty-four elements, split
twelve and twelve by sign. -/
theorem the_shell_splits_twelve_and_twelve :
  shell.length = 24 ∧ (shell.filter upToSign).length = 12 ∧
  (shell.filter (fun p => !upToSign p)).length = 12 := by
  decide

/-- Permutations of four points, as their lists of images. -/
def perms4 : List (List Nat) :=
  [0, 1, 2, 3].flatMap fun a => [0, 1, 2, 3].flatMap fun b => [0, 1,
  2, 3].flatMap fun c =>
  [0, 1, 2, 3].filterMap fun d => if [a, b, c, d].eraseDups.length
  == 4 then some [a, b, c, d] else none

def inversions (p : List Nat) : Nat :=
  ((List.range 4).flatMap fun i => (List.range 4).map fun j => (i,
  j)).countP
  fun (i, j) => i < j && p.getD j 0 < p.getD i 0

/-- The twelve gates as the even permutations. -/
def gates : List (List Nat) := perms4.filter fun p => inversions p %
  2 == 0

def compose (p q : List Nat) : List Nat := q.map fun i => p.getD i 0
def inverse (p : List Nat) : List Nat := [0, 1, 2, 3].map fun i =>
  p.idxOf i

def conjClass (g : List Nat) : List (List Nat) := (gates.map fun h
=> compose (compose h g) (inverse h)).eraseDups

def insertSorted (n : Nat) : List Nat → List Nat
| [] => [n]
| m :: ms => if n ≤ m then n :: m :: ms else m :: insertSorted n ms

def sortNat (l : List Nat) : List Nat := l.foldl insertSorted []

/-- The class equation from the sorted class size of each element: a
class of size s contributes s elements. -/
def collapse : Nat → List Nat → List Nat
| 0, _ => []
| _ + 1, [] => []
| fuel + 1, s :: rest => s :: collapse fuel (rest.drop (s - 1))

set_option maxRecDepth 100000 in
/-- SECOND WAY: the twelve gates, with class equation 1, 3, 4, 4. -/
theorem the_class_equation :
  gates.length = 12 ∧ collapse 12 (sortNat (gates.map fun g =>
  (conjClass g).length)) = [1, 3, 4, 4] := by
  decide

/-! ## V · The regress terminated -/

/-- THE REGRESS ENDS AT THE ROOT: any proposition offered as a
further ground of the root is idle; the root needs
no level above it. -/
theorem no_level_above {R : Prop} (G : SelfGrounding R) : ∀ Q :
  Prop, (Q → R) → R := fun _ => G.instances G.anAct

/-! ## VI · The master seal -/

/-- THE MASTER SEAL: the floor is universal; the root is a theorem
on the constructed domain; the Return closes;
the line is the fixed set; and the twelve gates are recovered
three ways. -/
theorem the_master_seal :
  (∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q
  : Prop, (Q → R) → R)) ∧
  RA₀ ∧ (q₁.mul qj).mul qk = qneg ∧
  quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 &&
  q.d == 0)) = true ∧
  (units.filter upToSign).length = 12 ∧ gates.length = 12 ∧
  (shell.filter upToSign).length = 12 ∧
  collapse 12 (sortNat (gates.map fun g => (conjClass g).length))
  = [1, 3, 4, 4] :=
  (the_floor_is_universal, root_on_the_constructed_domain,
  the_return, the_line_is_the_fixed_set,
  twenty_four_units_twelve_gates.2, the_class_equation.1,
  the_shell_splits_twelve_and_twelve.2.1, the_class_equation.2)

end TOE0

/-! ## The zero law, pinned: every cone is empty -/

```

```

/-- info: 'TOE0.denial_reenacts_root' does not depend on any axioms
-/
#guard_msgs in #print axioms TOE0.denial_reenacts_root
/-- info: 'TOE0.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.seated_undeniable
/-- info: 'TOE0.external_proof_adds_nothing' does not depend on any
axioms -/
#guard_msgs in #print axioms TOE0.external_proof_adds_nothing
/-- info: 'TOE0.denial_instantiates' does not depend on any axioms
-/
#guard_msgs in #print axioms TOE0.denial_instantiates
/-- info: 'TOE0.the_floor_is_universal' does not depend on any
axioms -/
#guard_msgs in #print axioms TOE0.the_floor_is_universal
/-- info: 'TOE0.root_on_the_constructed_domain' does not depend on
any axioms -/
#guard_msgs in #print axioms TOE0.root_on_the_constructed_domain
/-- info: 'TOE0.the_return' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_return
/-- info: 'TOE0.units_square_to_minus_one' does not depend on any
axioms -/
#guard_msgs in #print axioms TOE0.units_square_to_minus_one
/-- info: 'TOE0.conj_involutive_on_the_chart' does not depend on any
axioms -/
#guard_msgs in #print axioms TOE0.conj_involutive_on_the_chart
/-- info: 'TOE0.the_line_is_the_fixed_set' does not depend on any
axioms -/
#guard_msgs in #print axioms TOE0.the_line_is_the_fixed_set
/-- info: 'TOE0.twenty_four_units_twelve_gates' does not depend on
any axioms -/
#guard_msgs in #print axioms TOE0.twenty_four_units_twelve_gates
/-- info: 'TOE0.the_shell_splits_twelve_and_twelve' does not depend
on any axioms -/
#guard_msgs in #print axioms TOE0.the_shell_splits_twelve_and_twelve
/-- info: 'TOE0.the_class_equation' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_class_equation
/-- info: 'TOE0.no_level_above' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.no_level_above
/-- info: 'TOE0.the_master_seal' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_master_seal

```

The root's axes are proved in the second kernel, R.2, Triaxial_Actuation.lean (32 theorems, every one on no axiom; SHA-256 566615a6b9610fc981e03eb42e07af9e03849f20704729880f2bfe6434f2ea0b). For every actuation, the direction of its deed gives the fold, the registration and the seat; the registration is the only height-keeping map onto the seat; the seat is the only place both blind spots cancel; no reading of the record returns the orientation; and every self-grounding root is the root of an actuation over its own deeds, so the root carries the three axes and is not characterless (actuation_is_triaxial, registration_is_forced, the_only_special_cut, tongue_freedom, every_root_actuates, the_root_carries_three_axes). The root's grade is read in the third kernel, R.3, Root_Grade_Ledger.lean (18 theorems, every one on no axiom; SHA-256 c18b68b29d2b6be5058d56d3a7f9e966bcde6ce099217e0f5945db4c829c42b8). On the grade ledger of the Master Codex (Islam 2026k), carried with its weakest-link join law, the Codex's rule reads a warranted root, one with a proof of its statement, at theorem grade, and an unwarranted root, the Codex's own declared axiom, at premise grade; the root of R.1 is warranted by a theorem with no axiom and no hypothesis, so on that rule it stands at theorem grade, the grade is read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA-Tongue co-location inherits the root's grade exactly (a_warranted_root_

is_theorem_grade, the_ledger_reads_the_warrant, the_grade_is_not_elected, ra_is_theorem_grade, coloc_grade_is_the_roots_grade, the_ledger_seal). Both kernels compile with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0, declare no axiom, import nothing, and pin every cone as *does not depend on any axioms*; both are carried from the master plan of the series, Seven Rows, One Root (Islam 2026j), each extended in this revision of the series by one additive section, R.2 by its section IX and R.3 by its section VII, carried alike by every paper of the series, and the paper that reads them on its row cites them for exactly what they state.

```

/-
  Triaxial_Actuation.lean · actuation is triaxial

Core Lean 4: no import, no axiom declared, no sorry, no
native_decide, and every theorem rests on no
axiom at all, not even propositional extensionality, quotients or
choice.

An actuation is a self-grounding root whose acts carry a
direction: each act has a side and a height,
and reversing the deed reverses the side and keeps the height.
  From the direction alone the kernel
constructs three axes and proves them forced:
I   The fold: reversing the deed is an involution.
II  The registration: it keeps the height and forgets the side;
it is blind to the fold, closed with
it under combination, and it is the only height-keeping map
onto the seat.
III The seat: the fold's fixed set, where the fold and the
registration both fix the act, and off
which neither does.
IV  The third axis: off the seat the fold's orbit has two acts
with one record, so the record leaves
exactly the orientation open; every reading blind to the fold
reads both alike.
V   The root through the axes: a reversed deed is still an act
and instances the root; the axes are
carried by every actuation, and they force no orientation.
VI  The bridge atom: the integers, with negation as the fold, are
an actuation.
VII The triaxial seal.
VIII The Tongue: no reading of the record returns the orientation;
the seat is the only special cut.
IX  Every root actuates: a self-grounding root is the root of an
actuation over its own acts, so the
root carries the three axes; the registration is the fold's
record and lands on the seat, where
the orientation an act and its reversal share stays open; the
even pair leaves two acts, the
orientation locks one; and a degenerate direction, one side
only, is the characterless case.
-/

namespace TriAct

/-! ## The root, self-grounding -/

/-- A self-grounding root: acts occur, and every act instances the
root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

/-! ## Actuation: a self-grounding root whose acts carry a direction
-/

/-- The direction of a deed: a side with a reversal, whose one fixed
side is the zero side. -/
structure Direction where
  Side      : Type
  rev       : Side → Side
  zero      : Side
  rev_rev   : ∀ s, rev (rev s) = s

```

```

  rev_zero : rev zero = zero
  fix_zero : ∀ s, rev s = s → s = zero

/-- An actuation: a self-grounding root whose acts are pairs of a
side and a height. -/
structure Actuation (R : Prop) where
  dir      : Direction
  Height   : Type
  anAct    : dir.Side × Height
  instances : dir.Side × Height → R

variable {R : Prop}

/-- Every actuation is a self-grounding root. -/
def Actuation.toSelfGrounding (A : Actuation R) : SelfGrounding R :=
  (A.dir.Side × A.Height, A.anAct, A.instances)

/-- The fold: reversing the deed reverses the side and keeps the
height. -/
def fold (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side
× A.Height := (A.dir.rev p.1, p.2)

/-- The registration: it keeps the height and forgets the side. -/
def reg (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side ×
A.Height := (A.dir.zero, p.2)

/-- The seat: the acts the fold fixes. -/
def OnSeat (A : Actuation R) (p : A.dir.Side × A.Height) : Prop :=
  fold A p = p

/-! ## I · the fold -/

theorem fold_involutive (A : Actuation R) (p : A.dir.Side ×
A.Height) : fold A (fold A p) = p := by
cases p with
| mk s h => show (A.dir.rev (A.dir.rev s), h) = (s, h); rw
[A.dir.rev_rev]

/-! ## II · the registration -/

/-- The registration forgets the side: an act and its reversal have
one record. -/
theorem reg_forgets_side (A : Actuation R) (p : A.dir.Side ×
A.Height) : reg A (fold A p) = reg A p := rfl

/-- The registration keeps the height. -/
theorem reg_keeps_height (A : Actuation R) (p : A.dir.Side ×
A.Height) : (reg A p).2 = p.2 := rfl

/-- The even pair is closed under combination: folding a record
changes nothing. -/
theorem fold_of_reg (A : Actuation R) (p : A.dir.Side × A.Height) :
fold A (reg A p) = reg A p := by
show (A.dir.rev A.dir.zero, p.2) = (A.dir.zero, p.2); rw
[A.dir.rev_zero]

/-- The registration is idempotent. -/
theorem reg_idempotent (A : Actuation R) (p : A.dir.Side × A.Height) :
reg A (reg A p) = reg A p := rfl

/-- THE REGISTRATION IS FORCED, NOT ELECTED: any map that keeps the
height and lands on the seat is
the registration. -/
theorem registration_is_forced (A : Actuation R) (g : A.dir.Side ×
A.Height → A.dir.Side × A.Height)
(keeps : ∀ p, (g p).2 = p.2) (lands : ∀ p, OnSeat A (g p)) : ∀ p,
g p = reg A p := by
intro p
have hs : (g p).1 = A.dir.zero := by
have hf := lands p
have h1 : A.dir.rev (g p).1 = (g p).1 := congrArg Prod.fst hf
exact A.dir.fix_zero _ h1
have hh := keeps p
cases hg : g p with
| mk s h =>
  rw [hg] at hs hh
  show (s, h) = (A.dir.zero, p.2)
  rw [show s = A.dir.zero from hs, show h = p.2 from hh]

/-! ## III · the seat -/

/-- The seat is exactly the zero side. -/
theorem seat_iff_zero_side (A : Actuation R) (p : A.dir.Side ×
A.Height) : OnSeat A p ↔ p.1 = A.dir.zero :=

```

```

(fun h => A.dir.fix_zero _ (congrArg Prod.fst h),
 fun h => by
   cases p with
   | mk s t =>
     show (A.dir.rev s, t) = (s, t)
     have hs : s = A.dir.zero := h
     rw [hs, A.dir.rev_zero])

/-- ON THE SEAT BOTH BLIND SPOTS CANCEL: the fold and the
registration both fix the act. -/
theorem seat_cancels_both (A : Actuation R) (p : A.dir.Side ×
A.Height) (h : OnSeat A p) :
  fold A p = p ∧ reg A p = p := by
  refine (h, ?_)
  have hz : p.1 = A.dir.zero := (seat_iff_zero_side A p).mp h
  cases p with
  | mk s t =>
    have hs : s = A.dir.zero := hz
    show (A.dir.zero, t) = (s, t)
    rw [hs]

/-- OFF THE SEAT BOTH STAND: neither the fold nor the registration
fixes the act. -/
theorem off_seat_both (A : Actuation R) (p : A.dir.Side × A.Height)
(h : ¬ OnSeat A p) :
  fold A p ≠ p ∧ reg A p ≠ p := by
  refine (h, fun hr => h ?_)
  have hz : p.1 = A.dir.zero := (congrArg Prod.fst hr).symm
  exact (seat_iff_zero_side A p).mpr hz

/-! ## IV · the third axis -/

/-- THE RECORD LEAVES EXACTLY THE ORIENTATION: off the seat, an act
and its reversal are two acts with
one record. -/
theorem record_leaves_the_orientation (A : Actuation R) (p :
A.dir.Side × A.Height) (h : ¬ OnSeat A p) :
  fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
(h, rfl)

/-- Every reading blind to the fold reads an act and its reversal
alike. -/
theorem even_readings_are_blind (A : Actuation R) {α : Type} (f :
A.dir.Side × A.Height → α)
(blind : ∀ p, f (fold A p) = f p) (p : A.dir.Side × A.Height) :
f (fold A p) = f p :=
blind p

/-- The registration is such a reading, and so is every reading of
the record. -/
theorem readings_of_the_record_are_blind (A : Actuation R) {α :
Type} (k : A.dir.Side × A.Height → α)
(p : A.dir.Side × A.Height) : k (reg A (fold A p)) = k (reg A p)
:= rfl

/-! ## V · the root through the axes -/

/-- A REVERSED DEED IS STILL AN ACT, AND INSTANCES THE ROOT: the
denial spends the deed. -/
theorem denial_instances_root (A : Actuation R) (p : A.dir.Side ×
A.Height) : R :=
A.instances (fold A p)

/-- The root holds on every actuation, by its act. -/
theorem root_holds (A : Actuation R) : R := A.instances A.anAct

/-- THE AXES FORCE NO ORIENTATION: off the seat both orientations
are acts, both instance the root, and
the record reads them alike; the triaxial structure leaves the one
orientation open. -/
theorem axes_force_no_orientation (A : Actuation R) (p : A.dir.Side
× A.Height) (h : ¬ OnSeat A p) :
  R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
(A.instances p, A.instances (fold A p), h, rfl)

/-! ## VI · the bridge atom: the integers are an actuation -/

theorem int_neg_neg : ∀ s : Int, -(s) = s
| Int.ofNat 0 => rfl
| Int.ofNat (n + 1) => rfl
| Int.negSucc _ => rfl

theorem self_neg_zero : ∀ v : Int, -v = v → v = 0
| Int.ofNat 0, _ => rfl

```

```

| Int.ofNat (n + 1), h => Int.noConfusion h
| Int.negSucc _, h => Int.noConfusion h

/-- The integer direction: negation, with zero its one fixed side. -/
def intDir : Direction := (Int, fun s => -s, 0, int_neg_neg, rfl,
self_neg_zero)

/-- The bridge atom: points (d, t), the fold (d, t) ↦ (-d, t), the
registration (d, t) ↦ (0, t). -/
def atom : Actuation True := (intDir, Int, ((0 : Int), (0 : Int)),
fun _ => True.intro)

/-- THE BRIDGE ATOM, AS AN ACTUATION: the fold fixes (d, t) exactly
when d = 0, the registration
leaves it unchanged exactly then, and off the seat the record leaves
the orientation. -/
theorem the_bridge_atom (d t : Int) :
  (OnSeat atom (d, t) ↔ d = 0) ∧ (reg atom (d, t) = (d, t) ↔ d = 0)
  ∧
  (d ≠ 0 → fold atom (d, t) ≠ (d, t) ∧ reg atom (fold atom (d, t))
= reg atom (d, t)) :=
(seat_iff_zero_side atom (d, t),
(fun h => (congrArg Prod.fst h).symm, fun h => by rw [h]; rfl),
fun hd => record_leaves_the_orientation atom (d, t) (fun hs => hd
((seat_iff_zero_side atom (d, t)).mp hs)))

/-- The atom is not characterless: off the seat it has two acts with
one record. -/
theorem the_atom_is_not_characterless :
  fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg
atom (fold atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0
: Int)) :=
(fun h => Int.noConfusion (congrArg Prod.fst h), rfl)

/-! ## VII · the triaxial seal -/

/-- ACTUATION IS TRIAXIAL: every actuation carries a fold that is an
involution, a registration that
keeps the height, forgets the side and is the only such map onto the
seat, and a seat where both fix
the act and off which neither does; off the seat the record leaves
exactly the orientation; every act
and its reversal instance the root; and the axes force no
orientation. -/
theorem actuation_is_triaxial (A : Actuation R) :
  (∀ p, fold A (fold A p) = p) ∧
  (∀ p, reg A (fold A p) = reg A p ∧ (reg A p).2 = p.2 ∧ fold A
(reg A p) = reg A p) ∧
  (∀ g : A.dir.Side × A.Height → A.dir.Side × A.Height,
  (∀ p, (g p).2 = p.2) → (∀ p, OnSeat A (g p)) → ∀ p, g p = reg
A p) ∧
  (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
  (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A p ≠ p) ∧
  (∀ p, ¬ OnSeat A p → R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) =
reg A p) :=
(fold_involutive A, fun p => (reg_forgets_side A p,
reg_keeps_height A p, fold_of_reg A p),
registration_is_forced A, seat_cancels_both A, off_seat_both A,
axes_force_no_orientation A)

/-- THE TRIAXIAL SEAL: actuation is triaxial, universally; the
bridge atom is an actuation; and it is
not characterless. -/
theorem the_triaxial_seal :
  (∀ (R : Prop) (A : Actuation R), (∀ p, fold A (fold A p) = p) ∧
  (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
  (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A (fold A p) = reg A
p)) ∧
  (∀ d t : Int, OnSeat atom (d, t) ↔ d = 0) ∧
  (fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg
atom (fold atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0
: Int))) :=
(fun _ => (fold_involutive A, seat_cancels_both A,
fun p h => record_leaves_the_orientation A p h),
fun d t => seat_iff_zero_side atom (d, t),
the_atom_is_not_characterless)

/-! ## VIII · the Tongue: freedom and the only special cut -/

/-- TONGUE FREEDOM: no reading of the record returns the
orientation. If a record is blind to the fold
and an orientation is flipped by it, the orientation is not a
function of the record. -/

```

```

theorem tongue_freedom {α β : Type} (τ : α → α) (ρ : α → β) (g : β →
  Bool) (d : α → Bool) (x : α)
  (blind : ρ (τ x) = ρ x) (flip : d (τ x) = !d x) : d ≠ fun y => g
  (ρ y) := by
  intro h
  have h1 : d (τ x) = g (ρ (τ x)) := congrFun h (τ x)
  have h2 : d x = g (ρ x) := congrFun h x
  rw [blind] at h1
  rw [← h2] at h1
  rw [h1] at flip
  cases hd : d x with
  | false => rw [hd] at flip; exact Bool.noConfusion flip
  | true => rw [hd] at flip; exact Bool.noConfusion flip

/-- THE ONLY SPECIAL CUT: the seat is exactly where both blind spots
  cancel; nowhere else do the fold
  and the registration both fix an act. -/
theorem the_only_special_cut (A : Actuation R) (p : A.dir.Side ×
  A.Height) :
  (fold A p = p ∧ reg A p = p) ↔ OnSeat A p :=
  (fun h => h.1, fun h => seat_cancels_both A p h)

/-! ## IX · every root actuates: the root carries the three axes -/

/-- THE REGISTRATION IS THE FOLD'S RECORD: it reads an act and its
  reversal alike and keeps the height, and two
  acts have one record exactly when they have one height. -/
theorem registration_is_the_folds_record (A : Actuation R) (p q :
  A.dir.Side × A.Height) :
  reg A (fold A p) = reg A p ∧ (reg A p = reg A q ↔ p.2 = q.2) :=
  (rfl, (fun h => show (reg A p).2 = (reg A q).2 from congrArg
  Prod.snd h, fun h => by
  cases p with
  | mk s t =>
  cases q with
  | mk s' t' =>
  have ht : t = t' := h
  show (A.dir.zero, t) = (A.dir.zero, t')
  rw [ht]))

/-- THE REGISTRATION LANDS ON THE SEAT: every record is a seat
  point. -/
theorem reg_lands_on_seat (A : Actuation R) (p : A.dir.Side ×
  A.Height) : OnSeat A (reg A p) :=
  fold_of_reg A p

/-- THE ORIENTATION IS OPEN AT THE SEAT: off the seat, an act and
  its reversal land on one seat point, the record
  of both; at that place the record has left exactly the orientation
  open. -/
theorem open_at_the_seat (A : Actuation R) (p : A.dir.Side ×
  A.Height) (h : ¬ OnSeat A p) :
  OnSeat A (reg A p) ∧ reg A (fold A p) = reg A p ∧ fold A p ≠ p :=
  (reg_lands_on_seat A p, rfl, h)

/-- TWO AXES LEAVE TWO: off the seat, every reading blind to the
  fold, the record among them, reads an act and
  its reversal alike: two acts, one reading. -/
theorem even_pair_leaves_two (A : Actuation R) (p : A.dir.Side ×
  A.Height) (h : ¬ OnSeat A p) {α : Type}
  (f : A.dir.Side × A.Height → α) (blind : ∀ q, f (fold A q) = f
  q) :
  fold A p ≠ p ∧ f (fold A p) = f p ∧ reg A (fold A p) = reg A p :=
  (h, blind p, rfl)

/-- THREE AXES LOCK ONE: the record together with the orientation,
  the side, determines the act. -/
theorem orientation_locks_the_act (A : Actuation R) (p q :
  A.dir.Side × A.Height)
  (hr : reg A p = reg A q) (hs : p.1 = q.1) : p = q := by
  cases p with
  | mk s t =>
  cases q with
  | mk s' t' =>
  have ht : t = t' := congrArg Prod.snd hr
  have hs' : s = s' := hs
  rw [hs', ht]

/-- A degenerate direction has the zero side and no other. -/
def Direction.Degenerate (D : Direction) : Prop := ∀ s, s = D.zero

/-- THE CHARACTERLESS CASE: over a degenerate direction every act is
  on the seat, and the fold and the

```

```

registration are both the identity: one scalar, no fold, no seat to
  lock. -/
theorem degenerate_is_characterless (A : Actuation R) (h :
  A.dir.Degenerate) (p : A.dir.Side × A.Height) :
  OnSeat A p ∧ fold A p = p ∧ reg A p = p :=
  have hs : OnSeat A p := (seat_iff_zero_side A p).mpr (h p.1)
  (hs, seat_cancels_both A p hs)

/-- The integer direction is not degenerate. -/
theorem intDir_not_degenerate : ¬ intDir.Degenerate := fun h =>
  have h0 : (1 : Int) = 0 := h (1 : Int)
  absurd h0 (by decide)

/-- EVERY ROOT ACTUATES: a self-grounding root is the root of the
  actuation over its own acts, with the integer
  direction; the root's deeds sit at the zero side and instance the
  root as before. -/
def SelfGrounding.toActuation (S : SelfGrounding R) : Actuation R :=
  (intDir, S.act, ((0 : Int), S.anAct), fun p => S.instances p.2)

theorem every_root_actuates (S : SelfGrounding R) :
  (∀ a : S.act, OnSeat S.toActuation ((0 : Int), a)) ∧
  (∀ a : S.act, ¬ OnSeat S.toActuation ((1 : Int), a)) ∧
  ¬ S.toActuation.dir.Degenerate :=
  (fun _ => rfl,
  fun a h =>
  have h0 : (1 : Int) = 0 := (seat_iff_zero_side S.toActuation
  ((1 : Int), a)).mp h
  absurd h0 (by decide),
  intDir_not_degenerate)

/-- THE ROOT CARRIES THREE AXES: for every self-grounding root, its
  actuation carries the fold, an involution; the
  registration, keeping the height, forgetting the side, the only
  height-keeping map onto the seat; and the seat,
  where both fix the act; and it is not characterless: at every height
  an act off the seat, its reversal a second
  act with the same record, both instantiating the root, the orientation
  between them forced by nothing. -/
theorem the_root_carries_three_axes (S : SelfGrounding R) :
  (∀ p, fold S.toActuation (fold S.toActuation p) = p) ∧
  (∀ p, reg S.toActuation (fold S.toActuation p) = reg
  S.toActuation p ∧ (reg S.toActuation p).2 = p.2) ∧
  (∀ g : Int × S.act → Int × S.act, (∀ p, (g p).2 = p.2) → (∀ p,
  OnSeat S.toActuation (g p)) →
  ∀ p, g p = reg S.toActuation p) ∧
  (∀ p, OnSeat S.toActuation p → fold S.toActuation p = p ∧ reg
  S.toActuation p = p) ∧
  (∀ a : S.act, fold S.toActuation ((1 : Int), a) ≠ ((1 : Int), a) ∧
  reg S.toActuation (fold S.toActuation ((1 : Int), a)) = reg
  S.toActuation ((1 : Int), a)) ∧
  (∀ p, ¬ OnSeat S.toActuation p →
  R A R ∧ fold S.toActuation p ≠ p ∧ reg S.toActuation (fold
  S.toActuation p) = reg S.toActuation p) :=
  (fold_involutive S.toActuation,
  fun p => (reg_forgets_side S.toActuation p, reg_keeps_height
  S.toActuation p),
  registration_is_forced S.toActuation,
  seat_cancels_both S.toActuation,
  fun a => record_leaves_the_orientation S.toActuation ((1 : Int),
  a) ((every_root_actuates S).2.1 a),
  axes_force_no_orientation S.toActuation)

end TriAct

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-- info: 'TriAct.fold_involutive' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_involutive
/-- info: 'TriAct.reg_forgets_side' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_forgets_side
/-- info: 'TriAct.reg_keeps_height' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_keeps_height
/-- info: 'TriAct.fold_of_reg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_of_reg
/-- info: 'TriAct.reg_idempotent' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_idempotent
/-- info: 'TriAct.registration_is_forced' does not depend on any
  axioms -/
#guard_msgs in #print axioms TriAct.registration_is_forced
/-- info: 'TriAct.seat_iff_zero_side' does not depend on any axioms
  -/
#guard_msgs in #print axioms TriAct.seat_iff_zero_side
/-- info: 'TriAct.seat_cancels_both' does not depend on any axioms
  -/

```

```

#guard_msgs in #print axioms TriAct.seat_cancels_both
/-- info: 'TriAct.off_seat_both' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.off_seat_both
/-- info: 'TriAct.record_leaves_the_orientation' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.record_leaves_the_orientation
/-- info: 'TriAct.even_readings_are_blind' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.even_readings_are_blind
/-- info: 'TriAct.readings_of_the_record_are_blind' does not depend
on any axioms -/
#guard_msgs in #print axioms TriAct.readings_of_the_record_are_blind
/-- info: 'TriAct.denial_instances_root' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.denial_instances_root
/-- info: 'TriAct.root_holds' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.root_holds
/-- info: 'TriAct.axes_force_no_orientation' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.axes_force_no_orientation
/-- info: 'TriAct.int_neg_neg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.int_neg_neg
/-- info: 'TriAct.self_neg_zero' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.self_neg_zero
/-- info: 'TriAct.the_bridge_atom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_bridge_atom
/-- info: 'TriAct.the_atom_is_not_characterless' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.the_atom_is_not_characterless
/-- info: 'TriAct.actuation_is_triaxial' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.actuation_is_triaxial
/-- info: 'TriAct.the_triaxial_seal' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.the_triaxial_seal
/-- info: 'TriAct.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.tongue_freedom
/-- info: 'TriAct.the_only_special_cut' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.the_only_special_cut
/-- info: 'TriAct.registration_is_the_folds_record' does not depend
on any axioms -/
#guard_msgs in #print axioms TriAct.registration_is_the_folds_record
/-- info: 'TriAct.reg_lands_on_seat' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.reg_lands_on_seat
/-- info: 'TriAct.open_at_the_seat' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.open_at_the_seat
/-- info: 'TriAct.even_pair_leaves_two' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.even_pair_leaves_two
/-- info: 'TriAct.orientation_locks_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.orientation_locks_the_act
/-- info: 'TriAct.degenerate_is_characterless' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.degenerate_is_characterless
/-- info: 'TriAct.intDir_not_degenerate' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.intDir_not_degenerate
/-- info: 'TriAct.every_root_actuates' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.every_root_actuates
/-- info: 'TriAct.the_root_carries_three_axes' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.the_root_carries_three_axes

/-
Root_Grade_Ledger.lean · the root at theorem grade, and RA-Tongue
co-location with it

Core Lean 4: no import, no axiom declared, no sorry, no
native_decide, and every theorem rests on no
axiom at all, not even propositional extensionality, quotients or
choice.

The floor Codex (Master Codex v5.1.0, Codex.lean) grades every
claim on one ledger and joins claims
at the weakest link. It declares the root as `axiom RA` and pins
`raClaim.grade = .premise`; its
co-location claim, the root joined with the wall, therefore
inherits `.premise`. This kernel carries
the same ledger and the same join law with no axiom declared,
warrants the root by a theorem with no

```

```

axiom and no hypothesis, and proves that the co-location claim
then stands at theorem grade.
I The grade ledger and the weakest-link join, as in the Codex.
II The root, a theorem: the constructed domain of TOE_Zero.lean,
warranted on no axiom.
III The wall, a theorem: the Codex's K4 frame, its target not a
function of its record, proved on
no axiom through Tongue freedom.
IV The claims: the root at theorem grade, the wall at theorem
grade, and their co-location.
V Why the Codex read premise: a join's grade is its weakest
link, and co-location's grade is the
root's grade exactly; with the root at theorem grade,
co-location is a theorem.
VI The ledger seal.
VII The grade read from the warrant, as the Codex reads it: a
root with a warrant, a proof of its
statement, is theorem grade; a root posited without one, the
Codex's `axiom RA`, is premise grade;
the two claims of section IV carry exactly the grades this
rule reads.
-/

namespace RGL

/-! ## I · the grade ledger, as in the Codex -/

inductive Grade : Type
| premise | corroboration | operational | structural | engineering
| conditional | theoremConditional | analytic | theorem
deriving DecidableEq

def Grade.rank : Grade → Nat
| .premise => 0 | .corroboration => 1 | .operational => 2 |
.structural => 3
| .engineering => 4 | .conditional => 5 | .theoremConditional => 6
| .analytic => 7 | .theorem => 8

def Grade.weakest (a b : Grade) : Grade := if a.rank ≤ b.rank then a
else b

/-- A claim carries its statement, its grade and its warrant, a
proof of the statement. -/
structure Claim : Type where
stmt : Prop
grade : Grade
warrant : stmt

/-- Two claims join at their weakest link. -/
def Claim.join (c₁ c₂ : Claim) : Claim where
stmt := c₁.stmt ∧ c₂.stmt
grade := Grade.weakest c₁.grade c₂.grade
warrant := (c₁.warrant, c₂.warrant)

theorem join_grade (c₁ c₂ : Claim) : (Claim.join c₁ c₂).grade =
Grade.weakest c₁.grade c₂.grade := rfl

/-! ## II · the root, a theorem (the constructed domain of
TOE_Zero.lean) -/

structure SelfGrounding (R : Prop) where
Act : Type
anAct : Act
instances : Act → R

theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
G.instances G.anAct

def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ => show (0 : Int)
< 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM: no axiom, no
hypothesis. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable
ra₀

/-! ## III · the wall, a theorem (the Codex's K4 frame) -/

/-- Tongue freedom: if a record is blind to the fold and an
orientation is flipped by it, the
orientation is not a function of the record. -/

```

```

theorem tongue_freedom {α β : Type} (τ : α → α) (ρ : α → β) (g : β →
  Bool) (d : α → Bool) (x : α)
  (blind : ρ (τ x) = ρ x) (flip : d (τ x) = !d x) : d ≠ fun y => g
  (ρ y) := by
  intro h
  have h1 : d (τ x) = g (ρ (τ x)) := congrFun h (τ x)
  have h2 : d x = g (ρ x) := congrFun h x
  rw [blind] at h1
  rw [← h2] at h1
  rw [h1] at flip
  cases hd : d x with
  | false => rw [hd] at flip; exact Bool.noConfusion flip
  | true => rw [hd] at flip; exact Bool.noConfusion flip

namespace K4

def val : Nat → Nat
| 0 => 11 | 1 => 851 | 2 => 13 | 3 => 1273 | 4 => 17 | 5 => 437
| 6 => 19 | 7 => 2119 | 8 => 23 | 9 => 1703 | 10 => 29 | 11 => 869
| _ => 0

def tau : Nat → Nat := fun i =>
  if i < 12 then (if i % 2 = 0 then i + 1 else i - 1) else i

def target : Nat → Bool := fun i => i % 2 == 0

def rho : Nat → Nat := fun i =>
  (val i % 2) * 1000 + (val i % 3) * 100 + (val i % 5) * 10 + (val i
  % 7)

/-- The record is blind to the fold at the first pair. -/
theorem register_even_at_zero : rho (tau 0) = rho 0 := by decide

/-- The target is flipped by the fold at the first pair. -/
theorem target_odd_at_zero : target (tau 0) = !target 0 := by decide

/-- THE WALL: no reading of the record returns the target. -/
theorem wall (g : Nat → Bool) : target ≠ fun y => g (rho y) :=
  tongue_freedom tau rho g target 0 register_even_at_zero
  target_odd_at_zero

end K4

/-! ## IV · the claims -/

/-- The root at theorem grade, warranted by a theorem with no axiom
  and no hypothesis. -/
def raClaim0 : Claim where
  stmt := RA0
  grade := .theorem
  warrant := root_on_the_constructed_domain

/-- The wall at theorem grade, as in the Codex. -/
def wallClaim : Claim where
  stmt := ∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)
  grade := .theorem
  warrant := K4.wall

/-- RA-Tongue co-location: the root joined with the wall, as in the
  Codex. -/
def colocClaim0 : Claim := Claim.join raClaim0 wallClaim

/-- The Codex's root, graded as the Codex grades it, for comparison:
  the same statement form, at
  premise grade. -/
def raClaimPremise : Claim where
  stmt := RA0
  grade := .premise
  warrant := root_on_the_constructed_domain

/-! ## V · why the Codex read premise, and what co-location is now -/

/-- A join with a theorem-grade claim has exactly the other claim's
  grade. -/
theorem weakest_with_theorem (g : Grade) : Grade.weakest g .theorem
  = g := by
  cases g <|> rfl

/-- CO-LOCATION'S GRADE IS THE ROOT'S GRADE, EXACTLY: the wall is a
  theorem, so the join reads the
  root's grade and nothing else. -/
theorem coloc_grade_is_the_roots_grade (r : Claim) :
  (Claim.join r wallClaim).grade = r.grade := by
  show Grade.weakest r.grade Grade.theorem = r.grade

```

```

exact weakest_with_theorem r.grade

/-- WHY THE CODEX READ PREMISE: with the root labelled premise, the
  co-location claim is premise. -/
theorem premise_root_gives_premise_coloc : (Claim.join
  raClaimPremise wallClaim).grade = .premise := rfl

/-- THE ROOT IS A THEOREM-GRADE CLAIM. -/
theorem ra_is_theorem_grade : raClaim0.grade = .theorem := rfl

/-- RA-TONGUE CO-LOCATION IS A THEOREM-GRADE CLAIM. -/
theorem coloc_is_theorem_grade : colocClaim0.grade = .theorem := rfl

/-- The co-location claim's statement holds: the root and the wall,
  both proved. -/
theorem coloc_holds : colocClaim0.stmt := colocClaim0.warrant

/-! ## VI · the ledger seal -/

/-- THE LEDGER SEAL: the root is a theorem on no axiom and no
  hypothesis; the wall is a theorem; their
  co-location holds and stands at theorem grade; and co-location's
  grade is the root's grade exactly,
  which is why the Codex, labelling the root premise, read premise. -/
theorem the_ledger_seal :
  RA0 ∧ (∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)) ∧
  colocClaim0.stmt ∧ colocClaim0.grade = .theorem ∧
  (∀ r : Claim, (Claim.join r wallClaim).grade = r.grade) ∧
  (Claim.join raClaimPremise wallClaim).grade = .premise :=
  (root_on_the_constructed_domain, K4.wall, coloc_holds,
  coloc_is_theorem_grade, premise_root_gives_premise_coloc)

/-! ## VII · the grade read from the warrant, as the Codex reads it
  -/

/-- The evidence a ledger entry carries for a statement: posited
  with no warrant, or warranted by a proof. -/
inductive Evidence (P : Prop) : Type
| posited
| warranted (w : P)

/-- THE CODEX'S RULE FOR THE ROOT'S GRADE: evidence at theorem grade
  is a proof with its axioms printed. A root
  that comes with a warrant, a proof of its statement, is theorem
  grade; a root posited with none, as the Codex
  declares 'axiom RA', is premise grade. -/
def rootGrade : Evidence RA0 → Grade
| .warranted _ => .theorem
| .posited => .premise

/-- A WARRANTED ROOT IS THEOREM GRADE: the root of section II
  carries its warrant, so the rule reads theorem. -/
theorem a_warranted_root_is_theorem_grade :
  rootGrade (.warranted root_on_the_constructed_domain) = .theorem
  := rfl

/-- AN UNWARRANTED ROOT IS PREMISE GRADE: with no warrant the rule
  reads premise, the Codex's own reading of
  its declared axiom. -/
theorem an_unwarranted_root_is_premise_grade : rootGrade .posited =
  .premise := rfl

/-- THE LEDGER READS THE WARRANT: the root claim of section IV
  carries the grade the rule reads from its warrant,
  and the comparison claim carries the grade the rule reads from none.
  -/
theorem the_ledger_reads_the_warrant :
  raClaim0.grade = rootGrade (.warranted raClaim0.warrant) ∧
  raClaimPremise.grade = rootGrade .posited :=
  (rfl, rfl)

/-- THE GRADE IS NOT ELECTED: the rule reads theorem from a warrant
  and premise from none, never the other way;
  whatever label an entry carries, the warranted root's grade under
  the rule is theorem. -/
theorem the_grade_is_not_elected :
  (∀ w : RA0, rootGrade (.warranted w) = .theorem) ∧ rootGrade
  .posited ≠ .theorem ∧
  (∀ w : RA0, rootGrade (.warranted w) ≠ rootGrade .posited) :=
  (fun _ => rfl, fun h => Grade.noConfusion h, fun _ h =>
  Grade.noConfusion h)

end RGL

```

```

/! ## Cones, pinned as printed: every theorem on no axiom -/
/-- info: 'RGL.join_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.join_grade
/-- info: 'RGL.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.seated_undeniable
/-- info: 'RGL.root_on_the_constructed_domain' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.root_on_the_constructed_domain
/-- info: 'RGL.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.tongue_freedom
/-- info: 'RGL.K4.register_even_at_zero' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.K4.register_even_at_zero
/-- info: 'RGL.K4.target_odd_at_zero' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.K4.target_odd_at_zero
/-- info: 'RGL.K4.wall' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.K4.wall
/-- info: 'RGL.weakest_with_theorem' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.weakest_with_theorem
/-- info: 'RGL.coloc_grade_is_the_roots_grade' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.coloc_grade_is_the_roots_grade
/-- info: 'RGL.premise_root_gives_premise_coloc' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.premise_root_gives_premise_coloc
/-- info: 'RGL.ra_is_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.ra_is_theorem_grade
/-- info: 'RGL.coloc_is_theorem_grade' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.coloc_is_theorem_grade
/-- info: 'RGL.coloc_holds' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.coloc_holds
/-- info: 'RGL.the_ledger_seal' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.the_ledger_seal
/-- info: 'RGL.a_warranted_root_is_theorem_grade' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.a_warranted_root_is_theorem_grade
/-- info: 'RGL.an_unwarranted_root_is_premise_grade' does not depend
on any axioms -/
#guard_msgs in #print axioms
RGL.an_unwarranted_root_is_premise_grade
/-- info: 'RGL.the_ledger_reads_the_warrant' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.the_ledger_reads_the_warrant
/-- info: 'RGL.the_grade_is_not_elected' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.the_grade_is_not_elected

```

AUTHOR'S PROVENANCE AND METHOD DISCLOSURE

The arc was found within Trisduction, the author's verification architecture, through its operating system PhysOS^T (Islam 2026c), and compiled with the assistance of an AI scribe; every theorem stands on its compiled proof, reproducible by any reader from the appendices. The author declares no competing interests. Correspondence: islammm@alumni.iu.edu.

Nature of Root Axiom

The root is triaxial, and that is the force of the cut. A characterless existence has no triaxiality: one scalar, no fold, no seat, nothing to lock. The root, to exist is to actuate, carries three axes, and they are not chosen: for every actuation, the direction of its deed gives the fold, the registration and the seat, and the registration is the only height-keeping map onto the seat. The deed is the direction: a denial spends the deed and instances the root. The registration is the fold's record: it keeps the height and forgets the side, and the two even axes, symmetry and record, are closed under combination and blind. The seat is the fixed set, where the two blind spots

cancel, and the third axis is the one orientation the even pair cannot supply. Two axes never determine the point; three do. The seven rows are one cut because each is an involution with a seat and one missing orientation. The root makes the third axis native as a place: the seat, and the one orientation still open there; the axes themselves force no orientation. Which way that orientation points is the value, and it is the root read on that seat. The root alone still forces no value.

Key to the statement, in the words of its receipts. The three axes are the two even axes, the fold's symmetry and the record, and the orientation, the side of a point of the actuation, which the even pair cannot supply; the fold, the registration and the seat are what the direction of the deed gives, and the seat is where the side is read. The point two axes never determine is a point off the seat: there the even pair does not tell a point from its reversal, and the record with the side locks the point; on the seat both blind spots cancel and nothing is open there to determine. A point of the actuation is a side and a height, and every point instances the root: the points are the actuation's deeds, and the deed is the direction in the sense that its side is its direction, reversed by the fold. The characterless case, one scalar, is a degenerate direction: the fold is the identity and the seat is total, which is to say no fold and no seat to lock. The axes are not chosen in the sense that, given the direction, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat; the root's own actuation takes the integer direction, under which it is not characterless, while a degenerate direction over the same deeds would be (every_root_actuates, degenerate_is_characterless). The root read on that seat is the root read on the row at the seat point where a point and its reversal land with one record; that the orientation between them is the row's value bit is the paper's structural reading, a second statement beside its kernel's theorems, and no kernel states a map between the two. Each row carries the actuation's seat as the root of an actuation over its own deeds; a row's own seat, where its kernel states one, is a further statement of that kernel, named in the last row of the table, and on P versus NP it is vacant. A deed of the root is an inhabitant of its type of acts, which the root kernel's own comments call an act; in the root's actuation those deeds sit at the zero side, on the seat, and the points off the seat are the sides the direction supplies; the act, unqualified, is existence read on computation, the field the closing theorem consumes.

Receipts, each cited for what its statement says; R.1, R.2 and R.3 are the three kernels of Appendix R, every theorem of each on no axiom; the three theorems of the Master Codex are cited, not carried.

Clause of the statement	Receipt
the root carries three axes, not chosen: every self-grounding root is the root of an actuation over its own deeds, and for every actuation, given the direction of its deed, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat	every_root_actuates, the_root_carries_three_axes, actuation_is_tri_axial, registration_is_forced (Appendix R.2)

Clause of the statement	Receipt
the registration is the only height-keeping map onto the seat	registration_is_forced (R.2)
the axes force no orientation	axes_force_no_orientation (R.2)
the deed is the direction: a deed of the actuation is a point, a side and a height, its side its direction, reversed by the fold; a reversed point is still a deed and instances the root; a denial spends the deed and instances the root	fold, denial_instances_root (R.2); denial_reenacts_root, denial_instantiates (R.1)
the registration is the fold's record: it keeps the height, forgets the side and reads a point and its reversal alike; the fold is an involution; the even pair is closed under combination	registration_is_the_folds_record, reg_forgets_side, reg_keeps_height, fold_involutive, fold_of_reg (R.2)
the two even axes are blind: no reading of the record returns the orientation; symmetry keyless, the line property keyed	readings_of_the_record_are_blind, tongue_freedom (R.2); symmetry_is_keyless, line_property_is_keyed, exactly_one_stands (Islam 2026k, Master Codex v5.1.0, commit 5e55fca, cited for their statements, not carried; that Codex declares its root as an axiom, this paper's kernels declare none)
the seat is the fixed set, where both blind spots cancel; off the seat both stand	seat_iff_zero_side, seat_cancels_both, off_seat_both, the_only_special_cut (R.2)
the registration lands on the seat; off the seat a point and its reversal land on one seat point with one record, and the orientation is what stays open there; every reading of the record reads both alike	reg_lands_on_seat, open_at_the_seat, record_leaves_the_orientation, readings_of_the_record_are_blind (R.2)
the characterless case, no fold and no seat to lock: over a degenerate direction, one side only, the fold is the identity and every point is on the seat; the root's actuation is not degenerate	degenerate_is_characterless, intDir_not_degenerate, every_root_actuates (R.2)
the bridge atom is an actuation, and not characterless	the_bridge_atom, the_atom_is_not_characterless, the_triaxial_seal (R.2)
the scalar ground in the algebra, a third reading beside the actuation's: the Return, the line the conjugation fixes and the twelve gates, bound with the root	the_return, the_line_is_the_fixed_set, twenty_four_units_twelve_gates, the_class_equation, the_master_seal (R.1)

Clause of the statement	Receipt
the root at theorem grade on the Codex's ledger rule, the grade read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA-Tongue co-location inheriting it	a_warranted_root_is_theorem_grade, the_ledger_reads_the_warrant, the_grade_is_not_elected, ra_is_theorem_grade, coloc_grade_is_the_roots_grade, the_ledger_seal (R.3)
two axes never determine the point; three do: off the seat the even pair does not tell a point from its reversal, and the record with the side, which carries the orientation, locks the point; on the seat both blind spots cancel and nothing is open to determine	even_pair_leaves_two, orientation_locks_the_act, seat_cancels_both (R.2); in its own coordinates, for independent axes in $\mathbb{F}_2^3$, two axes leave two, three axes lock one (this paper's kernel), a second statement
the root alone forces no value; the root read on computation is the separation	undeniable_root_forces_no_value; root_on_computation_is_sep (this paper's kernels)
the seven rows are one cut: each row's root is an actuation, so each carries an involution, the fold, with a seat and one missing orientation	every_root_actuates, the_root_carries_three_axes (R.2), stated for every self-grounding root and read at the constructed root ra_0 of the root-laws kernel (PNP_Root.lean), whose structure repeats R.2's SelfGrounding field for field; this row's own involution, complementation, fixes no bit: its seat is vacant (locus_vanishes, a1_no_seat, seat_vacant), and its own open bit is the keyed separation bit, read by no record of the vanished seat (separation_keyed, vanished_seat_reads_no_machine); the actuation's seat and open orientation are the root's, over its deeds, a second statement; the row ledger of Islam (2026i, Book III), cited

Clause of the statement	Receipt